



Sailboat
This is

UML y Patrones

2ª Edición

Larman

Este manual presenta una introducción práctica al análisis y diseño orientado a objetos (A/D OO) y a aspectos relacionados de desarrollo iterativo.

El objetivo principal del libro es ayudar a los estudiantes y a los desarrolladores a crear diseños orientados a objetos mediante la aplicación de un conjunto de principios y heurísticas explicables.

El libro va dirigido a desarrolladores y estudiantes con experiencia en un lenguaje de programación orientado a objetos, pero con poca experiencia en el análisis y diseño orientados a objetos, así como a estudiantes de informática o cursos de ingeniería del software que estudien la tecnología de objetos y a aquellas personas que, familiarizadas con el A/D OO, quieran aprender la notación UML, aplicar patrones, o mejorar y perfeccionar sus habilidades de análisis y diseño.

Esta nueva edición presenta mejoras importantes respecto a la anterior, tales como:

- Se actualizan los casos de uso para seguir el enfoque de [Cockburn01] que ha tenido bastante aceptación.
- Se utiliza el conocido Proceso Unificado (*Unified Process, UP*) como ejemplo de proceso iterativo. Por tanto, todos los artefactos se nombran de acuerdo con la terminología del UP, como Modelo de Dominio.
- Tratamiento actualizado de los patrones de diseño.
- Introducción al análisis arquitectural.
- Introducción de Variaciones Protegidas como una extensión GRASP.
- Actualización de la notación UML más reciente.
- Discusión de algunos aspectos prácticos de dibujo utilizando pizarras o herramientas CASE para UML.



LibroSite es una página web asociada al libro, con una gran variedad de recursos y material adicional tanto para los profesores como para estudiantes. Apoyos, docencia, ejercicios de autocontrol, enlaces relacionados, material de investigación, etc., hacen de LibroSite el complemento académico perfecto para este libro.

www.librosite.net/larman

PEARSON
Educación

www.pearsoneducacion.com

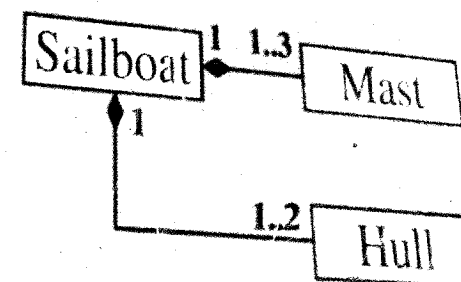
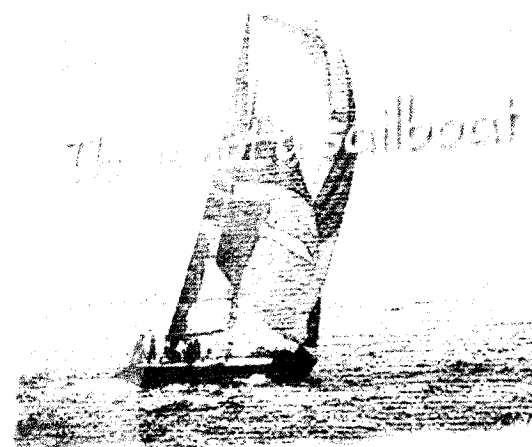
ISBN 84-205-3438-2



2ª Edición

UML y Patrones

Una introducción



This is not a sailboat

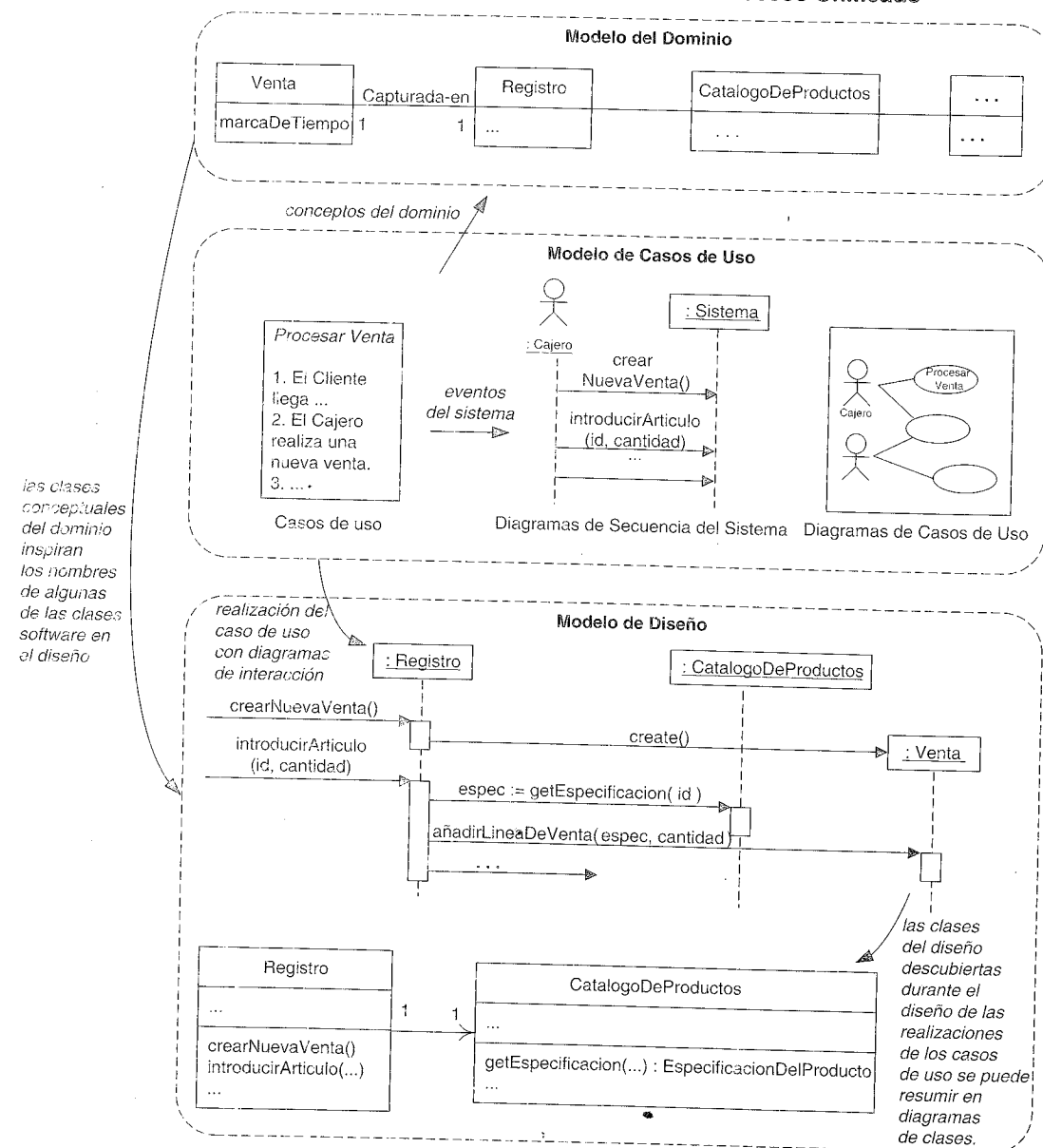


Craig Larman

Muestra de los artefactos del Proceso Unificado y evolución temporal (c-comenzar; r-refinar)

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Muestra de las relaciones entre los artefactos del Proceso Unificado



Patrones de Software Generales para la Asignación de Responsabilidades (GRASP)

Patrón	Descripción
Experto en Información	¿Un principio general del diseño de objetos y la asignación de responsabilidades? Asigne una responsabilidad al experto en información, —la clase que tiene la información necesaria para llevar a cabo la responsabilidad.
Creador	¿Quién crea? (Nótese que la Factoría es una solución alternativa frecuente.) Asigne a la clase B la responsabilidad de crear una instancia de la clase A si se cumple alguno de los puntos siguientes: 1. B contiene a A 2. B agrega a A 3. B tiene los datos de inicialización de A 4. B registra a A 5. B utiliza estrechamente a A
Controlador	¿Quién gestiona un evento del sistema? Asigne la responsabilidad de gestionar un mensaje de un evento del sistema a una clase que represente una de estas opciones: 1. Representa el sistema global, dispositivo o un subsistema (controlador de fachada). 2. Representa un escenario de caso de uso en el que tiene lugar el evento del sistema (controlador de caso de uso o sesión).
Bajo Acoplamiento (evaluativo)	¿Cómo dar soporte a las bajas dependencias y al incremento de la reutilización? Asigne responsabilidades de manera que el acoplamiento (innecesario) se mantenga bajo.
Alta Cohesión (evaluativo)	¿Cómo mantener manejable la complejidad? Asigne responsabilidades de manera que la cohesión permanezca alta.
Polimorfismo	¿Quién es el responsable cuando el comportamiento varía en función del tipo? Cuando las alternativas o comportamientos relacionados varían según el tipo (clase), asigne la responsabilidad del comportamiento —utilizando operaciones polimórficas— a los tipos para los que varía el comportamiento.
Fabricación Pura	¿Quién es el responsable cuando está desesperado, y no quiere violar los principios de alta cohesión y bajo acoplamiento? Asigne un conjunto altamente cohesivo de responsabilidades a una clase de “comportamiento” artificial o de conveniencia que no representa un concepto del dominio del problema —algo inventado—, para dar soporte a la alta cohesión, bajo acoplamiento y la reutilización.
Indirección	¿Cómo asignar responsabilidades para evitar el acoplamiento directo? Asigne la responsabilidad a un objeto intermedio para mediar entre otros componentes o servicios, de manera que no se acoplan directamente.
Variaciones Protegidas	¿Cómo asignar responsabilidades a los objetos, subsistemas, y sistemas de manera que las variaciones o inestabilidad en estos elementos no influya de manera no deseable en otros elementos? Identifique los puntos de variaciones predecibles o inestabilidad; asigne las responsabilidades para crear una “interfaz” estable alrededor de ellos.

24045

PRÉSTAMO 48 HORAS

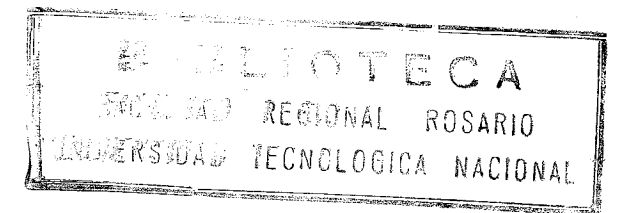
Fecha de devolución	
07/04/08.	21 ABR 2008
14/04/08	28/05/09
28/05/08	19/06/09
05/05/08	10/08/09
13/05/08	13/08/09
01 SEP 2008	18/08/09
01 DIC 2008	7 DEC 2008
15/02/08	7/9/09
07 MAR 2009	25/09/09
27/3/09	29/09/09
30/3/09	14 NOV 2009
06/04/09	19/11/09
14/04/09	17 MAR 2010
17 ABR 2009	
Evite suspensiones : devuelva en término	

UML Y PATRONES

UNA INTRODUCCIÓN AL ANÁLISIS
Y DISEÑO ORIENTADO A OBJETOS
Y AL PROCESO UNIFICADO

Segunda edición

24045



UML Y PATRONES

UNA INTRODUCCIÓN AL ANÁLISIS
Y DISEÑO ORIENTADO A OBJETOS
Y AL PROCESO UNIFICADO

Segunda edición

UTN Facultad Regional Rosario Biblioteca

Autor: Larman, C

Uml Y Patrones

Email: biblioteca@frro.utn.edu.ar

Tel.: 4481871 Int. 111



24045

CRAIG LARMAN

Traducción:

Begoña Moros Valle

Universidad de Murcia

Supervisión de la traducción y revisión técnica:

Jesús García Molina

Universidad de Murcia

BIBLIOTECA	
Fecha:	08-03-05
Registro:	7167
Inventario:	24045
Sig. Top:	681.3 6294
Ejemplar:	1 (uso) 2a. ed.
Proc.:	compuer

PEARSON
Prentice
Hall

Madrid • México • Santafé de Bogotá • Buenos Aires • Caracas • Lima
Montevideo • San Juan • San José • Santiago • São Paulo • White Plains

LARMAN, C.

UML Y PATRONES. Una introducción al análisis y diseño orientado a objetos y al proceso unificado. Segunda edición

PEARSON EDUCACIÓN, S.A., Madrid, 2003

ISBN: 84-205-3438-2

Materia: Informática 681.3

Formato 195 × 250

Páginas: 624

Todos los derechos reservados.

Queda prohibida, salvo excepción prevista en la ley, cualquier forma de reproducción, distribución, comunicación pública y transformación de esta obra sin contar con autorización de los titulares de propiedad intelectual. La infracción de los derechos mencionados puede ser constitutiva de delito contra la propiedad intelectual (*arts. 270 y sgts. Código Penal*).

DERECHOS RESERVADOS

© 2003 respecto a la segunda edición en español por:

PEARSON EDUCACIÓN, S.A.

Ribera del Loira, 28

28042 Madrid (España)

LARMAN, C.

UML Y PATRONES. Una introducción al análisis y diseño orientado a objetos y al proceso unificado. Segunda edición

ISBN: 84-205-3438-2

Depósito Legal: M.23.390-2004

Última reimpresión, 2004

PRENTICE HALL es un sello editorial autorizado de PEARSON EDUCACIÓN, S.A.

Traducido de:

APPLYING UML AND PATTERNS: An Introduction to Object-Oriented Analysis and Design and the Unified Process, Second edition, by Craig Larman.

Published by Pearson Education, Inc. Publishing as Prentice Hall PTR.

Prentice Hall, Inc.

© 2002 All rights reserved.

ISBN: 0-13-092569-1

Edición en español:

Equipo editorial:

Editor: David Fayerman Aragón

Técnico editorial: Ana Isabel García

Equipo de producción:

Director: José Antonio Clares

Técnico: José Antonio Hernán

Diseño de cubierta: equipo de diseño de PEARSON EDUCACIÓN, S.A.

Composición: COPIBOOK, S.L.

Impreso por: LVEL, S. A.

IMPRESO EN ESPAÑA - PRINTED IN SPAIN

Para Julie

Sin su apoyo, esto no habría sido posible.

Para Haley y Hannah

Gracias por aguantar a un papá distraído, ¡otra vez!

RESUMEN DE CONTENIDO

Parte 1: Introducción

1. Análisis y diseño orientado a objetos	3
2. Desarrollo iterativo y el Proceso Unificado	13
3. Caso de estudio: el sistema de punto de venta NuevaEra	27

Parte 2: Inicio

4. Inicio	33
5. Comprensión de los requisitos	39
6. Modelo de Casos de Uso: escritura de requisitos en contexto	43
7. Identificación de otros requisitos	79
8. Del inicio a la elaboración	103

Parte 3: Elaboración en la Iteración 1

9. Modelo de Casos de Uso: representación de los diagramas de secuencia del sistema	113
10. Modelo del Dominio: visualización de conceptos	121
11. Modelo del Dominio: añadir asociaciones	145
12. Modelo del Dominio: añadir atributos	157
13. Modelo de Casos de Uso: añadir detalles con los contratos de las operaciones	167
14. De los requisitos al diseño en esta iteración	181
15. Notación de los diagramas de interacción	185
16. GRASP: diseño de objetos con responsabilidades	201
17. Modelo de Diseño: realización de los casos de uso con los patrones GRASP	231
18. Modelo de Diseño: determinación de la visibilidad	261

19.	Modelo de Diseño: creación de los diagramas de clases de diseño	267
20.	Modelo de Implementación: transformación de los diseños en código . .	281

Parte 4: Elaboración en la Iteración 2

21.	La Iteración 2 y sus requisitos	291
22.	GRASP: más patrones para asignar responsabilidades	305
23.	Diseño de las realizaciones de casos de uso con los patrones de diseño GoF .	321

Parte 5: Elaboración en la Iteración 3

24.	La Iteración 3 y sus requisitos	359
25.	Relaciones entre casos de uso	361
26.	Modelado de la generalización	369
27.	Refinamiento del modelo del dominio	383
28.	Añadir nuevos DSSs y contratos	401
29.	Modelado del comportamiento con diagramas de estado	407
30.	Diseño de la arquitectura lógica con patrones	417
31.	Organización de los paquetes de los modelos de diseño e implementación .	443
32.	Introducción al análisis arquitectural y el SAD	451
33.	Diseño de más realizaciones de casos de uso con objetos y patrones	473
34.	Diseño de un framework de persistencia con patrones	501

Parte 6: Temas especiales

35.	Sobre el dibujo de diagramas y las herramientas	531
36.	Introducción a cuestiones relacionadas con la planificación iterativa y el proyecto	539
37.	Comentarios acerca del desarrollo iterativo y el UP	553
38.	Más notación UML	567

Presentación	XIX
------------------------	-----

Prólogo	XXI
-------------------	-----

Parte 1: Introducción

Capítulo 1: Análisis y diseño orientado a objetos	3
---	---

Aplicación de UML y patrones en el A/DOO	3
Asignación de responsabilidades	5
¿Qué es análisis y diseño?	6
¿Qué son el análisis y diseño orientados a objetos?	6
Un ejemplo	7
UML	10
Lecturas adicionales	10

Capítulo 2: Desarrollo iterativo y el Proceso Unificado	13
---	----

La idea más importante del UP: desarrollo iterativo	14
Conceptos y buenas prácticas del UP adicionales	18
Las fases del UP y términos orientados a la planificación	19
Las disciplinas del UP (eran flujos de trabajo)	20
Adaptación del proceso y el Marco de Desarrollo	22
El UP ágil	23
El ciclo de vida “en cascada” secuencial	24
No se entendió el UP cuando...	25
Lecturas adicionales	25

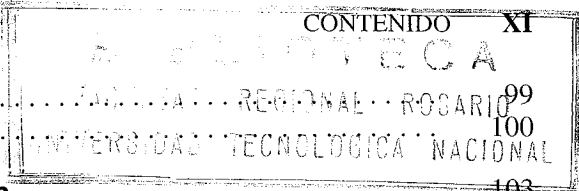
Capítulo 3: Caso de estudio: el sistema de punto de venta NuevaEra .	27
--	----

El sistema de punto de venta NuevaEra	27
Capas arquitectónicas y el énfasis del caso de estudio	28
Estrategia del libro: aprendizaje y desarrollo iterativo	29

Parte 2: Inicio

Capítulo 4: Inicio	33
Inicio: una analogía	34
La fase de inicio podría ser muy breve	35
¿Qué artefactos podrían crearse en la fase de inicio?	35
No se entendió la fase de inicio cuando...	36
Capítulo 5: Comprensión de los requisitos	39
Tipos de requisitos	40
Lecturas adicionales	41
Capítulo 6: Modelo de Casos de Uso: escritura de requisitos en contexto	43
Objetivos e historias	44
Antecedentes	44
Casos de uso y valor añadido	45
Casos de uso y requisitos funcionales	46
Tipos de casos de uso y formatos	46
Ejemplo completo: Procesar Venta	47
Explicación de las secciones	52
Objetivos y alcance de un caso de uso	56
Descubrimiento de actores principales, objetivos y casos de uso	60
Enhorabuena: se han escrito los casos de uso y no son perfectos	64
Escritura de casos de uso en un estilo esencial independiente de la interfaz de usuario	65
Actores	67
Diagramas de casos de uso	68
Requisitos en contexto y lista de características de bajo nivel	70
Los casos de uso no son orientados a objetos	71
Casos de uso en el UP	72
Caso de estudio: casos de uso en la fase de inicio de NuevaEra	76
Lecturas adicionales	76
Artefactos UP y contexto del proceso	77
Capítulo 7: Identificación de otros requisitos	79
Ejemplos del PDV NuevaEra	80
Ejemplo NuevaEra: Especificación Complementaria (Parcial)	80
Comentario: Especificación Complementaria	84
Ejemplo NuevaEra: Visión (Parcial)	87
Comentario: Visión	90
Ejemplo NuevaEra: un Glosario (Parcial)	94
Comentario: Glosario (Diccionario de Datos)	95
Especificaciones fiables: ¿un Oxímoron?	96
Artefactos disponibles en el sitio web del proyecto	97
¿Poco UML durante la fase de inicio?	97
Otros artefactos de requisitos en el UP	97

Lecturas adicionales	99
Artefactos UP y contexto del proceso	100



Capítulo 8: Del inicio a la elaboración	103
Punto de control: ¿qué sucedió en el inicio?	104
En la elaboración	105
Planificación de la siguiente iteración	106
Requisitos y énfasis de la iteración 1: habilidades de A/DOO fundamentales	108
¿Qué artefactos podrían crearse en la elaboración?	108
No se entendió la elaboración cuando...	110

Parte 3: Elaboración en la Iteración 1

Capítulo 9: Modelo de Casos de Uso: representación de los diagramas de secuencia del sistema	113
Comportamiento del sistema	114
Diagramas de secuencia del sistema	114
Ejemplo de un DSS	115
DSS entre sistemas	115
DSS y los casos de uso	116
Eventos del sistema y los límites del sistema	116
Asignación de nombres a los eventos y operaciones	117
Mostrar el texto del caso de uso	118
Los DSS y el Glosario	118
DSS en el UP	118
Lecturas adicionales	119
Artefactos del UP	120
Capítulo 10: Modelo del Dominio: visualización de conceptos	121
Modelos del Dominio	122
Identificación de las clases conceptuales	126
Clases conceptuales candidatas para el dominio de ventas	129
Guías para el modelado del negocio	130
Resolución de clases conceptuales similares: Registro vs. "TPDV"	132
Modelado del mundo <i>irreal</i>	133
Clases conceptuales de especificación o descripción	133
Notación UML, modelos y métodos: perspectivas múltiples	136
Reducción del salto en la representación	138
Ejemplo: el Modelo del Dominio del PDV NuevaEra	140
Modelos del Dominio en el UP	140
Lecturas adicionales	142
Artefactos del UP	142

Capítulo 11: Modelo del Dominio: añadir asociaciones	145
Asociaciones	145
Notación de las asociaciones en UML	146

Localización de las asociaciones—lista de asociaciones comunes	147
Guías para las asociaciones	148
Roles	149
¿Cómo de detalladas deben ser las asociaciones?	150
Asignación de nombres a las asociaciones	151
Múltiples asociaciones entre dos tipos	152
Asociaciones e implementación	152
Asociaciones del Modelo del Dominio del PDV NuevaEra	153
Modelo del Dominio del PDV NuevaEra	154
Capítulo 12: Modelo del Dominio: añadir atributos	157
Atributos	157
Notación de los atributos en UML	158
Tipos de atributos válidos	158
Clases de tipos de datos no primitivos	160
Deslizarse al diseño: ningún atributo como clave ajena	162
Modelado de cantidades y unidades de los atributos	162
Atributos en el Modelo del Dominio de NuevaEra	163
Multiplicidad de la LineaDeVenta al Artículo	163
Conclusión del Modelo del Dominio	164
Capítulo 13: Modelo de Casos de Uso: añadir detalles con los contra- tos de las operaciones	167
Contratos	167
Ejemplo de contrato: introducirArticulo	168
Secciones del contrato	169
Postcondiciones	169
Discusión: postcondiciones de introducirArticulo	171
La escritura de los contratos da lugar a actualizaciones en el Modelo del Dominio	172
¿Cuándo son útiles los contratos? ¿Contratos vs. casos de uso?	173
Guías: contratos	173
Ejemplo del PDV NuevaEra: contratos	174
Cambios en el Modelo del Dominio	175
Contratos, operaciones y UML	176
Contratos de las operaciones en el UP	177
Lecturas adicionales	178
Capítulo 14: De los requisitos al diseño en esta iteración	181
Iterativamente hacer lo correcto, y hacerlo correcto	181
¿No lleva eso semanas en hacerse? No, no exactamente	182
Pasar al diseño de objetos	182
Capítulo 15: Notación de los diagramas de interacción	185
Diagramas de secuencia y colaboración	186
Ejemplo de diagrama de colaboración: realizarPago	187
Ejemplo de diagrama de secuencia: realizarPago	187
Los diagramas de interacción son importantes	188

Notación general de los diagramas de interacción	189
Notación básica de los diagramas de colaboración	190
Notación básica de los diagramas de secuencia	195
Capítulo 16: GRASP: diseño de objetos con responsabilidades	201
Responsabilidades y métodos	202
Responsabilidades y los diagramas de interacción	203
Patrones	204
GRASP: Patrones de Principios Generales para Asignar Responsabilidades ..	205
Notación del diagrama de clases UML	206
Experto en Información (o Experto)	207
Creador	211
Bajo Acoplamiento	214
Alta Cohesión	217
Controlador	221
Diseño de objetos y tarjetas CRC	229
Lecturas adicionales	230
Capítulo 17: Modelo de Diseño: realización de los casos de uso con los patrones GRASP	231
Realizaciones de casos de uso	232
Comentarios sobre los artefactos	232
Realizaciones de los casos de uso para la iteración de NuevaEra	236
Diseño de objetos: crearNuevaVenta	236
Diseño de objetos: introducirArticulo	239
Diseño de objetos: finalizarVenta	243
Diseño de objetos: realizarPago	247
Diseño de objetos: ponerEnMarcha	252
Conexión de la capa de UI con la capa del dominio	255
Realizaciones de casos de uso en el UP	258
Resumen	258
Capítulo 18: Modelo de Diseño: determinación de la visibilidad	261
Visibilidad entre objetos	261
Visibilidad	262
Representación de la visibilidad en UML	266
Capítulo 19: Modelo de Diseño: creación de los diagramas de clases de diseño	267
Cuándo crear los DCD	267
Ejemplo de DCD	268
Terminología del DCD y el UP	268
Clases del Modelo de Dominio vs. clases del Modelo de Diseño	269
Creación de un DCD del PDV NuevaEra	269
Notación para los detalles de los miembros	277
DCD, dibujo y herramientas CASE	278
DCD en el UP	278
Artefactos del UP	280

Capítulo 20: Modelo de Implementación: transformación de los diseños en código	281
Programación y el proceso de desarrollo	282
Transformación de los diseños en código	284
Creación de las definiciones de las clases a partir de los DCDs	284
Creación de métodos a partir de los diagramas de interacción	287
Clases contenedoras/colecciones en el código	288
Manejo de excepciones y de errores	289
Definición del método Venta--crearLineaDeVenta	289
Orden de implementación	290
Programar probando primero	291
Resumen de la transformación de los diseños en código	292
Introducción a la solución del programa	292

Parte 4: Elaboración en la Iteración 2

Capítulo 21: La Iteración 2 y sus requisitos	299
Énfasis de la Iteración 2: diseño de objetos y patrones	299
De la Iteración 1 a la 2	300
Requisitos de la Iteración 2	301
Refinamiento de los artefactos orientados al análisis en esta iteración	303
Capítulo 22: GRASP: más patrones para asignar responsabilidades	305
Polimorfismo	306
Fabricación Pura	308
Indirección	312
Variaciones Protegidas	313
Capítulo 23: Diseño de las realizaciones de casos de uso con los patrones de diseño GoF	321
Adaptador (GoF)	322
Descubrimientos del "análisis" durante el diseño: Modelo del Dominio	324
Factoría (GoF)	326
Singleton (GoF)	328
Conclusiones del problema de los servicios externos con diversas interfaces	331
Estrategia (GoF)	332
Composite (GoF) y otros principios de diseño	337
Fachada (GoF)	346
Observador/Publicar-Suscribir/Modelo de Delegación de Eventos (GoF)	348
Conclusión	356
Lecturas adicionales	356

Parte 5: Elaboración en la Iteración 3

Capítulo 24: La Iteración 3 y sus requisitos	359
Requisitos de la Iteración 3	359
Énfasis de la Iteración 3	359

Capítulo 25: Relaciones entre casos de uso	361
La relación de inclusión (<i>include</i>)	362
Terminología: casos de uso concretos, abstractos, base y adicional	364
La relación de extensión (<i>extend</i>)	365
La relación de generalización (<i>generalize</i>)	366
Diagramas de casos de uso	367
Capítulo 26: Modelado de la generalización	369
Nuevos conceptos para el Modelo del Dominio	369
Generalización	371
Definición de superclases y subclases conceptuales	373
Cuándo definir una clase conceptual	375
Cuándo definir una superclase conceptual	377
Jerarquías de clases conceptuales del PDV NuevaEra	377
Clases conceptuales abstractas	380
Modelado de los cambios de estado	381
Jerarquías de clases y herencia en el software	382
Capítulo 27: Refinamiento del modelo del dominio	383
Clases asociación	383
Agregación y composición	385
Intervalos de tiempo y precios de los productos: arreglar un "error" de la Iteración 1	390
Nombres de los roles de asociación	390
Roles como conceptos vs. roles en asociaciones	391
Elementos derivados	392
Asociaciones calificadas	393
Asociaciones reflexivas	394
Elementos ordenados	394
Utilización de paquetes para ordenar el Modelo del Dominio	394
Capítulo 28: Añadir nuevos DSSs y contratos	401
Nuevos diagramas de secuencia del sistema	401
Nuevas operaciones del sistema	403
Nuevos contratos de operaciones del sistema	404
Capítulo 29: Modelado del comportamiento con diagramas de estados	407
Eventos, estados y transiciones	407
Diagramas de estados	408
¿Diagramas de estados en el UP?	409
Diagramas de estados de casos de uso	409
Diagramas de estados de casos de uso para la aplicación del PDV	410
Clases que se benefician de los diagramas de estados	411
Representación de eventos externos e internos	412
Notación adicional de los diagramas de estados	413
Lecturas adicionales	415

Capítulo 30: Diseño de la arquitectura lógica con patrones	417
Arquitectura del software	418
Patrón de arquitectura: Capas (<i>Layers</i>)	420
Principio de Separación Modelo-Vista	440
Lecturas adicionales	442
Capítulo 31: Organización de los paquetes de los modelos de diseño e implementación	443
Guías para la organización de paquetes	444
Notación adicional de los paquetes en UML	450
Lecturas adicionales	450
Capítulo 32: Introducción al análisis arquitectural y el SAD	451
Análisis arquitectural	452
Tipos y vistas de la arquitectura	454
La ciencia: identificación y análisis de los factores de la arquitectura	454
Ejemplo: tabla de factores parcial de la arquitectura del PDV NuevaEra	457
El arte: resolución de los factores de la arquitectura	459
Resumen de los temas del análisis arquitectural	466
Análisis arquitectural en el UP	467
Lecturas adicionales	471
Capítulo 33: Diseño de más realizaciones de casos de uso con objetos y patrones	473
Mantenimiento de los servicios ante los fallos mediante servicios locales; rendimiento con el almacenamiento local	473
Manejo de fallos	479
Mantenimiento de los servicios ante los fallos mediante un Proxy (GoF)	484
Diseño para los requisitos no funcionales o de calidad	488
Acceso a los dispositivos físicos externos con adaptadores; comprar vs. construir ..	488
Factoría Abstracta (GoF) para familias de objetos relacionados	490
Gestión de pagos con Polimorfismo y Hacerlo Yo Mismo	493
Conclusión	498
Capítulo 34: Diseño de un framework de persistencia con patrones ..	501
El problema: objetos persistentes	502
La solución: un servicio de persistencia a partir de un framework de persistencia ..	502
Frameworks	503
Requisitos para el servicio y framework de persistencia	503
Ideas claves	504
Patrón: Representación de Objetos como Tablas	504
Perfil (<i>Profile</i>) de modelado de datos en UML	505
Patrón: Identificador de Objeto	505
Acceso al servicio de persistencia con una Fachada	506
Correspondencia de los objetos: patrón Conversor (<i>Mapper</i>) de Base de Datos o Intermediario (<i>Broker</i>) de Base de Datos	507

Diseño del framework con el patrón Método Plantilla	509
Materialización con el patrón Método Plantilla	510
Configuración de conversores con una FactoríaDeConversores	515
Patrón: Gestión de Caché	515
Reunir y ocultar sentencias SQL en una clase	516
Estados transaccionales y el patrón Estado	517
Diseño de una transacción con el Patrón Command	520
Materialización perezosa con un Proxy Virtual	522
Cómo representar las relaciones en tablas	524
Superclase ObjetoPersistente y separación de intereses	525
Cuestiones sin resolver	526

Parte 6: Temas especiales

Capítulo 35: Sobre el dibujo de diagramas y las herramientas	529
Diseño especulativo y razonamiento visual	529
Sugerencias para dibujar los diagramas de UML en el proceso de desarrollo	530
Herramientas y características de ejemplo	533
Ejemplo dos	534
Capítulo 36: Introducción a cuestiones relacionadas con la planificación iterativa y el proyecto	537
Priorización de los requisitos	538
Priorización de los riesgos del proyecto	541
Planificación adaptable vs. predictiva	541
Planes de Fase y de Iteración	543
Plan de Iteración: ¿qué hacemos en la siguiente iteración?	543
Traza de los requisitos a través de las iteraciones	544
La (in)validez de las primeras estimaciones	546
Organización de los artefactos del proyecto	547
Algunas cuestiones de la planificación de la iteración del equipo	548
No se entendió la planificación en el UP cuando... ..	549
Lecturas adicionales	550
Capítulo 37: Comentarios acerca del desarrollo iterativo y el UP ...	551
Buenas prácticas y conceptos del UP adicionales	551
Las fases de construcción y transición	553
Otras prácticas interesantes	554
Motivos para fijar la duración de una iteración	555
El ciclo de vida secuencial en "cascada"	555
Ingeniería de usabilidad y diseño de interfaces de usuario	561
El Modelo de Análisis del UP	561
El producto del RUP	562
Los desafíos y mitos de la reutilización	563

Capítulo 38: Más notación UML	565
Notación general	565
Diagramas de implementación	566
Clase plantilla (parametrizada, genérica)	567
Diagramas de actividades	567
Bibliografía	571
Glosario	577
Índice alfabético	583

PRESENTACIÓN

Programar es divertido, pero desarrollar software de calidad es difícil. Entre las ideas espléndidas, los requisitos o la “visión”, y un producto software funcionando, hay mucho más que programar. El análisis y el diseño que definen cómo solucionar el problema, qué programar, y la expresión de este diseño de forma que sea fácil de comunicar, revisar, implementar y evolucionar constituyen la parte central de este libro. Esto es lo que aprenderás.

El Lenguaje Unificado de Modelado (UML) se ha convertido en el lenguaje aceptado universalmente para los planos del diseño software. UML es el lenguaje visual utilizado a lo largo de este libro para ilustrar las ideas de diseño, poniendo énfasis en cómo aplican los desarrolladores realmente los elementos UML utilizados con más frecuencia, más que en características oscuras del lenguaje.

La importancia de los patrones en la creación de sistemas complejos es reconocida desde hace tiempo en otras disciplinas. Los patrones de diseño software son los que nos permiten describir fragmentos de diseño y reutilizar ideas de diseño, ayudando a beneficiarse de la experiencia de otros. Los patrones dan nombre y forma a heurísticas abstractas, reglas y buenas prácticas de técnicas orientadas a objetos. Ningún ingeniero razonable quiere partir de una pizarra en blanco, y este libro ofrece una paleta de patrones de diseño que pueden utilizarse fácilmente.

Pero el diseño de software parece un poco árido y misterioso cuando no se presenta en el contexto de un proceso de ingeniería del software. Y sobre este tema, estoy encantado de que para su segunda edición, Craig Larman haya elegido adoptar e introducir el Proceso Unificado, mostrando cómo puede aplicarse de un modo relativamente simple y poco ceremonioso. Al presentar el caso de estudio a través de un proceso centrado en la arquitectura, dirigido por el riesgo e iterativo, los consejos de Craig tienen un contexto realista; expone la dinámica de lo que ocurre en realidad en el desarrollo de software, y muestra las fuerzas externas que entran en juego. Las actividades del diseño están conectadas a otras tareas, y ya no aparecen como actividades puramente cerebrales de transformaciones sistemáticas o intuición creativa. Y Craig y yo estamos convencidos de los beneficios de un desarrollo iterativo, que verás ilustrado con todo detalle a lo largo del libro.

Así es que para mí, este libro tiene la mezcla correcta de ingredientes. Aprenderás un método sistemático para abordar el Análisis y Diseño Orientado a Objetos (A/DOO) de un gran profesor, un metodologista brillante, y un “gurú de la OO” que lo ha enseñado a miles de personas alrededor del mundo. Craig describe el método en el contexto del Pro-

ceso Unificado. Presenta gradualmente patrones de diseño más sofisticados —esto hará el libro muy útil y fácil de usar cuando te enfrentes con retos de diseño del mundo real. Y utiliza la notación más ampliamente aceptada.

Me siento honrado de haber tenido la oportunidad de trabajar directamente con el autor de este importante libro. Disfruté leyendo la primera edición, y me encantó que me pidiera que revisara el borrador de su segunda edición. Nos encontramos varias veces e intercambiamos muchos correos electrónicos. He aprendido mucho de Craig, incluso sobre nuestro propio proceso de trabajo en el Proceso Unificado y cómo mejorarlo y aplicarlo en varios contextos organizacionales. Estoy seguro de que aprenderás mucho, bastante, al leer este libro, incluso si ya estás familiarizado con el A/DOO. Y, como yo, a menudo volverás a él, para refrescar tu memoria o para comprender mejor las explicaciones y experiencias de Craig.

En un proceso iterativo, el resultado de la segunda iteración mejora la primera. Similarmemente, la escritura madura, supongo; incluso si tienes la primera edición, disfrutarás y te beneficiarás de la segunda.

¡Feliz lectura!

*Philippe Kruchten
Rational Fellow
Rational Software Canada
Vancouver, BC*

*Diseñar sistemas
de objetos robustos
y de fácil
mantenimiento*

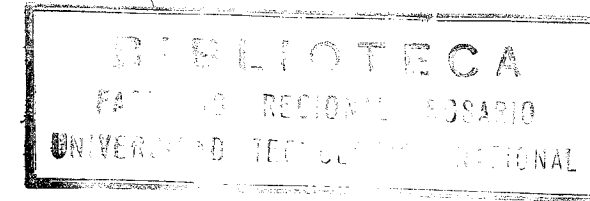
*Seguir un mapa
a través de los
requisitos, análisis,
diseño
y codificación*

*Usar UML para
ilustrar los modelos
de análisis y diseño*

*Mejorar los diseños
aplicando los
patrones de diseño
GRASP y
los de la "pandilla
de los cuatro"*

*Aprendizaje
eficiente
siguiendo una
presentación
refinada*

*Aprendizaje
mediante
un ejercicio
realista*



PRÓLOGO

¡Gracias por leer este libro! Ésta es una introducción práctica al análisis y diseño orientado a objetos (A/DOO), y a aspectos relacionados de desarrollo iterativo. Estoy agradecido de que la primera edición fuese recibida por todo el mundo como una introducción sencilla al A/DOO, traducida a muchos idiomas. Por tanto, esta segunda edición refina y se construye sobre el contenido de la primera, más que reemplazarla. Quiero dar las gracias sinceramente a todos los lectores de la primera edición.

Aquí tienes los beneficios que te proporcionará el libro.

Primero, el uso de la tecnología de objetos ha proliferado en el desarrollo de software, y el dominio del A/DOO es crítico para crear sistemas de objetos robustos y de fácil mantenimiento.

Segundo, si eres nuevo en el A/DOO, te preguntarás comprensiblemente cómo avanzar por este tema tan complejo; este libro presenta un mapa bien definido —el Proceso Unificado— de manera que te puedas mover en un proceso paso a paso, desde los requisitos al código.

Tercero, el Lenguaje Unificado de Modelado (UML) ha emergido como la notación estándar para el modelado; por tanto, te resultará útil familiarizarte con él. Este libro enseña las técnicas del A/DOO utilizando la notación UML.

Cuarto, los patrones de diseño comunican los estilos y soluciones consideradas como “buenas prácticas”, que los expertos en el diseño orientado a objetos utilizan para la creación de sistemas. En este libro aprenderás a aplicar patrones de diseño, incluyendo los populares patrones de la “pandilla de los cuatro” (*gang-of-four*) y los patrones GRASP que comunican los principios fundamentales de asignación de responsabilidades en el diseño orientado a objetos. Aprender y aplicar patrones acelerará el dominio del análisis y el diseño.

Quinto, la estructura y el enfoque del libro se basa en años de experiencia en la enseñanza y asesoramiento a miles de personas en el arte del A/DOO. Refleja esa experiencia proporcionando un enfoque eficiente, probado y refinado para aprender la materia de manera que se optimiza tu inversión en leer y aprender.

Sexto, examina de manera exhaustiva un único caso de estudio —para ilustrar de manera realista el proceso de A/DOO completo—, y examina en profundidad detalles espinosos del problema; es un ejercicio realista.

Traducción a código

Séptimo, muestra cómo obtener código Java a partir de los artefactos del diseño de objetos.

Diseño de una arquitectura de capas

Octavo, explica cómo diseñar una arquitectura de capas y relaciona la capa de interfaz gráfica de usuario con las capas del dominio y servicios técnicos.

Diseño de un framework

Por último, muestra cómo diseñar un framework orientado a objetos aplicándolo a la creación de uno para el almacenamiento persistente en una base de datos.

Objetivos

El objetivo global es:

Ayudar a los estudiantes y a los desarrolladores a crear diseños orientados a objetos mediante la aplicación de un conjunto de principios y heurísticas explicables.

Estudiando y aplicando la información y las técnicas que se presentan aquí, adquirirás experiencia en la comprensión de un problema en términos de sus procesos y conceptos, y en el diseño de una solución utilizando objetos.

A quiénes está dirigido este libro

Este libro es una *introducción* al A/DOO, al análisis de requisitos relacionado, y al desarrollo iterativo, con el Proceso Unificado como ejemplo de un proceso; la intención no es ser un texto avanzado. Va destinado a la siguiente audiencia:

- Desarrolladores y estudiantes con experiencia en un lenguaje de programación orientado a objetos, pero que son nuevos —o relativamente nuevos— en el análisis y diseño orientado a objetos.
- Estudiantes de informática o cursos de ingeniería del software que estudien la tecnología de objetos.
- Aquellos familiarizados con el A/DOO que quieran aprender la notación UML, aplicar patrones, o que quieran mejorar y perfeccionar sus habilidades de análisis y diseño.

Requisitos

Se asumen —y se necesitan— algunos conocimientos previos para aprovechar este libro:

- Conocimiento y experiencia en un lenguaje de programación orientado a objetos como Java, C#, C++ o Smalltalk.
- Conocimiento de los conceptos generales de la tecnología de objetos, como clase, instancia, interfaz, polimorfismo, encapsulación y herencia.

No se definen los conceptos fundamentales de la tecnología de objetos.

Ejemplos Java

En general, el libro presenta ejemplos de código en Java o plantea implementaciones Java, debido a que su uso está muy extendido. Sin embargo, las ideas presentadas son aplicables a la mayoría —si no a todos— los lenguajes orientados a objetos.

Organización del libro

La estrategia global en la organización de este libro es introducir las cuestiones de análisis y diseño en un orden similar al de un proyecto de desarrollo de software a través de una fase de “inicio” (término del Proceso Unificado) seguido por tres iteraciones (ver Figura P.1).

1. Los capítulos de la fase de inicio introducen los fundamentos del análisis de requisitos.
2. La iteración 1 introduce el A/DOO básico y cómo asignar responsabilidades a los objetos.
3. La iteración 2 se centra en el diseño de objetos, especialmente en introducir algunos “patrones de diseño” muy utilizados.
4. La iteración 3 introduce una variedad de temas, como el análisis de la arquitectura y el diseño de frameworks.

El Libro

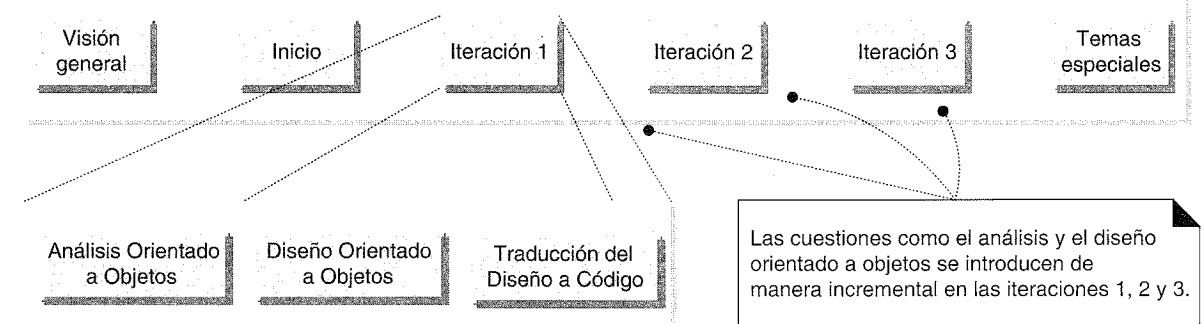


Figura P.1. El libro está organizado como un proyecto de desarrollo.

Recursos web relacionados

- Por favor, consúltese www.craiglarman.com para artículos relacionados con la tecnología de objetos, patrones y procesos.
- Se pueden encontrar recursos para los profesores en www.phptr.com/larman.

Mejoras a la primera edición

Aunque mantiene el mismo núcleo central que la primera edición, la segunda ha refinado muchos aspectos como:

- Se actualizan los casos de uso para seguir el enfoque de [Cockburn01] que ha tenido bastante aceptación.
- Se utiliza el bien conocido Proceso Unificado (UP) como ejemplo de proceso iterativo dentro del cual introducir el A/DOO. Por tanto, todos los artefactos se nombran de acuerdo con la terminología del UP, como Modelo de Dominio.
- Nuevos requisitos en el caso de estudio, que nos encaminan a la tercera iteración.
- Tratamiento actualizado de los patrones de diseño.
- Introducción al análisis arquitectural.
- Introducción de Variaciones Protegidas como un patrón GRASP.
- Balance al 50% entre los diagramas de secuencia y colaboración.
- Actualizada a la notación UML más reciente.
- Discusión de algunos aspectos prácticos de dibujo utilizando pizarras o herramientas CASE para UML.

Reconocimientos

En primer lugar, un agradecimiento muy especial a mis amigos y colegas de Valtech, desarrolladores de objetos a nivel mundial y expertos en el desarrollo iterativo, quienes de alguna manera contribuyeron a apoyar, o revisar el libro, incluyendo a Chris Tarr, Michel Ezran, Tim Snyder, Curtis Hite, Celso González, Pascal Roques, Ken DeLong, Brett Schuchert, Ashley Johnson, Chris Jones, Thomas Liou, Darryl Gebert, Frank Rodorigo, Jean-Yves Hardy, y muchos más que no puedo nombrar.

A Philippe Kruchten por escribir la presentación del libro, revisar y ayudarme de muchas formas.

A Martin Fowler y Alistair Cockburn por muchas discusiones esclarecedoras sobre procesos y diseños, comentarios y revisiones.

A John Vlissides y Cris Kobryn por sus amables comentarios.

A Chelsea Systems y John Gray por ayudarme con algunos requisitos inspirados en su sistema PDV con tecnología Java ChelseaStore.

A Pete Coad y Dave Astels de TogetherSoft por su apoyo.

Muchas gracias a los otros revisores, incluyendo Steve Adolph, Bruce Anderson, Len Bass, Gary K. Evans, Al Goerner, Luke Hohmann, Eric Lefebvre, David Nunn, y Robert J. White.

Gracias a Paul Becker de Prentice-Hall por creer que la primera edición sería un proyecto que merecía la pena, y a Paul Petralia y Patti Guerrieri por dirigir la segunda.

Por último, agradezco especialmente a Graham Glass por abrir una puerta.

Semblanza del autor

Craig Larman trabaja como Director de Procesos para Valtech, una compañía consultora internacional con sucursales en Europa, Asia y América del Norte, especializada en

desarrollo de sistemas de negocio electrónico, tecnologías de objetos y desarrollo iterativo con el Proceso Unificado.

Desde mediados de los ochenta, Craig ha ayudado a miles de desarrolladores a aplicar el análisis, diseño y programación orientada a objetos, y a organizaciones a adoptar las prácticas del desarrollo iterativo.

Después de una carrera fracasada como músico callejero, construyó sistemas en APL, PL/I, y CICS en los setenta. Desde comienzos de los ochenta —después de una completa recuperación— comenzó a interesarse por la inteligencia artificial (teniendo poca propia), procesamiento del lenguaje natural y representación del conocimiento, y construyó sistemas de conocimiento con máquinas Lisp, Prolog y Smalltalk. Toca mal la guitarra eléctrica en su banda *Los Requisitos Cambiantes* (antes se llamaba *Los Requisitos*, pero algunos miembros de la banda cambiaron...) a la que dedica su tiempo libre.

Es licenciado en informática por la Universidad de Simon Fraser en Vancouver, Canadá.

Contacto

Se puede contactar con Craig en clarman@ieee.org y www.craiglarman.com. Serán bienvenidas las preguntas de los lectores y profesores, y peticiones de conferencias, asesoramiento y consultoría.

Convenciones tipográficas

Esto es un **término nuevo** en una frase. Esto es un nombre de *Clase* o un *método* en una frase. Esto es una referencia a un autor [Bob67]. El operador de resolución de alcance independiente del lenguaje “--” se utiliza para indicar una clase y su método asociado como sigue: *NombreClase--NombreMetodo*.

Notas de producción

El manuscrito de este libro se creó con Adobe FrameMaker. Todos los dibujos se hicieron con Microsoft Visio. La fuente del texto es New Century Schoolbook. Las imágenes impresas finales se generaron con ficheros PDF utilizando Adobe Acrobat Distilled, a partir del PostScript generado por un controlador AGFA.

Parte 1

INTRODUCCIÓN

ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

El cambio de enfoque (a patrones) tendrá un efecto profundo y duradero en el modo en el que escribimos los programas.

Ward Cunningham y Ralph Johnson

Objetivos

- Comparar y contrastar análisis y diseño.
 - Definir análisis y diseño orientado a objetos (A/DOO).
 - Ilustrar con un pequeño ejemplo.
-

1.1. Aplicación de UML y patrones en el A/DOO

¿Qué significa tener un buen diseño de objetos? Este libro es una herramienta para ayudar a los desarrolladores y a los estudiantes a aprender las habilidades fundamentales en el análisis y diseño orientado a objetos (A/DOO). Estas técnicas son esenciales para la creación de software de fácil mantenimiento, robusto y bien definido, utilizando las tecnologías y lenguajes de objetos como Java, C++ , Smalltalk y C#.

El proverbio “tener un martillo no te hace un arquitecto” es especialmente cierto con respecto a la tecnología de objetos. Conocer un lenguaje orientado a objetos (como Java) es necesario pero un primer paso insuficiente para crear sistemas de objetos. También es preciso conocer cómo “pensar en objetos”.

Esto es una introducción al A/DOO mientras aplicamos el Lenguaje Unificado de Modelado (UML), patrones y el Proceso Unificado. No se debe entender como un texto avanzado; se centra en el dominio de los fundamentos, como el modo de asignar responsabilidades a los objetos, la notación UML utilizada con más frecuencia, y patrones de

*Esto es una
introducción*

diseño comunes. Al mismo tiempo, principalmente en los capítulos posteriores, el material progresa a unos pocos temas de nivel intermedio, tal como el diseño de frameworks.

Utilización
de UML

El libro no trata únicamente de UML. UML es una notación visual estándar. Tan útil como aprender una notación, hay más cosas orientadas a objetos importantes que aprender; concretamente, cómo pensar en objetos —cómo diseñar sistemas orientados a objetos—. UML no es A/DOO o un método, es simplemente una notación. No es tan útil aprender a hacer diagramas UML sintácticamente correctos y quizás una herramienta CASE para UML, si no se es capaz de crear un diseño excelente o evaluar y mejorar uno existente. Ésta es la habilidad más difícil y valiosa. En consecuencia, este libro es una introducción al diseño de objetos.

Con todo necesitamos un lenguaje para el A/DOO y los “planos de software”, tanto como una herramienta para pensar, como una forma de comunicación con otros. Por tanto, explora cómo *aplicar* UML al servicio del A/DOO, y cubre la notación UML utilizada con más frecuencia. Se presta especial atención a ayudar a que la gente aprenda el arte y la ciencia de construir sistemas de objetos, más que en la notación.

Aplicación
de patrones
y asignación de
responsabilidades

¿Cómo deberíamos asignar las **responsabilidades** a las clases de objetos? ¿Cómo deberían interaccionar los objetos? ¿Qué clases deberían hacer qué? Éstas son preguntas claves en el diseño de un sistema. Ciertas soluciones contrastadas a problemas de diseño se pueden expresar (y se han expresado) como principios, heurísticas o **patrones** de buenas prácticas —llamados fórmulas de solución de problemas que codifican principios de diseño ejemplares—. Este libro, al enseñar cómo aplicar patrones, favorece un aprendizaje más rápido y un uso propio de experto de estos estilos de diseño de objetos fundamentales.

Un caso
de estudio

Esta introducción al A/DOO se ilustra con un **único caso de estudio** que se sigue a lo largo del libro, profundizando en el análisis y diseño lo suficiente para tener en cuenta, y solucionar, algunos de los horribles detalles que se deben considerar y solucionar en un problema real.

Casos de uso
y análisis
de requisitos

El A/DOO (y todo el diseño software) está fuertemente relacionado con la actividad que es un requisito previo del **análisis de requisitos**, que incluye escribir **casos de uso**. Por tanto, el caso de estudio comienza con una introducción a este tema, a pesar de que realmente no es orientado a objetos.

Un ejemplo
de proceso
iterativo,
el Proceso
Unificado

Dadas muchas posibles actividades desde los requisitos hasta la implementación, ¿cómo debería proceder un desarrollador o un equipo? El análisis de requisitos y el A/DOO requieren que se presenten en el contexto de algún proceso de desarrollo. En este caso, se utiliza como *ejemplo* de **proceso de desarrollo iterativo** el bien conocido **Proceso Unificado**, en cuyo marco se presentan estos temas. Sin embargo, los temas de análisis y diseño que se cubren son comunes a muchos enfoques, y aprenderlos en el contexto del Proceso Unificado no invalida su aplicabilidad a otros métodos.

En conclusión, este libro ayuda a los estudiantes y desarrolladores:

- A aplicar principios y patrones para crear mejores diseños de objetos.
- A seguir un conjunto de actividades comunes en el análisis y diseño, basado en el Proceso Unificado como un ejemplo.
- A crear los diagramas utilizados con más frecuencia en la notación UML.

Esto se ilustra en el contexto de un único caso de estudio.



Figura 1.1. Temas y habilidades que se cubren.

Muchas otras habilidades son importantes

La construcción de software conlleva innumerables habilidades y pasos más allá del análisis de requisitos, el A/DOO, y la programación orientada a objetos. Por ejemplo, la ingeniería de usabilidad y el diseño de interfaces de usuario son claves para el éxito; de igual modo que el diseño de bases de datos.

Sin embargo, esta introducción se centra en el A/DOO, y no pretende cubrir todas las cuestiones del desarrollo de software. Es una parte de un dibujo más grande.

1.2. Asignación de responsabilidades

Hay muchas posibles actividades y artefactos en una introducción al A/DOO, y un gran número de principios y directrices. Suponga que debemos elegir una única habilidad práctica entre todos los temas aquí expuestos —una habilidad como una “isla desierta”—. ¿Cuál sería?

Una habilidad clave y fundamental en el A/DOO es la asignación cuidadosa de responsabilidades a los componentes software.

¿Por qué? Porque es una actividad que debe efectuarse —o mientras se dibuja un diagrama UML o programando— e influye fuertemente sobre la robustez, mantenimiento y reutilización de los componentes software.

Por supuesto, hay otras habilidades necesarias en el A/DOO, pero en esta introducción se hace hincapié en la asignación de responsabilidades porque suele ser una habi-

lidad que requiere esfuerzo el llegar a dominarla y al mismo tiempo tiene una importancia vital. En un proyecto real, un desarrollador podría no tener la oportunidad de abordar ninguna otra actividad de análisis o diseño —el proceso de desarrollo con “prisas por codificar”—. Pero incluso en esta situación, la asignación de responsabilidades es inevitable.

En consecuencia, los pasos de diseño de este libro se centran en los principios de asignación de responsabilidades.

Se presentan y aplican nueve principios fundamentales en el diseño de objetos y asignación de responsabilidades. Se organizan en una ayuda al aprendizaje denominada patrones GRASP.

1.3. ¿Qué es análisis y diseño?

El **Análisis** pone énfasis en una *investigación* del problema y los requisitos, en vez de ponerlo en una solución. Por ejemplo, si se desea un nuevo sistema de información informatizado para una biblioteca, ¿cómo se utilizará?

“Análisis” es un término amplio, es más adecuado calificarlo, como *análisis de requisitos* (un estudio de los requisitos) o *análisis de objetos* (un estudio de los objetos del dominio).

El **Diseño** pone énfasis en una *solución conceptual* que satisface los requisitos, en vez de ponerlo en la implementación. Por ejemplo, una descripción del esquema de una base de datos y objetos software. Finalmente, los diseños pueden ser implementados.

Como con el análisis, es más apropiado calificar el término como *diseño de objetos* o *diseño de bases de datos*.

El análisis y el diseño se han resumido en la frase *hacer lo correcto (análisis)*, y *hacerlo correcto (diseño)*.

1.4. ¿Qué son el análisis y el diseño orientados a objetos?

Durante el **análisis orientado a objetos**, se presta especial atención a encontrar y describir los objetos —o conceptos— en el dominio del problema. Por ejemplo, en el caso del sistema de información de la biblioteca, algunos de los conceptos son *Libro*, *Biblioteca*, y *Socio*.

Durante el **diseño orientado a objetos**, se presta especial atención a la definición de los objetos software y en cómo colaboran para satisfacer los requisitos. Por ejemplo, en el sistema de la biblioteca, un objeto software *Libro* podría tener un atributo *título* y un método *obtenerCapitulo* (ver Figura 1.2).

Por último, durante la implementación o programación orientada a objetos, los objetos de diseño se implementan, como la clase Java *Libro*.

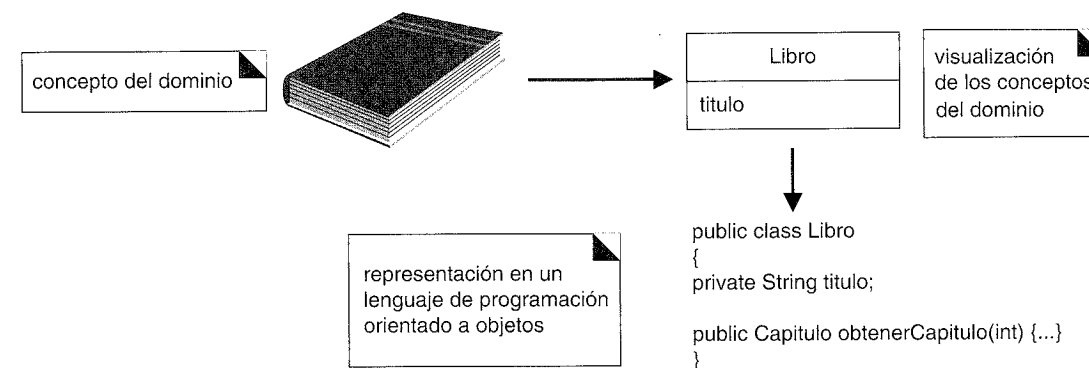


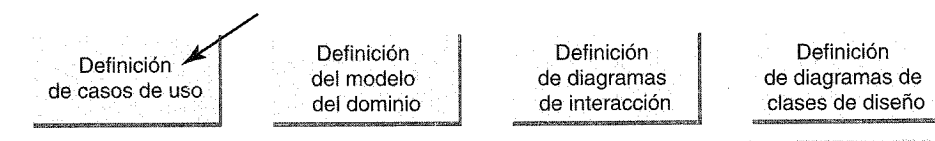
Figura 1.2. La orientación a objetos presta especial atención a la representación de los objetos.

1.5. Un ejemplo

Antes de sumergirnos en los detalles del análisis de requisitos y el A/DOO, esta sección presenta, de modo superficial, unos pocos pasos y diagramas claves, utilizando un ejemplo sencillo —un “juego de dados” en el que un jugador lanza dos dados—. Si el total es siete, gana; en otro caso, pierde.

Definición de los casos de uso

El análisis de requisitos podría incluir una descripción de los procesos del dominio relacionados, que podrían representarse como **casos de uso**.

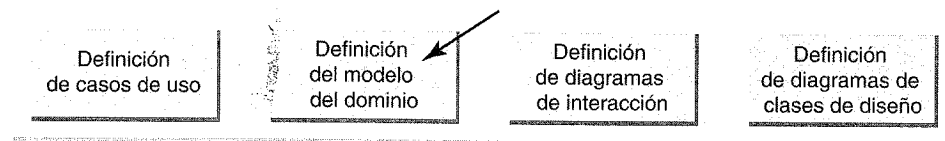


Los casos de uso no son artefactos orientados a objetos —son simplemente historias escritas—. Sin embargo, son una herramienta muy popular en análisis de requisitos y son una parte importante del Proceso Unificado. Por ejemplo, aquí está una versión breve del caso de uso *Jugar una Partida de Dados*:

Jugar una partida de dados: Un jugador recoge y lanza los dados. Si el valor de las caras de los dados suman siete, gana; en otro caso, pierde.

Definición de un modelo del dominio

La finalidad del análisis orientado a objetos es crear una descripción del dominio desde la perspectiva de la clasificación de objetos. Una descomposición del dominio conlleva una identificación de los conceptos, atributos y asociaciones que se consideran significativas. El resultado se puede expresar en un **modelo del dominio**, que se ilustra mediante un conjunto de diagramas que muestran los objetos o conceptos del dominio.



Por ejemplo, la Figura 1.3 muestra un modelo del dominio parcial.

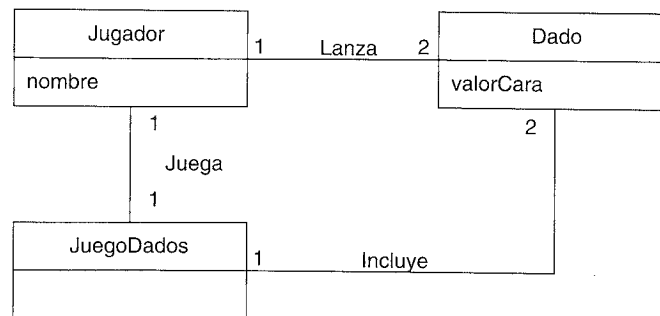


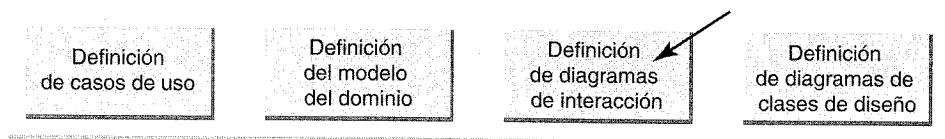
Figura 1.3. Modelo del dominio parcial del juego de dados.

Este modelo ilustra los conceptos importantes *Jugador*, *Dado* y *JuegoDados*, con sus asociaciones y atributos.

Nótese que un modelo del dominio no es una descripción de los objetos software, es una visualización de los conceptos en el dominio del mundo real.

Definición de los diagramas de interacción

La finalidad del diseño orientado a objetos es definir los objetos software y sus colaboraciones. Una notación habitual para ilustrar estas colaboraciones es el **diagrama de interacción**. Muestra el flujo de mensajes entre los objetos software y, por tanto, la invocación de métodos.



Por ejemplo, supongamos que se desea la implementación de un juego de dados. El diagrama de interacción de la Figura 1.4 ilustra los pasos esenciales del juego, enviando mensajes a las clases *JuegoDados* y *Dado*.

Nótese que aunque en el mundo real un *jugador* lanza los dados, en el diseño software el objeto *JuegoDados* “tira” los dados (es decir, envía mensajes a los objetos *Dado*). Los diseños de los objetos software y los programas se inspiran en los dominios del mundo real, pero *no* son modelos directos o simulaciones del mundo real.

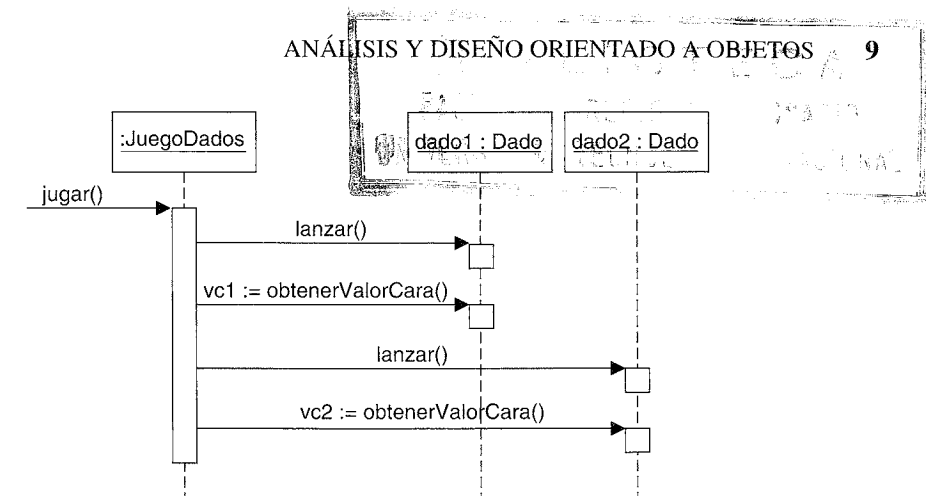
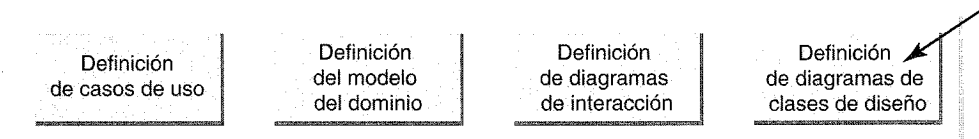


Figura 1.4. Diagrama de interacción que muestra los mensajes entre los objetos software.

Definición de los diagramas de clases de diseño

Además de la vista *dinámica* de las colaboraciones entre los objetos que se muestra mediante los diagramas de interacción, es útil crear una vista *estática* de las definiciones de las clases mediante un **diagrama de clases de diseño**.



Por ejemplo, en el juego de dados, un estudio del diagrama de interacción nos conduce al diagrama de clases de diseño parcial que se muestra en la Figura 1.5. Puesto que se envía el mensaje *jugar* al objeto *JuegoDados*, *JuegoDados* requiere un método *jugar*, mientras la clase *Dado* requiere los métodos *lanzar* y *obtenerValorCara*.

A diferencia del modelo de dominio, este diagrama no muestra conceptos del mundo real, sino clases software.

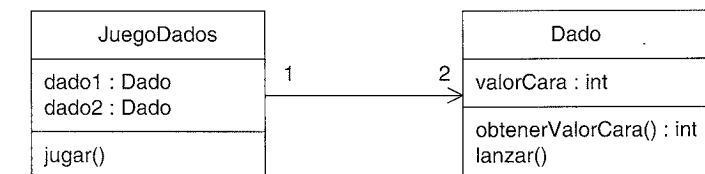


Figura 1.5. Diagrama de clases de diseño parcial.

Resumen

El juego de dados es un problema sencillo, presentado para centrar nuestra atención en unos pocos pasos y artefactos del análisis y diseño. Para no complicar la introducción, no se ha explicado toda la notación UML que se ha mostrado. En los siguientes capítulos se explorará el análisis y el diseño y estos artefactos con más detalle.

1.6. UML

Citando textualmente:

El Lenguaje Unificado de Modelado (UML) es un lenguaje para especificar, visualizar, construir y documentar los artefactos de los sistemas software, así como para el modelado del negocio y otros sistemas no software [OMG01].

UML se ha convertido en la notación visual estándar *de facto* y *de iure* para el modelado orientado a objetos. Comenzó como una iniciativa de Grady Booch y Jim Rumbaugh en 1994 para combinar las notaciones visuales de sus dos populares métodos —los métodos de Booch y OMT (*Object Modeling Technique*)—. Más tarde se les unió Ivar Jacobson, el creador del método Objectory, y el grupo comenzó a ser conocido como los *tres amigos*. Muchos otros contribuyeron a UML, quizás más notablemente Cris Kobryn, que lidera el proceso de refinamiento que todavía continúa.

UML fue adoptado en 1997 como estándar por el OMG (*Object Management Group*, organización que promueve estándares para la industria), y continúa siendo refinado en nuevas versiones.

Este libro no cubre cada pequeño aspecto de UML, que es una notación muy grande (algunos dicen, demasiado grande¹). Se centra en los diagramas que se utilizan con más frecuencia, las características utilizadas más comúnmente en esos diagramas, y la notación central que no es probable que cambie en futuras versiones de UML.

¿Por qué no veremos mucho UML durante unos pocos capítulos?

Éste no es simplemente un libro de notación UML, sino que trata de algo más amplio como es la aplicación de UML, patrones y un proceso iterativo en el contexto de desarrollo de software. UML se aplica principalmente durante el A/DOO, precedido, normalmente, por el análisis de requisitos. Por tanto, los capítulos iniciales presentan una introducción a las cuestiones importantes de casos de uso y análisis de requisitos, seguidos entonces de capítulos de A/DOO y más detalles de UML.

1.7. Lecturas adicionales

UML Distilled, de Martin Fowler, es un libro que ha tenido mucho éxito, y que merece la pena leer, que describe de forma concisa la notación UML más importante.

The Rational Unified Process-An Introduction de Philippe Kruchten es una introducción concisa y muy conocida al Proceso Unificado (y su refinamiento en el Proceso Unificado de Rational).

¹ Entre los objetivos de la versión 2.0 de UML se incluye la simplificación y reducción de la notación. Este libro presenta la parte de UML más utilizada que es muy probable que se mantenga tras la simplificación.

Para una discusión más detallada sobre la notación UML (versión 1.3), merecen la pena *The Unified Modeling Language Reference Manual* y *The Unified Modeling Language User Guide*, de Booch, Jacobson y Rumbaugh. Nótese que la intención de estos textos no era enseñar cómo hacer el modelado de objetos o el A/DOO —son referencias de la notación de los diagramas UML—.

Para una descripción de la versión actual de UML, es necesario la especificación on-line *OMG Unified Modeling Language Specification* en www.omg.org. Los trabajos de revisión de UML y las versiones que van a estar disponibles pronto se pueden encontrar en www.celigent.com/uml.

Hay muchos libros sobre patrones software, pero el libro que más ha influido en el área es el clásico *Design Patterns* de Gamma, Helm, Johnson y Vlissides. Es una lectura verdaderamente obligada para aquellos que estudian el diseño de objetos. Sin embargo, no es un texto introductorio y es mejor que se lea después de conocer bien los fundamentos del diseño y la programación de objetos.

DESARROLLO ITERATIVO Y EL PROCESO UNIFICADO

*Las personas son más importantes que cualquier proceso.
Buenas personas con un buen proceso siempre actuarán mejor
que buenas personas sin procesos.*

Grady Booch

Objetivos

- Explicar la motivación del orden y contenido de los capítulos siguientes.
 - Definir un proceso iterativo y adaptable.
 - Definir los conceptos fundamentales del Proceso Unificado.
-

Introducción

El desarrollo iterativo es un enfoque para el desarrollo de software que requiere un entrenamiento y poseer ciertos conocimientos, y juega un papel central en el modo en que se presenta el A/DOO en este libro. El Proceso Unificado es un ejemplo de proceso iterativo para proyectos que utilizan el A/DOO, y da forma a la presentación del libro. En consecuencia, es útil leer este capítulo para clarificar estos conceptos fundamentales y su influencia en la estructura del libro.

Este capítulo resume algunas ideas claves; el lector se puede dirigir al Capítulo 37 para una discusión más detallada del UP y las prácticas de los procesos iterativos.

De manera informal, un **proceso de desarrollo de software** describe un enfoque para la construcción, desarrollo y, posiblemente, mantenimiento del software. El **Proceso Unificado** [JBR99] se ha convertido en un proceso de desarrollo de software de gran

éxito para la construcción de sistemas orientados a objetos. En particular, se ha adoptado ampliamente el **Proceso Unificado de Rational** o **RUP** (*Rational Unified Process*) [Kruchten00], un refinamiento detallado del Proceso Unificado.

El Proceso Unificado (UP) combina las prácticas comúnmente aceptadas como “buenas prácticas”, tales como ciclo de vida iterativo y desarrollo dirigido por el riesgo, en una descripción consistente y bien documentada. Por tanto, se utiliza en este libro como ejemplo de proceso para introducir el A/DOO.

Este libro comienza con una introducción al UP por dos motivos:

1. El UP es un proceso *iterativo*. El desarrollo iterativo es una práctica de gran valor que influye en el modo en el que se introduce el A/DOO en este libro y en cómo debe ser aplicado.
2. Las prácticas del UP proporcionan una estructura organizada de ejemplo para discutir sobre cómo hacer —y cómo aprender— el A/DOO.

Este texto proporciona una introducción al UP, no lo aborda por completo. Se centra en las ideas y artefactos comunes relacionados con una introducción al A/DOO y el análisis de requisitos.

¿Y si no me interesa el UP?

El UP se utiliza como ejemplo de proceso con el que explorar el análisis de requisitos y el A/DOO, puesto que es necesario introducir el tema en el contexto de algún proceso, y el uso del UP (o el refinamiento del RUP) está relativamente bastante extendido. Además, el UP presenta actividades comunes y buenas prácticas. No obstante, las ideas centrales de este libro —como casos de uso y patrones de diseño— son independientes de cualquier proceso particular, y pueden aplicarse a muchos.

2.1. La idea más importante del UP: desarrollo iterativo

El UP fomenta muchas buenas prácticas, pero una destaca sobre las demás: el **desarrollo iterativo**. En este enfoque, el desarrollo se organiza en una serie de mini-proyectos cortos, de duración fija (por ejemplo, cuatro semanas) llamados **iteraciones**; el resultado de cada uno es un sistema que puede ser probado, integrado y ejecutado. Cada iteración incluye sus propias actividades de análisis de requisitos, diseño, implementación y pruebas.

El ciclo de vida iterativo se basa en la ampliación y refinamiento sucesivos del sistema mediante múltiples iteraciones, con retroalimentación cíclica y adaptación como elementos principales que dirigen para converger hacia un sistema adecuado. El sistema crece incrementalmente a lo largo del tiempo, iteración tras iteración, y por ello, este enfoque también se conoce como **desarrollo iterativo e incremental** (ver Figura 2.1).

Las primeras ideas sobre procesos iterativos se conocieron como desarrollo en espiral y desarrollo evolutivo [Boehm88, Gilb88].

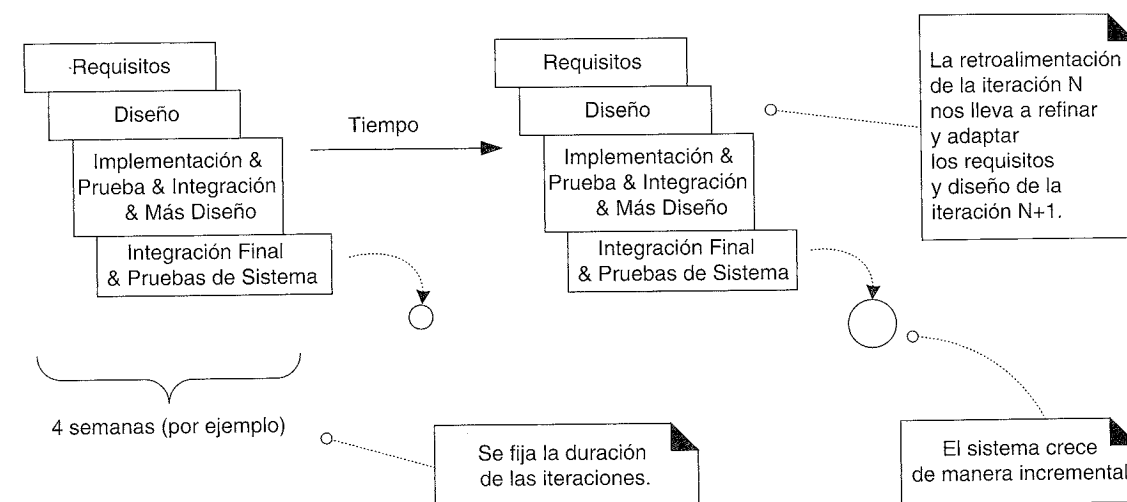


Figura 2.1. Desarrollo iterativo e incremental.

Ejemplo:

Como ejemplo (no como receta), en una iteración de dos semanas, a mitad de camino en un proyecto, quizás el lunes se dedique a distribuir y clarificar las tareas y requisitos de la iteración, mientras que una persona hace ingeniería inversa pasando el código de la última iteración a diagramas UML (mediante una herramienta CASE), e imprime y muestra los diagramas interesantes. El martes se dedica a diseñar por parejas en las pizarras, dibujando diagramas UML imprecisos que se capturan con cámaras digitales, y a escribir algo de pseudocódigo y notas de diseño. Durante los ocho días restantes, se dedica el tiempo a implementar, probar (unidad, aceptación, usabilidad...), ampliar el diseño, integrar, llevar a cabo construcciones diarias, pruebas de sistema y estabilización del sistema parcial. Otras actividades comprenden presentaciones y evaluaciones con el personal involucrado en el proyecto (*stakeholders*¹), y planificación de la siguiente iteración.

Nótese que en este ejemplo no hay prisa por codificar, ni una etapa de diseño más larga de lo que uno piensa que debería ser, en la que se pretende perfeccionar todos los detalles del diseño antes de programar. Se anticipa “un poco” del diseño con modelado basado en diagramas, utilizando dibujos UML realizados con rapidez y a grandes rasgos; quizás los desarrolladores dedican medio día, o un día entero, a trabajar diseñando en parejas.

El resultado de cada iteración es un sistema ejecutable, pero incompleto; no está preparado para ser puesto en producción. El sistema podría no estar listo para su puesta en producción hasta después de muchas iteraciones; por ejemplo, 10 ó 15.

La salida de una iteración *no* es un prototipo experimental o desechable, y el desarrollo iterativo no es prototipado. Más bien, la salida es un subconjunto con calidad de producción del sistema final.

Aunque, en general, cada iteración aborda nuevos requisitos y amplía el sistema incrementalmente, una iteración podría, ocasionalmente, volver sobre el software que ya existe y mejorarlo; por ejemplo, una iteración podría centrarse en mejorar el rendimiento de un subsistema, en lugar de extenderlo con nuevas características.

¹ N. del T.: Este término se refiere a todas las personas que de uno u otro modo están involucradas en un proyecto (directivos, usuarios, analistas, programadores...).

Aceptando los cambios: retroalimentación y adaptación

El subtítulo de un libro que trata el desarrollo iterativo es *Aceptar el Cambio* [Beck00]. Esta frase evoca una aptitud clave del desarrollo iterativo: En lugar de luchar contra el inevitable cambio que ocurre en el desarrollo de software intentando (normalmente sin éxito) especificar, congelar y “firmar” de manera completa y correcta a partir de un conjunto de requisitos fijos y diseñar antes de implementar, el desarrollo iterativo se basa en una aptitud de aceptación del cambio y la adaptación como motores inevitables y, de hecho, esenciales.

Esto no quiere decir que el desarrollo iterativo y el UP fomenten un proceso dirigido por “una adición de características” de manera incontrolada y reactiva. Los siguientes capítulos explorarán cómo el UP llega a un equilibrio entre la necesidad —por un lado— de llegar a un acuerdo y estabilizar un conjunto de requisitos, y —por otro lado— la realidad de los requisitos cambiantes, cuando el personal involucrado clarifica su visión o cambia el mercado.

Cada iteración conlleva la elección de un pequeño conjunto de requisitos y, rápidamente, diseñar, implementar y probar. En las primeras iteraciones, la elección de los requisitos y el diseño podrían no ser exactamente lo que se desea al final. Pero el acto de dar un pequeño paso con rapidez, antes de capturar todos los requisitos y que el diseño completo se haya definido de forma especulativa, nos lleva a una rápida retroalimentación —de los usuarios, desarrolladores y pruebas (tales como pruebas de carga y usabilidad)—.

Tener retroalimentación en una etapa temprana vale su peso en oro; más que las *especulaciones* sobre los requisitos y diseños correctos, la retroalimentación, a partir de la construcción y prueba realista de algo, aporta un conocimiento práctico y crucial, y una oportunidad de modificar o adaptar la comprensión de los requisitos o el diseño. Los usuarios finales tienen la oportunidad de ver rápidamente una parte del sistema y decir: “Sí, esto es lo que pedí, pero ahora que lo pruebo, lo que realmente quiero es algo un poco distinto”². Este proceso de “sí... pero” no es un signo de fallo; sino, ciclos estructurados frecuentes y tempranos de “sí... peros”, son un modo habilidoso de hacer progresar y descubrir qué es lo que tiene un valor real para el personal involucrado. Si, como se ha mencionado, esto no es un consentimiento, un desarrollo caótico y reactivo en el que los desarrolladores cambien continuamente de dirección —es posible llegar a un término medio—.

Además de clarificar los requisitos, actividades como la prueba de carga probarán si el diseño y la implementación parcial están en el camino correcto, o si en la siguiente iteración, se necesita un cambio en la arquitectura básica. Cuanto antes se resuelvan y *prueben* las decisiones de diseño críticas y arriesgadas mejor —el desarrollo iterativo proporciona los mecanismos para esto—.

En consecuencia, el trabajo se desarrolla a lo largo de una serie de ciclos estructurados de construir-retroalimentar-adaptar. No sorprende que la desviación del sistema del

² O más probable: “¡No entendí lo que quería!”

“verdadero camino” (en términos de sus requisitos y diseño finales) en las primeras iteraciones será mayor que en las últimas. A lo largo del tiempo, el sistema converge hacia este camino, como se ilustra en la Figura 2.2.

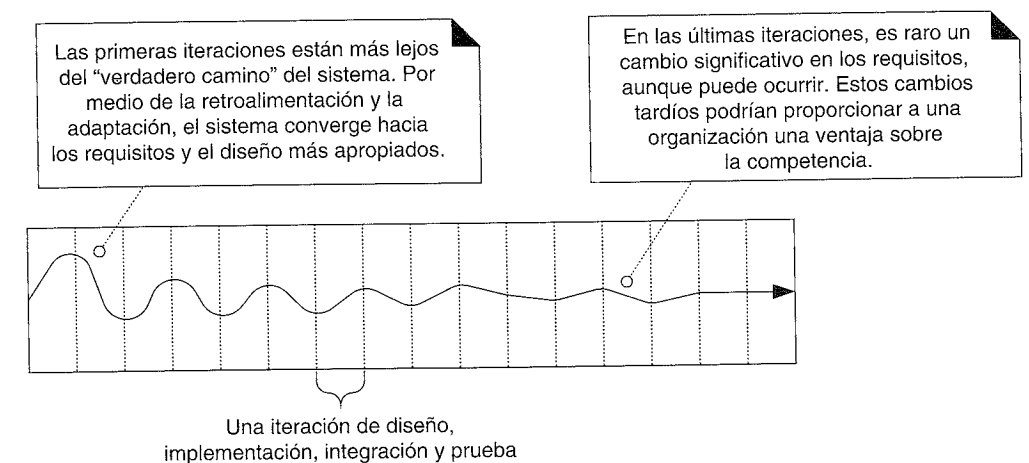


Figura 2.2. La retroalimentación iterativa y la adaptación nos conducen hacia el sistema deseado. La inestabilidad de los requisitos y el diseño disminuyen a lo largo del tiempo.

Beneficios del desarrollo iterativo

Los beneficios del desarrollo iterativo incluyen:

- Mitigación tan pronto como sea posible de riesgos altos (técnicos, requisitos, objetivos, usabilidad y demás)
- Progreso visible en las primeras etapas.
- Una temprana retroalimentación, compromiso de los usuarios y adaptación, que nos lleva a un sistema refinado que se ajusta más a las necesidades reales del personal involucrado.
- Gestión de la complejidad; el equipo no se ve abrumado por la “parálisis del análisis” o pasos muy largos y complejos.
- El conocimiento adquirido en una iteración se puede utilizar metódicamente para mejorar el propio proceso de desarrollo, iteración a iteración.

Longitud de una iteración y fijación de la duración

El UP (y desarrolladores con experiencia en aplicar procesos iterativos) recomiendan que la longitud de una iteración sea de dos a seis semanas. Pasos pequeños, rápida retroalimentación, y adaptación son las ideas fundamentales del desarrollo iterativo; iteraciones largas destruyen la motivación principal del desarrollo iterativo e incrementan el riesgo del proyecto. Menos de dos semanas y es difícil completar el trabajo suficiente para obtener resultados significativos y retroalimentación; más de seis u ocho semanas, y la complejidad se hace bastante abrumadora, y se retrasa la retroalimentación. Con iteraciones muy largas pierde su sentido el desarrollo iterativo. Lo corto es bueno.

Una idea clave es que se **fija la duración** de las iteraciones. Por ejemplo, si se elige que la siguiente iteración dure cuatro semanas, entonces el sistema parcial debería integrarse, probarse y estabilizarse en la fecha planificada —los retrasos son frustrantes—. Si parece que será difícil cumplir con el plazo fijado, la respuesta recomendada es eliminar tareas o requisitos de la iteración, e incluirlos en una iteración posterior, más que retrasar la fecha de terminación prevista. El Capítulo 37 resume los motivos para fijar la duración.

Equipos muy grandes (por ejemplo, varios cientos de desarrolladores) podrían requerir iteraciones de más de seis semanas para compensar los costes fijos de coordinación y comunicación; pero no se recomienda que sea más de tres a seis semanas. Por ejemplo, la exitosa sustitución de los noventa del sistema de control aéreo canadiense, se desarrolló siguiendo un ciclo de vida iterativo y otras prácticas del UP. Necesitó 150 programadores que se organizaron en iteraciones de seis meses³. Pero, nótese que incluso en el caso de una iteración de proyecto global de seis meses, un equipo encargado de un subsistema formado por 10 ó 20 desarrolladores puede dividir su trabajo en una serie de seis iteraciones de un mes.

Una iteración de seis meses es la excepción, en el caso de grandes equipos, no la regla. Reiterando lo dicho, el UP recomienda que la duración de una iteración sea entre dos y seis semanas.

2.2. Conceptos y buenas prácticas del UP adicionales

La idea fundamental para apreciar y utilizar el UP es el desarrollo iterativo, fijando iteraciones cortas, y adaptable.

Otra idea del UP implícita, pero muy importante, es el uso de las tecnologías de objetos, entre las que se encuentra el A/DOO y la programación orientada a objetos.

Algunos conceptos claves y buenas prácticas del UP son:

- Abordar cuestiones de alto riesgo y muy valiosas en las primeras iteraciones
- Involucrar continuamente a los usuarios para evaluación, retroalimentación y requisitos
- Construir en las primeras iteraciones una arquitectura que constituya un núcleo central consistente
- Verificar la calidad continuamente; pruebas muy pronto, con frecuencia y de manera realista
- Aplicar casos de uso
- Modelar software visualmente (con UML)
- Gestionar los requisitos con cuidado
- Manejar peticiones de cambio y gestión de configuraciones

El lector puede acudir al Capítulo 37 para una descripción más detallada de estas prácticas.

³ Philippe Kruchten, quien también dirigió el desarrollo del RUP, trabajó como arquitecto jefe en el proyecto.

2.3. Las fases del UP y términos orientados a la planificación

Un proyecto UP organiza el trabajo y las iteraciones en cuatro fases fundamentales:

- 1. Inicio:** visión aproximada, análisis del negocio, alcance, estimaciones imprecisas.
- 2. Elaboración:** visión refinada, implementación iterativa del núcleo central de la arquitectura, resolución de los riesgos altos, identificación de más requisitos y alcance, estimaciones más realistas.
- 3. Construcción:** implementación iterativa del resto de requisitos de menor riesgo y elementos más fáciles, preparación para el despliegue.
- 4. Transición:** pruebas beta, despliegue.

Estas fases se definen de una manera más completa en los capítulos siguientes.

Esto *no* se corresponde con el antiguo ciclo de vida “en cascada” o secuencial, en el que primero se definían todos los requisitos y, después, se realizaba todo, o la mayoría, del diseño.

La fase de Inicio no es una fase de requisitos; sino una especie de fase de viabilidad, donde se lleva a cabo sólo el estudio suficiente para decidir si continuar o no.

De igual modo, la fase de Elaboración no es la fase de requisitos o de diseño; sino que es una fase donde se implementa, de manera iterativa, la arquitectura que constituye el núcleo central y se mitigan las cuestiones de alto riesgo.

La Figura 2.3 ilustra los términos orientados a la planificación comunes del UP. Nótese que un ciclo de desarrollo (que termina con el lanzamiento de un sistema a producción) se compone de muchas iteraciones.

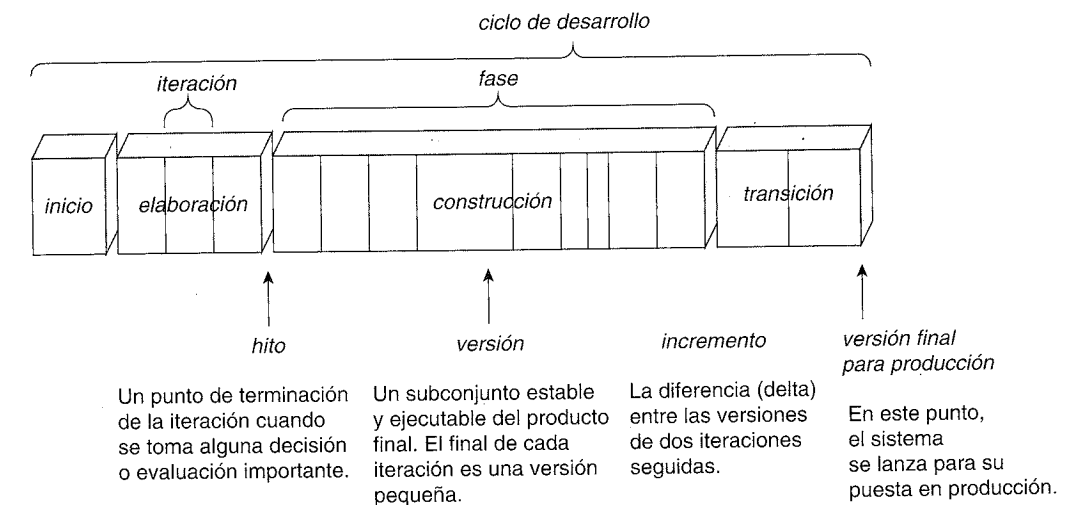


Figura 2.3. Términos orientados a la planificación en el UP.

2.4. Las disciplinas del UP (eran flujos de trabajo)

El UP describe actividades de trabajo, como escribir casos de uso, en **disciplinas** (llamadas originalmente **flujos de trabajo**)⁴. Informalmente, una disciplina es un conjunto de actividades (y artefactos relacionados) en un área determinada, como las actividades en el análisis de requisitos. En el UP, un **artefacto** es el término general para cualquier producto del trabajo: código, gráficos Web, esquema de base de datos, documentos de texto, diagramas, modelos, etcétera.

Hay varias disciplinas en el UP; este libro se centra en algunos artefactos de las siguientes tres:

- **Modelado del Negocio.** Cuando se desarrolla una única aplicación, esto incluye el modelado de los objetos del dominio. Cuando se está haciendo análisis del negocio a gran escala o reingeniería de procesos del negocio, esto incluye el modelado dinámico de los procesos del negocio de toda la empresa.
- **Requisitos.** Análisis de los requisitos para una aplicación, como escritura de casos de uso e identificación de requisitos no funcionales.
- **Diseño.** Todos los aspectos de diseño, incluyendo la arquitectura global, objetos, bases de datos, red y cosas parecidas.

Una lista más extensa de las disciplinas del UP se muestra en la Figura 2.4

En el UP, **Implementación** significa programar y construir el sistema, no despliegue. La disciplina **Entorno** se refiere a establecer las herramientas y adaptar el proceso al proyecto —esto es organizar la herramienta y el entorno del proceso—.

Disciplinas y fases

Como se ilustra en la Figura 2.4, durante una iteración, el trabajo se desarrolla en la mayoría o todas las disciplinas. Sin embargo, el esfuerzo relativo en estas disciplinas cambia a lo largo del tiempo. Las primeras iteraciones, naturalmente, tienden a aplicar un esfuerzo relativo mayor a los requisitos y al diseño, y en las posteriores disminuye, cuando los requisitos y el diseño central se estabilizan, mediante un proceso de retroalimentación y adaptación.

Relacionando esto con las fases del UP (inicio, elaboración...), la Figura 2.5 muestra el esfuerzo relativo de cambio con respecto a las fases; por favor, nótese que son una sugerencia, no es literal. En la elaboración, por ejemplo, las iteraciones tienden a tener un nivel relativamente alto de trabajo de requisitos y diseño, aunque, sin ninguna duda, también algo de implementación. Durante la construcción, se le da una importancia mayor a la implementación y más ligera al análisis de requisitos.

⁴ En el 2001, el antiguo término del UP “flujo de trabajo” se substituyó por el nuevo término “disciplina” para armonizar con un esfuerzo de estandarización internacional denominado OMG SPEM; debido al significado anterior en el UP, muchos continúan utilizando el término flujo de trabajo para referirse a disciplina, aunque no es estrictamente correcto. El término “flujo de trabajo” comenzó a tener un significado nuevo pero ligeramente distinto en el UP: en un proyecto particular, es una secuencia particular de actividades (quizás *entre* disciplinas) —un flujo de trabajo—.

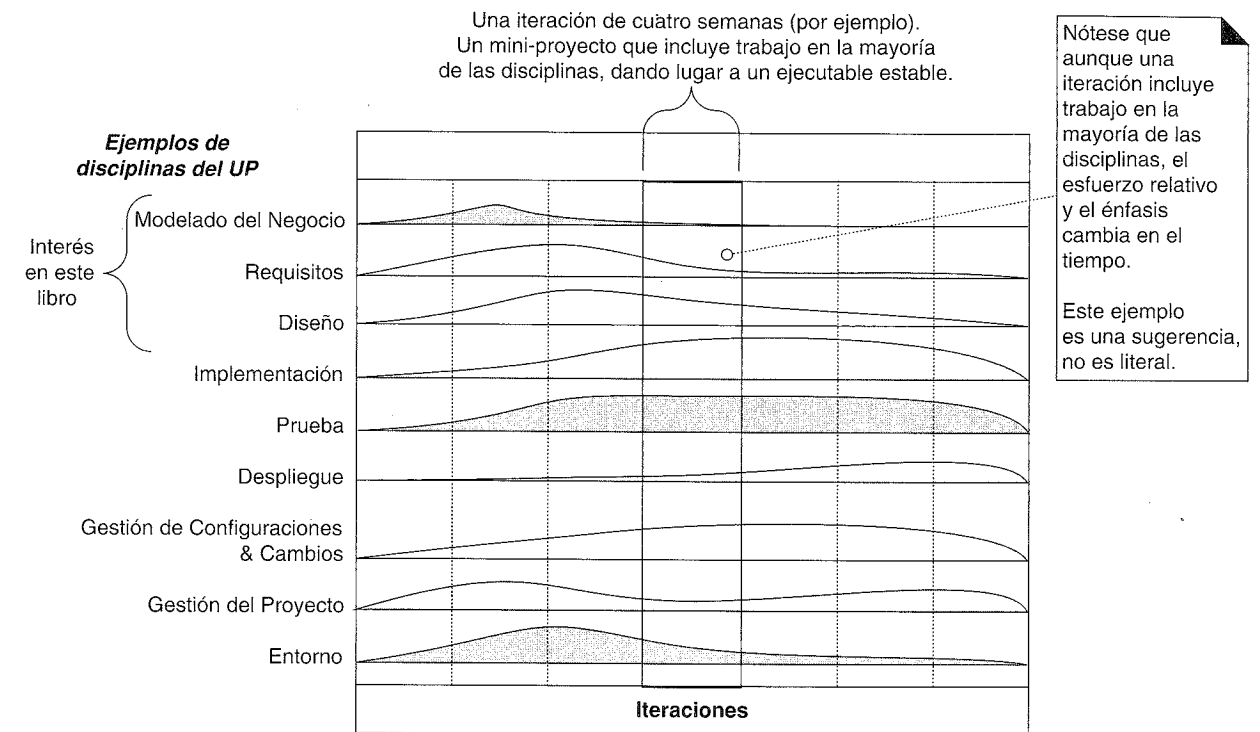


Figura 2.4. Disciplinas del UP⁵.

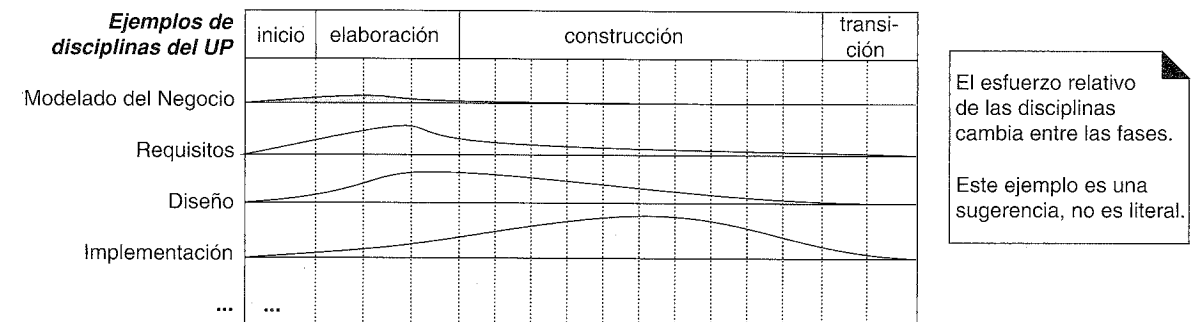


Figura 2.5. Disciplinas y fases.

Estructura del libro y fases y disciplinas del UP

Con respecto a las fases y disciplinas, ¿cuál es el enfoque del caso de estudio?

Respuesta:

El caso de estudio presta una atención especial a las fases de inicio y elaboración. Se centra en algunos artefactos de las disciplinas del Modelado del Negocio, Requisitos y Diseño, donde se aplican, fundamentalmente, el análisis de requisitos, el A/DOO, los patrones y UML.

⁵ Diagrama adaptado del producto RUP.

Los primeros capítulos presentan las actividades de la fase de inicio; los capítulos posteriores exploran varias iteraciones en la fase de elaboración. La siguiente lista y la Figura 2.6 describen la organización con respecto a las fases del UP.

- 1. Los capítulos de la fase de inicio introducen los temas esenciales del análisis de requisitos.
- 2. La iteración 1 presenta los fundamentos del A/DOO y cómo asignar reponsabilidades a los objetos.
- 3. La iteración 2 está enfocada al diseño de objetos, en concreto a introducir algunos de los “patrones de diseño” más utilizados.
- 4. La iteración 3 presenta una variedad de temas, como el análisis de la arquitectura y el diseño de frameworks.

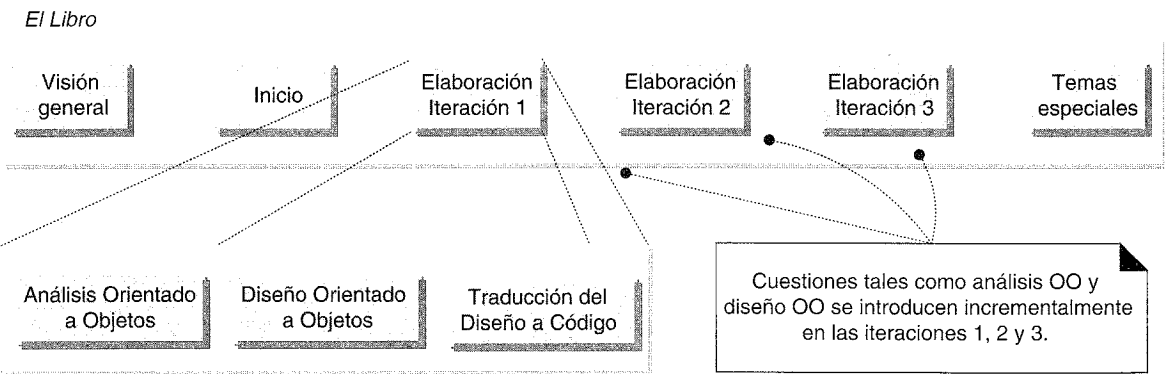


Figura 2.6. La organización del libro está relacionada con las fases e iteraciones del UP.

2.5. Adaptación del proceso y el Marco de Desarrollo

Artefactos opcionales

Algunas prácticas y principios del UP deben seguirse siempre, como el desarrollo iterativo y dirigido por el riesgo, y la verificación continua de la calidad.

Sin embargo, es importante saber que en el UP todas las actividades y artefactos (modelos, diagramas, documentos...) son *opcionales* —bueno, ¡quizás el código no!—. El conjunto de artefactos posibles del UP debería entenderse como un conjunto de medicinas en una farmacia. Exactamente igual que uno no toma medicinas indiscriminadamente, sino que las elige según la dolencia, en un proyecto UP, un equipo debería seleccionar un pequeño subconjunto de artefactos que sirvan para tratar sus problemas y necesidades particulares. En general, centrarse en un *pequeño* conjunto de artefactos que demuestran tener un gran valor práctico.

El Marco de Desarrollo

La elección de los artefactos del UP para un proyecto podría recogerse en un documento breve denominado **Marco de Desarrollo** (un artefacto en la disciplina Entorno). Por

ejemplo, la Tabla 2.1 podría ser el Marco de Desarrollo que describe los artefactos para el caso de estudio “Proyecto NuevaEra” presentado en este libro.

Los capítulos siguientes describen la creación de algunos de estos artefactos como el Modelo del Dominio, Modelo de Casos de Uso y el Modelo de Diseño.

Los artefactos de ejemplo que se presentan en este caso de estudio de ningún modo son suficientes, o adecuados, para todos los proyectos. Por ejemplo, un sistema de control de una máquina encontraría útil realizar muchos diagramas de estados. Un sistema de comercio electrónico basado en el web podría centrar su atención en los prototipos de interfaz de usuario. Un proyecto de desarrollo nuevo, en un campo en el que no se tenga experiencia tendría necesidades muy diferentes, en cuanto a los artefactos de diseño, de las de un proyecto de integración de sistemas.

Tabla 2.1 Ejemplo de Marco de Desarrollo de artefactos UP. c-comenzar; r-refinar

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c	r	
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

2.6. El UP ágil

Los metodologistas distinguen entre procesos pesados y ligeros, y procesos predictivos y adaptables. Un **proceso pesado** es un término peyorativo, que pretende sugerir un proceso con las siguientes cualidades [Fowler00]:

- Muchos artefactos creados en un ambiente burocrático.
- Rigidez y control.
- Planificación detallada, muy larga y elaborada.
- Predictivo más que adaptable.

Un **proceso predictivo** es aquel que intenta planificar y predecir en detalle las actividades y asignación de recursos (personal) en un intervalo relativamente largo de tiempo, tal como la totalidad de un proyecto. Los procesos predictivos normalmente si-

guen un ciclo de vida en “cascada” o secuencial —primero, definición de todos los requisitos; segundo, definición del diseño detallado; y tercero, implementación—. Frente a esto, un **proceso adaptable** es aquel que acepta el cambio como motor inevitable y fomenta la adaptación flexible; normalmente siguen un ciclo de vida iterativo. Un **proceso ágil** implica un proceso adaptable y ligero, listo para responder rápidamente a las necesidades cambiantes.

La intención de los autores del UP no era que fuese pesado o predictivo, aunque su amplio conjunto de actividades y artefactos opcionales, comprensiblemente, ha llevado a algunos a tener esta impresión. Más bien, la intención fue que se adoptara y aplicara con el espíritu de un proceso ágil —UP ágil—. Algunos ejemplos de cómo se pone esto en práctica:

- Optar por un conjunto pequeño de actividades y artefactos del UP. Algunos proyectos se beneficiarán más que otros, pero, en general, manténgalo simple.
- Puesto que el UP es iterativo, no se completan los requisitos y diseños antes de la implementación. Surgen de modo adaptable a lo largo de una serie de iteraciones, basadas en la retroalimentación.
- No hay un plan *detallado* para todo el proyecto. Hay un plan de alto nivel (denominado el **Plan de Fase**) que estima la fecha de terminación del proyecto y otros hitos importantes, pero no detalla los pasos de grano fino de estos hitos. Un plan detallado (llamado **Plan de Iteración**) sólo planifica con gran detalle una iteración por adelantado. La planificación detallada se lleva a cabo de manera adaptable de iteración en iteración. En el Capítulo 36 se pueden encontrar algunos comentarios sobre la planificación de proyectos iterativos y la justificación de este enfoque.

El caso de estudio hace hincapié en un número relativamente pequeño de artefactos y en el desarrollo iterativo, en el espíritu de un UP ágil.

2.7. El ciclo de vida “en cascada” secuencial

A diferencia del ciclo de vida iterativo del UP, una antigua alternativa era el ciclo de vida “en cascada”, lineal o secuencial [Royce70]. En su acepción habitual, definía los pasos más o menos de la siguiente forma:

1. Determinar, registrar y acordar un conjunto de requisitos completo y fijo.
2. Diseñar un sistema basado en estos requisitos.
3. Implementar en base al diseño.

Un estudio de dos años, presentado en el *MIT Sloan Management Review*, sobre los proyectos software con éxito, identificó cuatro factores comunes para el éxito; el primero de la lista era el desarrollo iterativo, en lugar de un proceso en cascada [MacCormack01]⁶.

⁶ Los otros eran: 2) al menos una incorporación diaria de nuevo código para la construcción del sistema completo, y rápida retroalimentación de los cambios del diseño (a través de las pruebas); 3) equipo con experiencia en expedir múltiples productos; y 4) centrarse pronto en la construcción y prueba de una arquitectura consistente. Tres de estos cuatro factores son prácticas explícitas en el UP.

Una descripción breve de estos problemas, y cómo pueden mitigarse mediante el desarrollo iterativo, se presentará en el Capítulo 37.

2.8. No se entendió el UP cuando...

Aquí presentamos algunos signos que indican que no se ha entendido lo que significa adoptar el UP y el desarrollo iterativo con la idea de agilidad que se pretende en el UP.

- Se piensa que inicio = requisitos, elaboración = diseño, y construcción = implementación (esto es, imponiendo un ciclo de vida en espiral sobre el UP).
- Se piensa que el objetivo de la elaboración es definir modelos de manera completa y cuidadosa, los cuales se traducen a código durante la construcción.
- Se intenta definir la mayoría de los requisitos antes de comenzar el diseño o la implementación.
- Se intenta definir la mayoría del diseño antes de comenzar a implementar; se intenta definir completamente y acordar una arquitectura antes de programar y probar iterativamente.
- Se dedica mucho tiempo a realizar trabajo sobre los requisitos y el diseño antes de comenzar a programar.
- Se cree que una iteración adecuada es de cuatro meses de duración, en lugar de cuatro semanas (excluyendo proyectos de cientos de desarrolladores).
- Se piensa que realizar los diagramas UML y las actividades de diseño constituyen el momento para definir diseños y modelos de manera completa y precisa con gran detalle, y se cree que programar es una simple traducción mecánica de éstos en código.
- Se piensa que adoptar el UP significa hacer muchas de las actividades posibles y crear muchos documentos, y se piensa o experimenta el UP como un proceso formal y exigente con muchos pasos que seguir.
- Se intenta planificar un proyecto en detalle desde el principio hasta el final; intenta predecir, de manera especulativa, todas las iteraciones y lo que debería ocurrir en cada una de ellas.
- Se quieren planes y estimaciones creíbles para los proyectos antes de que termine la fase de elaboración.

2.9. Lecturas adicionales

Una introducción al UP y su refinamiento en el RUP, que merece la pena leer es *The Rational Unified Process-An Introduction* de Philippe Kruchten, el arquitecto líder del RUP.

Se puede encontrar una descripción del UP original en *The Unified Software Development Process* de Jacobson, Booch y Rumbaugh. Merece la pena estudiarlo, pero se recomienda primero la introducción de Kruchten puesto que es más pequeña y concisa, y el RUP actualiza y refina el original UP.

Rational Software vende un producto basado en el web con la documentación del RUP on-line, el cual permite conocer con todo detalle todas las actividades y artefactos del RUP y las plantillas para la mayoría de los artefactos. En el Capítulo 37 se presentará una breve discusión. Una organización puede abordar un proyecto UP utilizando únicamente mentores y libros como fuentes de aprendizaje, pero algunos encuentran la documentación del RUP una ayuda útil para el aprendizaje y el proceso.

Las actividades del UP también se describen de manera poco estricta en una serie de libros editados por Ambler y Constantine (por ejemplo, *The Unified Process: Elaboration Phase* [Ambler00]). Estos libros contienen artículos publicados a lo largo de los años en la revista *Software Development*, clasificados en sus respectivas fases y actividades en términos de una taxonomía del UP. Nótese que los artículos no se escribieron originalmente para el UP, aunque, sin lugar a dudas contienen consejos útiles. También se advierte un pequeño error en las series: describen la fase de elaboración del UP como una fase en la que se crean prototipos desechables, de este modo reduce la necesidad para prestar atención a cuidar la programación o el diseño. Esto no es exacto; durante la elaboración se crean diseños y código con calidad de producción (aunque parcial). Ambler reconoce la imprecisión y puede corregirlo en las ediciones siguientes⁷.

Para obtener información sobre otros métodos ágiles se recomienda la serie de libros **Extreme Programming** (XP) [Beck00, BF00, JAH00], tales como *Extreme Programming Explained*. Algunas de las prácticas de XP se mencionan en capítulos posteriores. La mayoría de las prácticas de XP (tales como programar probando primero y desarrollo iterativo) son compatibles —o idénticas— a las prácticas del UP, y recomendando su adopción en un proyecto UP. Nótese que XP no inventó (ni pidió que se hiciese) el desarrollo iterativo, con iteraciones cortas de duración fija, y adaptable, que ha sido una práctica del UP y otros métodos iterativos durante años. Dos diferencias destacables —no es una lista completa— entre el UP y XP son: 1) el UP recomienda escribir de manera incremental los casos de uso y un documento de requisitos no funcionales (XP no); y, 2) el UP recomienda dedicar más tiempo a realizar los diagramas del diseño visual (como medio día o un día entero) cerca del comienzo de una iteración, antes de la programación. Los líderes de XP recomiendan muy poco, unos 30 minutos.

Highsmith justifica el valor del desarrollo iterativo en *Adaptive Software Development* [Highsmith00].

⁷ Ambler, comunicación privada.

CASO DE ESTUDIO: EL SISTEMA DE PUNTO DE VENTA NUEVAERA

Pocas cosas son más difíciles de soportar que un buen ejemplo.

Mark Twain

Introducción

Este capítulo describe brevemente el caso de estudio. Si se conoce el dominio del problema, podría saltarse. De hecho, se eligió este problema porque es familiar, pero suficientemente rico, con problemas interesantes de arquitectura y diseño, y, por tanto, nos permite concentrarnos en cómo llevar a cabo el análisis y diseño, en lugar de explicar el problema y el dominio.

3.1. El sistema de punto de venta NuevaEra

El caso de estudio es el sistema de punto de venta (PDV) NuevaEra. En este dominio del problema, en apariencia sencillo, veremos que hay requisitos muy interesantes y problemas de diseño que solucionar. Además, es un problema real; las organizaciones realmente escriben sistemas PDV utilizando la tecnología de objetos.

Un sistema PDV es una aplicación informática utilizada (en parte) para registrar ventas y realizar pagos; normalmente se utiliza en tiendas. Incluye componentes hardware, como un ordenador y un lector de códigos de barras, y software para ejecutar el sistema. Interactúa con varias aplicaciones de servicios, como un servicio de cálculo de impuestos y un control de inventario, de terceras partes. Estos sistemas deben ser, relativamente, tolerantes a fallos; es decir, incluso si los servicios remotos no están disponibles temporalmente (como el sistema de inventario), todavía deben ser capaces de

capturar las ventas y gestionar, al menos, los pagos en efectivo (de manera que no se impida que el negocio funcione de manera adecuada).

Un sistema PDV, progresivamente, debe soportar múltiples y variados terminales e interfaces del lado del cliente, como un terminal con un navegador Web de una arquitectura “cliente delgado” (*thin client*), un ordenador personal normal con una interfaz gráfica de usuario hecha con las clases Swing de Java, entrada de datos mediante una pantalla táctil, PDAs inalámbricos, etcétera.

Además, estamos creando un sistema PDV comercial que se venderá a diferentes clientes con necesidades dispares en términos de procesamiento de reglas del negocio. Cada cliente deseará un conjunto exclusivo de lógica a ejecutar en ciertos puntos predecibles en escenarios de uso del sistema, como cuando se inicia una venta o cuando se añade una nueva línea. Por tanto, necesitaremos un mecanismo para proporcionar esta flexibilidad y personalización.

Utilizando una estrategia de desarrollo iterativo, vamos a realizar las fases de requisitos y análisis, diseño e implementación orientados a objetos.

3.2. Capas arquitectónicas y el énfasis del caso de estudio

Un sistema de información orientado a objetos típico se diseña en función de varias capas arquitectónicas o subsistemas (ver Figura 3.1). A continuación, presentamos algunos ejemplos, no una lista completa:

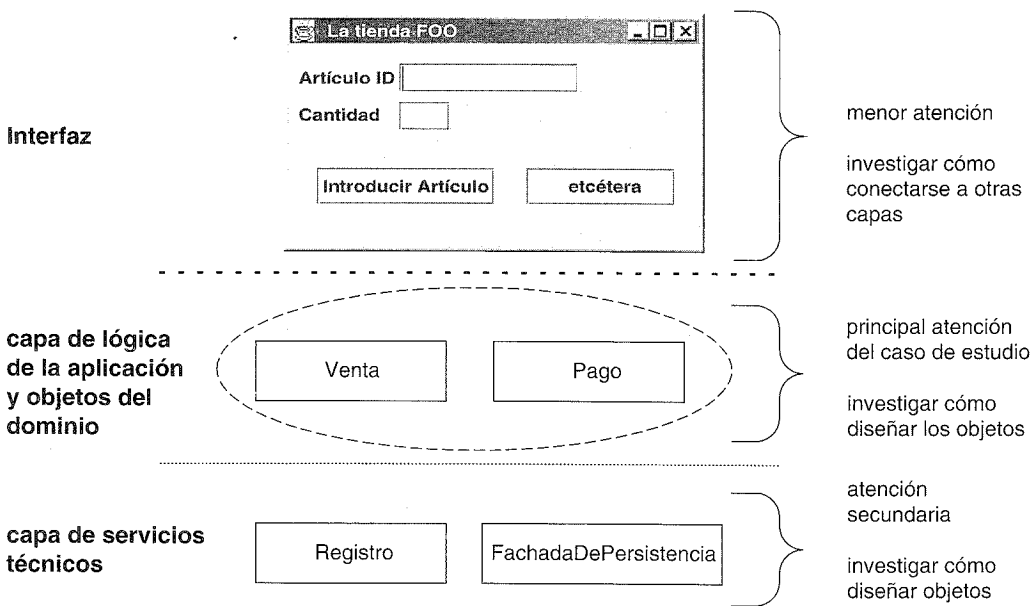


Figura 3.1. Capas y objetos de ejemplo en un sistema orientado a objetos, y el enfoque del caso de estudio.

- **Interfaz de Usuario:** interfaz gráfica; ventanas.
- **Lógica de la aplicación y Objetos del Dominio:** objetos software que representan conceptos del dominio (por ejemplo una clase software denominada *Venta*) que satisfacen los requisitos de la aplicación.
- **Servicios técnicos:** objetos de propósito general y subsistemas que proporcionan servicios técnicos de apoyo, como conexión con una base de datos o registrar los errores. Normalmente, estos servicios son independientes de la aplicación y se pueden reutilizar entre varios sistemas.

El A/DOO es, por lo general, más relevante para modelar la lógica de la aplicación y las capas de servicios técnicos.

El caso de estudio NuevaEra se centra principalmente en los objetos del dominio del problema, asignándoles responsabilidades para satisfacer los requisitos de la aplicación. El diseño orientado objetos también se aplica para crear un subsistema de servicio técnico para interactuar con una base de datos.

En este enfoque de diseño, la capa de interfaz de usuario tiene muy poca responsabilidad; se dice que es *delgada*. Las ventanas *no* contienen código que ejecute o procese la lógica de la aplicación, sino que, las peticiones de realización de tareas se envían a otras capas.

3.3. Estrategia del libro: aprendizaje y desarrollo iterativo

Este libro se organiza para seguir una estrategia de desarrollo iterativo. El A/DOO se aplica al sistema PDV NuevaEra en varias iteraciones; la primera iteración es para determinar algunas funciones básicas. Las iteraciones posteriores expanden la funcionalidad del sistema (ver Figura 3.2). Junto con el desarrollo iterativo, se introduce incrementalmente y de manera iterativa, la *presentación* de las cuestiones del análisis y diseño, la notación UML y los patrones. En la primera iteración, se presenta un núcleo básico de cuestiones de análisis, diseño y notación. La segunda iteración amplía con nuevas ideas, la notación UML y los patrones. Y, de igual modo, la tercera iteración.

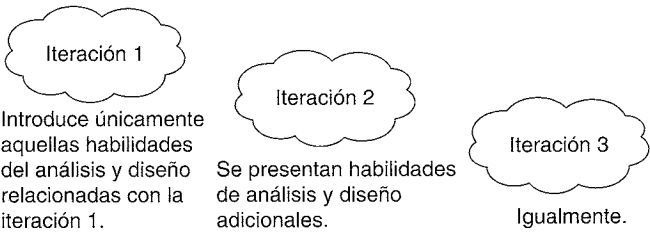
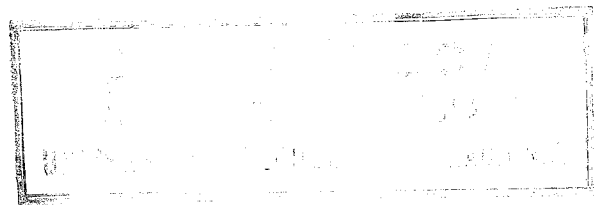


Figura 3.2. El camino de aprendizaje sigue iteraciones.



Parte 2
INICIO

Le mieux est l'ennemi du bien (Lo mejor es enemigo de lo bueno.)

Voltaire

Objetivos

- Definir la etapa de inicio.
- Motivar los capítulos siguientes de esta sección.

Introducción

Este capítulo define la fase de inicio de un proyecto. Si las ideas del proceso no son del interés del lector, o prefiere centrarse en primer lugar en aprender la actividad práctica principal de esta fase —modelado de casos de uso— entonces puede saltarse este capítulo.

La mayoría de los proyectos requieren una etapa inicial breve en la que se estudian los siguientes tipos de preguntas:

- ¿Cuál es la visión y el análisis del negocio para este proyecto?
- ¿Es viable?
- ¿Comprar y/o construir?
- Estimación aproximada del coste: ¿cuesta 10K-100K o millones de euros?
- ¿Deberíamos abordarlo o no seguir?

Para definir la visión y obtener una estimación (de la que no te puedes fiar) del orden de magnitud es necesario llevar a cabo alguna exploración de los requisitos. Sin embargo, el objetivo de la etapa de inicio no es definir todos los requisitos, o generar una estimación creíble o plan de proyecto. Aún a riesgo de simplificar demasiado, la idea es hacer la investigación justa para formar una opinión racional y justificable del propósito global y la viabilidad del nuevo sistema potencial, y decidir si merece la pena invertir en un estudio más profundo (el objetivo de la fase de elaboración).

Por tanto, la fase de inicio debería ser relativamente corta en la mayoría de los proyectos, una duración de una a unas pocas semanas. De hecho, en muchos proyectos, si la duración es superior a una semana, entonces se pierde la idea fundamental de la etapa de inicio: decidir si merece la pena una investigación seria (durante la elaboración), no llevar a cabo esta investigación.

La fase de inicio en una frase:
Vislumbrar el alcance del producto, visión y análisis del negocio.

El principal problema resuelto en una frase:
¿Está de acuerdo el personal involucrado en la visión del proyecto, y merece la pena invertir en un estudio serio?

4.1. Inicio: una analogía

En el negocio del petróleo, cuando se está considerando una nueva zona, algunos de los pasos que hay que abordar son:

- 1. Decidir si hay evidencias suficientes o un análisis del negocio que justifique perforaciones de exploración.
- 2. De ser así, llevar a cabo medidas y perforaciones de exploración.
- 3. Proporcionar información del alcance y estimación.
- 4. Pasos adicionales...

La fase de inicio es como el primer paso en esta analogía. En el primer paso, no se predice cuánto petróleo hay, o el coste o esfuerzo para extraerlo. Es prematuro —no hay suficiente información—. Aunque estaría bien ser capaz de responder a las preguntas de “cuánto” y “cuándo” sin el coste de la exploración, en el negocio del petróleo se entiende que no es realista.

En términos del UP, el paso de exploración realista se corresponde con la fase de elaboración. La fase de inicio que la precede es parecida a un estudio de viabilidad para decidir si incluso merece la pena invertir en perforaciones de exploración. Sólo después de la exploración (elaboración) tenemos los datos y los conocimientos para hacer, de algún modo, planes y estimaciones creíbles. Por tanto, en el desarrollo iterativo y el UP, los planes y estimaciones de la fase de inicio no deben considerarse fiables. Simplemente proporcionan una percepción del orden de magnitud del grado de esfuerzo, para ayudar en la decisión de continuar o no.

4.2. La fase de inicio podría ser muy breve

El propósito de la fase de inicio es establecer una visión común inicial de los objetivos del proyecto, determinar si es viable y decidir si merece la pena llevar a cabo algunas investigaciones serias en la fase de elaboración. Si se ha decidido de antemano que el proyecto se hará sin ninguna duda, y es claramente viable (quizás porque el equipo ha desarrollado proyectos parecidos antes), entonces la fase de inicio será especialmente breve. Podría incluir los primeros talleres de requisitos¹, planificación de la primera iteración y, entonces, rápidamente, cambiar a la elaboración.

4.3. ¿Qué artefactos podrían crearse en la fase de inicio?

La Tabla 4.1 presenta un listado de los artefactos comunes de la fase de inicio (o principio de la elaboración) e indica las cuestiones que deben abordarse. Los capítulos siguientes estudiarán algunos de ellos con más detalle, especialmente el Modelo de Ca-

Tabla 4.1. Ejemplo de artefactos de la fase de inicio.

Artefacto†	Comentario
Visión y Análisis del Negocio	Describe los objetivos y las restricciones de alto nivel, el análisis del negocio y proporciona un informe para la toma de decisiones.
Modelo de Casos de Uso	Describe los requisitos funcionales y los no funcionales relacionados.
Especificación Complementaria	Describe otros requisitos.
Glosario	Terminología clave del dominio.
Lista de Riesgos & Plan de Gestión del Riesgo darles respuesta.	Describe los riesgos del negocio, técnicos, recursos, planificación, y las ideas para mitigarlos o
Prototipos y pruebas-de-conceptos	Para clarificar la visión y validar las ideas técnicas.
Plan de Iteración	Describe qué hacer en la primera iteración de la elaboración.
Fase Plan de & Plan de Desarrollo de Software	Estimación de poca precisión de la duración y esfuerzo de la fase de elaboración. Herramientas, personas, formación y otros recursos.
Marco de Desarrollo	Una descripción de los pasos del UP y los artefactos adaptados para este proyecto. El UP siempre se debe adaptar al proyecto.

† Estos artefactos se completan sólo parcialmente en esta fase. Se refinarán de manera iterativa en las siguientes iteraciones. El nombre en mayúsculas indica que es un artefacto UP con ese nombre oficial.

¹ N. del T.: Se ha traducido “requirements workshop” por “taller de requisitos” para indicar una reunión de discusión sobre requisitos.

sos de Uso. Una idea clave con respecto al desarrollo iterativo es comprender que estos artefactos sólo se completan parcialmente en esta fase, se refinarán en iteraciones posteriores, e incluso no deberían crearse a menos que se considere probable que añadirán valor práctico real. Y, puesto que estamos en el inicio, la investigación y el contenido de los artefactos deberían ser ligeros.

Por ejemplo, el Modelo de Casos de Uso (que se describirá en los capítulos siguientes) podría listar los *nombres* de la mayoría de los casos de uso y actores esperados, pero quizás sólo describiría en detalle el 10% de los casos de uso —hecho con el propósito de desarrollar una visión de alto nivel y sin detalles del alcance, objetivo y riesgos del sistema.

Nótese que en la fase de inicio se podrían realizar algunas tareas de programación con el objeto de crear prototipos de “pruebas de conceptos”, para clarificar unos pocos requisitos mediante (generalmente) prototipos orientados a la interfaz de usuario, y hacer experimentos de programación para cuestiones técnicas críticas.

¿No es eso mucha documentación?

Hay que recordar que los artefactos se deberían considerar opcionales. Se deben elegir sólo aquellos que realmente añadan valor al proyecto, y desecharlos si no se prueba que merezcan la pena.

Lo importante de un artefacto no es el documento o el diagrama en sí mismo, sino el pensamiento, análisis y disposición activa (y entonces se registra, para evitar re-invencciones o tener que repetir las cosas de palabra). Como dijo el general Eisenhower: “Al preparar una batalla siempre he encontrado que los planes no son útiles, pero planificar indispensable” [Nixon90, BF00].

Hay que almacenar los artefactos digitalmente y on-line —disponibles en el sitio web del proyecto— en lugar de en papel.

Obsérvese también que los artefactos del UP de los proyectos anteriores se pueden utilizar en otros posteriores. Es normal que haya muchas similitudes entre los proyectos en cuanto a los artefactos de riesgos, gestión del proyecto, pruebas y entorno. Todos los proyectos UP organizarán (o deberían organizar) los artefactos de la misma manera, con los mismos nombres (Lista de Riesgos, Marco de Desarrollo, etc.). Esto simplifica la búsqueda de artefactos reutilizables de los proyectos precedentes en las nuevas aplicaciones del UP.

4.4. No se entendió la fase de inicio cuando...

- La duración es mayor de “unas pocas” semanas en la mayoría de los proyectos.
- Se intenta definir la mayoría de los requisitos.
- Se espera que los planes y estimaciones sean fiables.
- Se define la arquitectura; en lugar de hacerlo de manera iterativa en la fase de elaboración.

- Se cree que la secuencia adecuada de trabajo debería ser: 1) definición de los requisitos; 2) diseño de la arquitectura; 3) implementación.
- No hay artefacto de Análisis del Negocio o Visión.
- No se identificaron la mayoría de los nombres de los casos de uso y los actores.
- Se escribieron todos los casos de uso en detalle.
- Ninguno de los casos de uso se escribió en detalle; cuando del 10-20% se deberían escribir con detalle para obtener algún conocimiento realista del alcance del problema.

COMPRENSIÓN DE LOS REQUISITOS

*El nuestro es un mundo donde la gente no sabe lo que quiere
y está deseando atravesar el infierno para conseguirlo.*

Don Marquis

Objetivos

- Definir el modelo FURPS+.
 - Relacionar los tipos de requisitos con los artefactos del UP.
-

Introducción

No todos los requisitos se crean igual. Este capítulo presenta la clasificación de requisitos FURPS+.

Los **requisitos** son capacidades y condiciones con las cuales debe ser conforme el sistema —y más ampliamente, el proyecto [JBR99]—. El primer reto del trabajo de los requisitos es encontrar, comunicar y recordar (que normalmente significa registrar) lo que se necesita realmente, de manera que tenga un significado claro para el cliente y los miembros del equipo de desarrollo.

El UP fomenta un conjunto de buenas prácticas, una de las cuales es la *gestión de requisitos*. Esto no hace referencia a la actitud del ciclo de vida en cascada de definir completamente y estabilizar los requisitos en la primera fase del proyecto, sino más bien —en el contexto de que inevitablemente los deseos del personal involucrado son

cambiantes y poco claros— “un enfoque sistemático para encontrar, documentar, organizar y seguir la pista de los requisitos cambiantes de un sistema” [RUP]; en concreto, haciéndolo con destreza y sin ser descuidado. Fíjese en la palabra *cambiantes*; el UP acepta el cambio en los requisitos como un motor fundamental del proyecto. Otro término importante es *encontrar*; es decir, elicitar cuidadosamente mediante técnicas tales como escritura de casos de uso y talleres de requisitos.

Como se indica en la Figura 5.1, un estudio sobre los costes en proyectos reales en diferentes empresas reveló que el 37% de ellos estaban relacionados con los requisitos, de manera que las cuestiones de requisitos constituyen la principal causa de problemas [Standish94]. En consecuencia, es importante adquirir dominio en la gestión de requisitos. La respuesta del ciclo de vida en cascada a este dato sería intentar con ahínco pulir, estabilizar y fijar los requisitos antes de cualquier diseño o implementación, pero la historia demuestra que es una batalla perdida. La respuesta iterativa es utilizar un proceso que acepte el cambio y la retroalimentación como motores centrales en el descubrimiento de los requisitos.

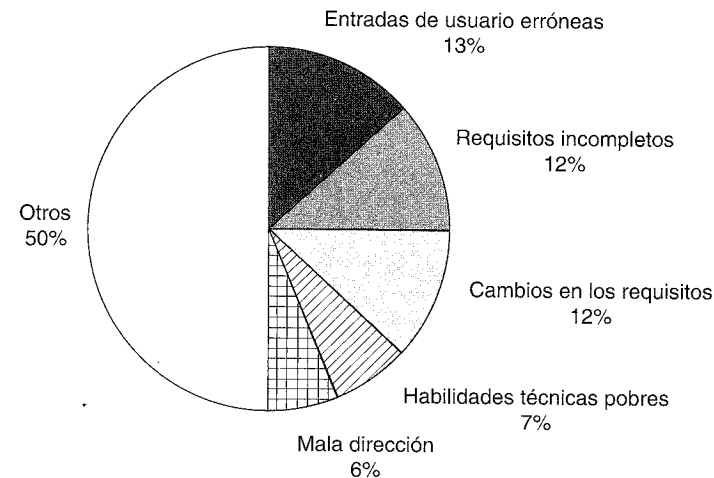


Figura 5.1. Factores del coste en proyectos software reales.

5.1. Tipos de requisitos

En el UP, los requisitos se clasifican de acuerdo con el modelo FURPS+ [Grady92], un útil nemotécnico que significa los siguientes cinco tipos de requisitos¹:

- **Funcional (*Functional*)**: características, capacidades y seguridad.
- **Facilidad de uso (*Usability*)**: factores humanos, ayuda, documentación.
- **Fiabilidad (*Reliability*)**: frecuencia de fallos, capacidad de recuperación de un fallo y grado de previsión.

¹ Hay varios sistemas de clasificación de requisitos y atributos de calidad publicados en libros y por organizaciones estándares, como el ISO 9126 (que es similar a la lista del FURPS+), y varias del Instituto de Ingeniería del Software (SEI, *Software Engineering Institute*); cualquiera de ellas se puede utilizar en un proyecto UP.

- **Rendimiento (*Performance*)**: tiempos de respuesta, productividad, precisión, disponibilidad, uso de los recursos.
- **Soporte (*Supportability*)**: adaptabilidad, facilidad de mantenimiento, internacionalización, configurabilidad.

El ‘+’ en FURPS+ indica requisitos adicionales, tales como:

- **Implementación**: limitación de recursos, lenguajes y herramientas, hardware...
- **Interfaz**: restricciones impuestas para la interacción con sistemas externos.
- **Operaciones**: gestión del sistema en su puesta en marcha.
- **Empaquetamiento**
- **Legales**: licencias, etcétera.

Resulta útil utilizar las categorías del FURPS+ (o algún esquema de clasificación) como una lista para comprobar que se cubren los requisitos, de manera que reducimos el riesgo de no considerar alguna faceta importante del sistema.

Algunos de estos requisitos se denominan colectivamente **atributos de calidad**, **requisitos de calidad**, o las “-ilities”² de un sistema. Éstos comprenden facilidad de uso (*usability*), fiabilidad (*reliability*), rendimiento (*performance*) y soporte (*supportability*). Lo normal es dividir los requisitos en **funcionales** (comportamiento) y **no funcionales** (todo lo demás); a algunos no les gusta esta amplia generalización [BCK98], pero se utiliza de manera muy extendida.

Los requisitos funcionales se estudian y recogen en el Modelo de Casos de Uso, el tema del siguiente capítulo, y en la lista de características del sistema del artefacto Visión. Los otros requisitos se pueden recoger en los casos de usos con los que están relacionados, o en el artefacto Especificación Complementaria. El artefacto Visión resume los requisitos de alto nivel que se elaboran en estos otros documentos. El Glosario agrupa y clarifica los términos que se utilizan en los requisitos. El Glosario en el UP también comprende el concepto de **diccionario de datos**, que reúne los requisitos relacionados con los datos, como reglas de validación, valores aceptables, etcétera. Los prototipos son un mecanismo para clarificar qué es lo que se quiere o es posible.

Como veremos cuando estudiemos el análisis arquitectural, los requisitos de calidad influyen fuertemente en la arquitectura de un sistema. Por ejemplo, un requisito de alto rendimiento y alta fiabilidad influirá en la elección de componentes software y hardware, y en sus configuraciones. La necesidad de fácil adaptación, debido a cambios frecuentes en los requisitos funcionales, igualmente dará forma al diseño del software.

5.2. Lecturas adicionales

En los capítulos siguientes se cubrirán referencias relacionadas con casos de uso. Se recomiendan como punto de partida para el estudio de los requisitos, textos sobre requisitos orientados a los casos de uso, como *Writing Effective Use Cases* [Cockburn01], en lugar de textos más generales (y normalmente, tradicionales).

² N. del T.: Terminación plural en inglés de los nombres de tales requisitos.

Hay un amplio debate sobre requisitos —y una extensa variedad de temas de ingeniería del software— bajo el paraguas de Cuerpo de Conocimientos en Ingeniería del Software (**SWEBOK**, *Software Engineering Body of Knowledge*) disponible en www.swebok.org.

El SEI (www.sei.cmu.edu) tiene varias propuestas relacionadas con los requisitos de calidad. El ISO 9126, el IEEE Std. 830, y el IEEE Std. 1061 son estándares relacionados con los requisitos y atributos de calidad, y están disponibles en la web en varios sitios.

Algunas advertencias con respecto a los libros generales sobre requisitos, incluso aquellos que proponen cubrir los casos de uso, desarrollo iterativo o, de hecho, incluso los requisitos en el UP:

1. La mayoría están escritos bajo la influencia de un ciclo de vida en cascada planteando la definición “precisa” de los requisitos por adelantado, antes de pasar al diseño y la implementación. Esto no pretende invalidar su valor más amplio o, a menudo, profundas y útiles visiones de los requisitos independientes del método, sino que pretende aclarar que no constituyen una visión fiable del desarrollo iterativo. Esto es debido a que la experiencia de los autores proviene principalmente de proyectos de ciclo de vida en cascada, trabajando para refinar, definir cuidadosamente y precisamente los requisitos, y terminar con la fase de requisitos antes de continuar con el diseño. Aquellos libros que también mencionan el desarrollo iterativo, lo abordan muy superficialmente, quizás añadiendo material “iterativo” apelando a las tendencias modernas. Por tanto, los libros y artículos de requisitos deberían leerse con cautela; uno podría sentirse seguro con la idea de intentar definir de manera cuidadosa todos los requisitos en la fase inicial, que no es consistente con un proceso iterativo.
2. Muchos libros generales sobre requisitos que también dicen incluir casos de uso, lo hacen muy superficialmente, o entienden mal el significado real de requisitos dirigidos por casos de uso. Esto podría deberse a que los autores poseen una experiencia de muchos años en métodos de requisitos tradicionales, y recientemente, han intentado incluir los casos de uso en sus métodos anteriores, sin darse cuenta de que la idea principal de los casos de uso, tal y como la concibe Ivar Jacobson y el UP, es hacer los casos de uso el elemento central del enfoque de requisitos global —sustituyendo a otros documentos de requisitos como elemento central—; los casos de uso impregnan y dirigen el trabajo de requisitos, en lugar de considerarse como una técnica de apoyo auxiliar a nivel inferior o medio, añadida a los documentos o enfoques de requisitos tradicionales.

En resumen, los libros generales sobre requisitos ofrecen consejos útiles sobre técnicas y cuestiones para recopilar los requisitos, escritos por personas expertas, pero normalmente, los presentan en el contexto de un proceso en cascada, sin tener un gran conocimiento de las implicaciones más profundas de los casos de uso. Cualquier consejo sobre el proceso que sea una variante de: “trate de definir la mayoría de los requisitos, y entonces pase al diseño y la implementación” no es consistente con el desarrollo iterativo y el UP.

MODELO DE CASOS DE USO: ESCRITURA DE REQUISITOS EN CONTEXTO

El primer paso indispensable para conseguir las cosas que quieres de la vida: decidir qué quieres.

Ben Stein

Objetivos

- Identificar y escribir casos de uso.
- Relacionar los casos de uso con los objetivos de los usuarios y los procesos de negocio básicos.
- Utilizar los formatos breve, informal y completo, en un estilo esencial.
- Relacionar el trabajo con casos de uso con el desarrollo iterativo.

Introducción

Merece la pena estudiar este capítulo durante la primera lectura del libro porque los casos de uso son un mecanismo ampliamente utilizado para descubrir y registrar los requisitos (especialmente los funcionales); influyen muchos aspectos de un proyecto, incluyendo el A/DOO. Merece la pena tanto saber sobre los casos de uso como crearlos.

La escritura de casos de uso —historias del uso de un sistema— es una técnica excelente para entender y describir los requisitos. Este capítulo explora los conceptos claves de los casos de uso y presenta casos de uso de ejemplo para la aplicación NuevaEra.

El UP define el **Modelo de Casos de Uso** en la disciplina Requisitos. Básicamente, es el conjunto de todos los casos de uso; es un modelo de la funcionalidad y entorno del sistema.

6.1. Objetivos e historias

Los clientes y los usuarios finales tienen objetivos (también conocidos como *necesidades*) y quieren sistemas informáticos que les ayuden a conseguirlos, que varían desde registrar las ventas hasta estimar el flujo de petróleo de futuros pozos. Hay varias formas de capturar estos objetivos y requisitos del sistema; las mejores son simples y familiares, porque esto hace que sea más fácil —especialmente para clientes y usuarios finales— contribuir a su definición o evaluación. Eso reduce el riesgo de perder el hilo.

Los casos de uso son un mecanismo para ayudar a mantenerlo simple y entendible para todo el personal involucrado. De manera informal, son historias del uso de un sistema para alcanzar los objetivos. A continuación presentamos un ejemplo de caso de uso en *formato breve*:

Procesar Venta: Un cliente llega a una caja con artículos para comprar. El cajero utiliza el sistema PDV para registrar cada artículo comprado. El sistema presenta una suma parcial y detalles de cada línea de venta. El cliente introduce los datos del pago, que el sistema valida y registra. El sistema actualiza el inventario. El cliente recibe un recibo del sistema y luego se va con los artículos.

A menudo, los casos de uso necesitan una elaboración mayor que ésta, pero la esencia es descubrir y registrar los requisitos funcionales, mediante la escritura de historias del uso de un sistema, para ayudar a cumplir los objetivos de varias de las personas involucradas; esto es, los *casos de uso*¹. Se supone que no es una idea difícil, aunque, de hecho podría ser difícil descubrir o decidir lo que es necesario, y escribirlo de manera coherente con un nivel de detalle útil.

Se ha escrito mucho acerca de los casos de uso, y si bien es útil, existe el riesgo entre las personas inteligentes y creativas, de oscurecer una idea sencilla con niveles de sofisticación. Normalmente es posible distinguir a un modelador de casos de uso novato (o a un analista serio de Tipo A) preocupándose en exceso con cuestiones secundarias como diagramas de casos de uso, relaciones de casos de uso, paquetes de casos de uso, atributos opcionales, etcétera, en lugar de escribir las historias. En otras palabras, el poder del mecanismo de casos de uso es la capacidad tanto de aumentar como de disminuir, en términos de sofisticación y formalidad, dependiendo de la necesidad.

6.2. Antecedentes

La idea de utilizar los casos de uso para describir los requisitos funcionales fue introducida en 1986 por Ivar Jacobson [Jacobson92], uno de los contribuidores principales al UML y UP. La idea de caso de uso de Jacobson ha tenido una gran influencia y ha sido ampliamente reconocida; siendo sus principales virtudes la simplicidad y utilidad. Aunque muchos han contribuido en este campo, se puede sostener que el siguiente paso más coherente, comprensible e influyente en la definición de qué son (o deberían ser) los casos de uso y cómo escribirlos, procede de Alistair Cockburn, resumido en su popular texto *Writing Effective Use Cases* [Cockburn01], basado en sus primeros trabajos y escritos publicados de 1992 en adelante. Esta introducción, por tanto, se basa y es consistente con este último trabajo.

¹ El término original en sueco se traduce literalmente como “caso de costumbre”.

6.3. Casos de uso y valor añadido

En primer lugar, algunas definiciones informales: un **actor** es algo con comportamiento, como una persona (identificada por un rol), sistema informatizado u organización; por ejemplo, un cajero.

Un **escenario** es una secuencia específica de acciones e interacciones entre los actores y el sistema objeto de estudio; también se denomina **instancia de caso de uso**. Es una historia particular del uso de un sistema, o un camino a través del caso de uso; por ejemplo, el escenario de éxito de compra de artículos con pago en efectivo, o el escenario de fallo al comprar debido al rechazo de la transacción de pago con la tarjeta de crédito.

Informalmente entonces, un **caso de uso** es una colección de escenarios con éxito y fallo relacionados, que describe a los actores utilizando un sistema para satisfacer un objetivo. Por ejemplo, a continuación presentamos un caso de uso en *formato informal* que incluye algunos escenarios alternativos:

Gestionar Devoluciones

Escenario principal de éxito: Un cliente llega a una caja con artículos para devolver. El cajero utiliza el sistema PDV para registrar cada uno de los artículos devueltos...

Escenarios alternativos:

Si se pagó con tarjeta de crédito, y se rechaza la transacción de reembolso a su cuenta, informar al cliente y pagarle en efectivo.

Si el identificador del artículo no se encuentra en el sistema, notificar al Cajero y sugerir la entrada manual del código de identificación (quizás esté alterado).

Si el sistema detecta fallos en la comunicación con el sistema de contabilidad externo...

El RUP proporciona una definición alternativa, aunque similar, de un caso de uso:

Un conjunto de instancias de caso de uso, donde cada instancia es una secuencia de acciones que un sistema ejecuta, produciendo un resultado observable de valor para un actor particular [RUP].

La expresión “*un resultado observable de valor*” es sutil pero importante, porque destaca el hecho de que el comportamiento del sistema debería preocuparse de proporcionar valor al usuario.

Una actitud clave en el trabajo con casos de uso es centrarse en la pregunta: “¿Cómo puedo, utilizando el sistema, proporcionar un valor observable al usuario, o cumplir sus objetivos?” en lugar de, simplemente, pensar en los requisitos del sistema en términos de una “lista de la lavandería” de características o funciones.

Quizás parece obvio destacar que se proporcione un valor observable para el usuario, pero la industria de software está plagada de proyectos fracasados que no proporciona-

ron lo que la gente realmente necesitaba. El enfoque de la lista de características y funciones para capturar los requisitos, puede contribuir a este resultado negativo, puesto que no fomenta que el personal involucrado considere los requisitos en un contexto amplio de uso del sistema, en un escenario para alcanzar algún resultado observable de valor, o algún objetivo. Por el contrario, los casos de uso sitúan las características y funciones en un contexto orientado al objetivo. De ahí el título del capítulo².

Ésta es la idea clave que Jacobson intentaba transmitir con el concepto de caso de uso: trabaja con los requisitos centrándote en cómo puede un sistema añadir valor y cumplir los objetivos.

6.4. Casos de uso y requisitos funcionales

Los casos de uso son requisitos; ante todo son requisitos funcionales que indican qué hará el sistema. En términos de los tipos de requisitos FURPS+, los casos de uso se refieren fundamentalmente a la “F” (funcional o de comportamiento), pero también pueden utilizarse para otros tipos, especialmente cuando esos otros tipos están estrechamente relacionados con un caso de uso. En el UP —y métodos más modernos— los casos de uso son el mecanismo principal que se recomienda para su descubrimiento y definición. Los casos de uso definen una promesa o contrato de la manera en la que se comportará un sistema.

Para ser claros: los casos de uso *son* requisitos (aunque no todos los requisitos). Algunos piensan en requisitos sólo como listas de características y funciones de la forma “el sistema deberá hacer...”. No es así, y una idea clave de los casos de uso es (por lo general) reducir la importancia o el uso de listas de características detalladas al estilo antiguo y más bien, escribir casos de uso para los requisitos funcionales. Veremos más sobre este punto en una sección posterior.

Los casos de uso son documentos de texto, no diagramas, y el modelado de casos de uso es, sobre todo, una acción de escribir texto, no dibujar. Sin embargo, UML define un diagrama de casos de uso para ilustrar los nombres de casos de uso y actores, y sus relaciones.

6.5. Tipos de casos de uso y formatos

Casos de uso de caja negra y las responsabilidades del sistema

Los casos de uso de **caja negra** son la clase más común y recomendada; no describen el funcionamiento interno del sistema, sus componentes o diseño, sino que se describe el sistema en base a las *responsabilidades* que tiene, que es una metáfora común y unificadora en el pensamiento orientado a objetos —los elementos software tienen responsabilidades y colaboran con otros elementos que tienen responsabilidades—.

² Procede del libro titulado muy apropiadamente *Uses Cases: Requirements in Context* [GK00] (el título del capítulo se adaptó con permiso de los autores).

A través de la definición de las responsabilidades del sistema con casos de uso de caja negra, es posible especificar *qué* debe hacer el sistema (los requisitos funcionales) sin decidir *cómo* lo hará (el diseño). De hecho, la definición de “análisis” frente al “diseño” se resume algunas veces como el “qué” frente al “cómo”. Éste es un tema importante en un buen desarrollo de software: evite durante el análisis de requisitos tomar decisiones acerca del “cómo”, y especifique el comportamiento externo del sistema, como una caja negra. Después, durante el diseño, cree una solución que satisfaga la especificación.

Estilo de caja negra	No
El sistema registra la venta	El sistema escribe la venta en una base de datos... o (incluso peor). El sistema genera una sentencia SQL INSERT para la venta...

Tipos de formalidad

Los casos de uso se escriben con formatos diferentes, dependiendo de la necesidad. Además del tipo de *visibilidad*, de caja negra frente a caja blanca, los casos de uso se escriben con varios grados de *formalidad*:

- **Breve:** resumen conciso de un párrafo, normalmente del escenario principal con éxito. El anterior ejemplo de *Procesar Venta* era breve.
- **Informal:** formato de párrafo en un estilo informal. Múltiples párrafos que comprenden varios escenarios. El anterior ejemplo de *Gestionar Devoluciones* era informal.
- **Completo:** el más elaborado. Se escriben con detalle todos los pasos y variaciones, y hay secciones de apoyo como precondiciones y garantías de éxito.

El ejemplo siguiente es un caso de uso en formato completo del caso de estudio NuevaEra.

6.6. Ejemplo completo: Procesar Venta

Los casos de uso completos muestran más detalles y están estructurados; son útiles para entender en profundidad los objetivos, tareas y requisitos. En el caso de estudio del PDV NuevaEra, se crearían en uno de los primeros talleres de requisitos con la colaboración del analista del sistema, expertos en la materia de estudio y los desarrolladores.

El formato usecases.org

Hay varias plantillas disponibles para los casos de uso completos. Sin embargo, quizás el formato más ampliamente extendido y compartido es la plantilla disponible en www.usecases.org. El siguiente ejemplo muestra este estilo.

El lector debe darse cuenta que éste es el principal ejemplo de caso de uso detallado para el caso de estudio del libro; muestra muchos elementos y cuestiones comunes.

Caso de uso UC1: Procesar Venta

Actor principal: Cajero.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- Compañía: Quiere registrar las transacciones con precisión y satisfacer los intereses de los clientes. Quiere asegurar que se registran los pagos aceptados por el Servicio de Autorización de Pagos. Quiere cierta tolerancia a fallos que permita capturar las ventas incluso si los componentes del servidor (ej. validación remota de crédito) no están disponibles. Quiere actualización automática y rápida de la contabilidad y el inventario.
- Agencia Tributaria del Gobierno: quiere recopilar los impuestos de cada venta. Podrían ser múltiples agencias: nacional, provincial y local.
- Servicio de Autorización de Pagos: Quiere recibir peticiones de autorización digital con el formato y protocolo correctos. Quiere registrar de manera precisa las cuentas por cobrar de la tienda.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la contabilidad y el inventario. Se registran las comisiones. Se genera el recibo. Se registran las autorizaciones de pago aprobadas.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.
2. El Cajero comienza una nueva venta.
3. El Cajero introduce el identificador del artículo.
4. El Sistema registra la línea de la venta y presenta la descripción del artículo, precio y suma parcial. El precio se calcula a partir de un conjunto de reglas de precios.

El Cajero repite los pasos 3-4 hasta que se indique.

5. El Sistema presenta el total con los impuestos calculados.
6. El Cajero le dice al Cliente el total y pide que le pague.
7. El Cliente paga y el Sistema gestiona el pago.
8. El Sistema registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario).
9. El Sistema presenta el recibo.
10. El Cliente se va con el recibo y las mercancías (si es el caso).

Extensiones (o Flujos Alternativos):

*a. En cualquier momento el Sistema falla:

Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.

1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
2. El Sistema reconstruye el estado anterior.
 - 2a. El Sistema detecta anomalías intentando la recuperación:
 1. El Sistema informa del error al Cajero, registra el error, y pasa a un estado limpio.
 2. El Cajero comienza una nueva venta.

3a. Identificador no válido:

1. El Sistema señala el error y rechaza la entrada.

3b. Hay muchos artículos de la misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):

1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.

3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:

1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
2. El Sistema muestra la suma parcial actualizada.

3-6b. El Cliente le pide al Cajero que cancele la venta:

1. El Cajero cancela la venta en el Sistema.

3-6c. El Cajero detiene la venta:

1. El sistema registra la venta para que esté disponible su recuperación en cualquier terminal PDV.

4a. El Sistema genera el precio de un artículo que no es el deseado (ej. el Cliente se queja por algo y se le ofrece un precio más bajo):

1. El Cajero introduce el precio alternativo.
2. El Sistema presenta el precio nuevo.

5a. El sistema encuentra algún fallo para comunicarse con el servicio externo del sistema de cálculo de impuestos.

1. El Sistema reinicia el servicio en el nodo PDV y continúa.

1a. El Sistema detecta que el servicio no se reinicia.

1. El Sistema señala el error.
2. El Cajero podría calcular e introducir manualmente el impuesto, o cancelar la venta.

5b. El Cliente dice que le son aplicables descuentos (ej. empleado, cliente preferente):

1. El Cajero señala la petición de descuento.
2. El Cajero introduce la identificación del Cliente.
3. El Sistema presenta el descuento total, basado en las reglas de descuento.

5c. El Cliente dice que tiene crédito en su cuenta, para aplicar a la venta:

1. El Cajero señala la petición de crédito.
2. El Cajero introduce la identificación del Cliente.
3. El Sistema aplica el crédito hasta que el precio = 0, y reduce el crédito que queda.

6a. El Cliente dice que su intención era pagar en efectivo pero que no tiene suficiente:

- 1a. El Cliente utiliza un método de pago alternativo.
- 1b. El Cliente le dice al Cajero que cancele la venta. El Cajero cancela la venta en el Sistema.

7a. Pago en efectivo:

1. El Cajero introduce la cantidad de dinero en efectivo entregada.
2. El Sistema muestra la cantidad de dinero a devolver y abre el cajón de caja.
3. El Cajero deposita el dinero entregado y devuelve el cambio al Cliente.
4. El Sistema registra el pago en efectivo.

7b. Pago a crédito:

1. El Cliente introduce la información de su cuenta de crédito.
2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
 - 2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
 1. El Sistema señala el error al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.
3. El Sistema recibe la aprobación del pago y lo notifica al Cajero.
 - 3a. El Sistema recibe la denegación del pago:
 1. El Sistema señala la denegación al Cajero.
 2. El Cajero le pide al Cliente un modo de pago alternativo.

- 4. El Sistema registra el pago a crédito, que incluye la aprobación del pago
- 5. El Sistema presenta el mecanismo de entrada para la firma del pago a crédito.
- 6. El Cajero le pide al Cliente que firme el pago a crédito. El Cliente introduce la firma.
- 7c. Pago con cheque...
- 7d. Pago a cuenta...
- 7e. El Cliente presenta cupones:
 - 1. Antes de gestionar el pago, el Cajero recoge cada cupón y el Sistema reduce el pago como sea oportuno. El sistema registra los cupones utilizados por razones de contabilidad.
 - 1a. El cupón introducido no es válido para ninguno de los artículos comprados
 - 1. El Sistema señala el error al Cajero.
- 9a. Hay rebajas en los artículos:
 - 1. El Sistema presenta los formularios de rebaja y los recibos de descuento para cada artículo con una rebaja.
- 9b. El Cliente solicita un vale-regalo (sin precio visible):
 - 1. El Cajero solicita el vale-regalo y el Sistema lo proporciona.

Requisitos especiales:

- Interfaz de Usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
- Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
- De algún modo, queremos recuperación robusta cuando falla el acceso a servicios remotos, como el sistema de inventario.
- Internacionalización del lenguaje del texto que se muestra.
- Reglas de negocio que se puedan añadir en tiempo de ejecución en los pasos 3 y 7.
- ...

Lista de tecnología y variaciones de datos:

- 3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.
- 3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.
- 7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.
- 7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Frecuencia: Podría ser casi continuo.

Temas abiertos:

- ¿Cuáles son las variaciones de la ley de impuestos?
- Explorar las cuestiones de recuperación de servicios remotos.
- ¿Cuál es la adaptación que se tiene que hacer para diferentes negocios?
- ¿Un cajero debe llevarse el dinero de la caja cuando salga del sistema?
- ¿Puede utilizar el cliente directamente el lector de tarjetas o tiene que hacerlo el cajero?

Este caso de uso es más bien ilustrativo que exhaustivo (aunque está basado en los requisitos de un sistema PDV real). Sin embargo, muestra suficiente detalle y complejidad para exponer de manera realista que los casos de uso completos pueden documentar muchos detalles de los requisitos. Este ejemplo nos servirá bien como modelo para muchos problemas de los casos de uso.

La variación de dos-columnas

Algunos prefieren el formato en dos columnas o conversacional, que destaca el hecho de que se establece una interacción entre los actores y el sistema. Se propuso por primera vez por Rebecca Wirfs-Brock en [Wirfs-Brock93], y también promueven su uso Constantine y Lockwood para ayudar en el análisis e ingeniería de usabilidad [CL99]. A continuación presentamos el mismo contenido utilizando el formato en dos columnas:

Caso de uso UC1: Procesar Venta

Actor principal:como antes...	
Escenario principal de éxito:	
Acción del actor (o intención)	Responsabilidad del Sistema
1. El Cliente llega a un terminal PDV con mercancías y/o servicios que comprar.	
2. El Cajero comienza una nueva venta.	
3. El Cajero introduce el identificador del artículo.	4. Registra cada línea de la venta y presenta la descripción del artículo y la suma parcial.
<i>El Cajero repite los pasos 3-4 hasta que se indique</i>	
6. El Cajero le dice al Cliente el total y pide que le pague.	5. El Sistema presenta el total con los impuestos calculados.
7. El Cliente paga.	8. El Sistema gestiona el pago.
	9. Registra la venta completa y envía la información de la venta y el pago al sistema de Contabilidad externo (para la contabilidad y las comisiones) y al sistema de Inventario (para actualizar el inventario). El Sistema presenta el recibo.
...	...

¿Cuál es el mejor formato?

No existe un mejor formato; unos prefieren el estilo de una columna, otros el de dos columnas. Las secciones se pueden añadir y quitar; los nombres de los títulos podrían cambiar. Ninguna de estas cosas es especialmente importante; la clave es escribir los detalles del escenario principal de éxito y sus extensiones de alguna forma. [Cockburn1] resume muchos formatos utilizables:

Práctica personal

Ésta es mi práctica, no una recomendación. Durante algunos años, utilicé el formato en dos columnas debido a su clara separación visual de la conversación. Sin embargo, he vuelto al estilo en una columna ya que es más compacto y más fácil de formatear, y el pequeño valor de la separación visual de la conversación, para mí, no supera estos beneficios. Encuentro que es todavía sencillo identificar visualmente las diferentes partes en la conversación (Cliente, Sistema,...) si cada parte y las respuestas del Sistema se asignan normalmente a sus propios pasos.

6.7. Explicación de las secciones

Elementos del prólogo

Son posibles muchos elementos opcionales en el prólogo. Sitúe al principio sólo los elementos que son importantes que se lean antes del escenario principal de éxito. Mueve el material de “encabezamiento” ajeno, al final de los casos de uso.

Actor principal: El actor principal que recurre a los servicios del sistema para cumplir un objetivo.

Importante: Personal involucrado y lista de intereses

Esta lista es más importante y práctica de lo que podría parecer a primera vista. Sugiere y delimita qué es lo que debe hacer el sistema. Citando a Cockburn:

El [sistema] funciona siguiendo un contrato entre el personal involucrado, donde los casos de usos detallan la parte de comportamiento del contrato... El caso de uso, como contrato de comportamiento, captura *todo* y *sólo* el comportamiento relacionado con la satisfacción de los intereses del personal involucrado [Cockburn01].

Esto contesta la pregunta: ¿Qué debería estar en un caso de uso? La respuesta es: lo que satisfaga los intereses de todo el personal involucrado. Además, empezando con el personal involucrado y sus intereses antes de escribir el resto del caso de uso, tenemos un modo de recordarnos cuáles deberían ser las responsabilidades más detalladas del sistema. Por ejemplo, ¿habría identificado una responsabilidad para gestionar la comisión del vendedor si no hubiese listado en primer lugar la persona involucrada vendedor y sus intereses? En el mejor de los casos al final, pero quizás lo habría echado en falta durante la primera sesión de análisis. El punto de vista del interés del personal involucrado proporciona un procedimiento metódico y completo para el descubrimiento y registro de todos los comportamientos requeridos.

Personal involucrado e intereses:

- Cajero: quiere entradas precisas, rápidas, y sin errores de pago, ya que las pérdidas se deducen de su salario.
- Vendedor: quiere que las comisiones de las ventas estén actualizadas.
- ...

Precondiciones y garantías de éxito (postcondiciones)

Las **precondiciones** establecen lo que *siempre debe* cumplirse antes de comenzar un escenario en el caso de uso. Las precondiciones *no* se prueban en el caso de uso, sino que son condiciones que se asumen que son verdad. Normalmente, una precondición implica un escenario de otro caso de uso que se ha completado con éxito, como inicio de sesión o el más general “el cajero se identifica y autentica”. Nótese que hay condiciones

que deben ser verdad, pero no tienen un valor práctico para que se escriban, como “el sistema tiene energía”. Las precondiciones comunican suposiciones importantes de las que el escritor del caso de uso piensa que los lectores deberían ser avisados.

Las **garantías de éxito** (o **postcondiciones**) establecen qué debe cumplirse cuando el caso de uso se completa con éxito —o bien el escenario principal de éxito o algún camino alternativo—. La garantía debería satisfacer las necesidades de todo el personal involucrado.

Precondiciones: El cajero se identifica y autentica.

Garantías de éxito (Postcondiciones): Se registra la venta. El impuesto se calcula de manera correcta. Se actualizan la Contabilidad y el Inventario. Se registran las comisiones. Se genera el recibo.

Escenario principal de éxito y pasos (o Flujo Básico)

También recibe el nombre de escenario del “camino feliz”, o más prosaico “Flujo Básico”. Describe el camino de éxito típico que satisface los intereses del personal involucrado. Nótese que, a menudo, *no* incluye ninguna condición o bifurcación. Aunque no es incorrecto o ilegal, se puede suponer que es más comprensible y extensible ser muy consistente, y postergar todo el manejo de caminos condicionales a la sección Extensiones.

Sugerencia

Posponga todas las sentencias condicionales y de bifurcación a la sección Extensiones.

El escenario recoge los pasos, que pueden ser de tres tipos:

1. Una interacción entre actores³.
2. Una validación (normalmente a cargo del sistema).
3. Un cambio de estado realizado por el sistema (por ejemplo, registrando o modificando algo).

El primer paso de un caso de uso, normalmente no está incluido en esta clasificación, sino que indica el evento que desencadena el comienzo del escenario.

Un estilo habitual es poner con mayúsculas los nombres de los actores para facilitar la identificación. También podemos observar el estilo que se utiliza para indicar una repetición.

Escenario principal de éxito (o Flujo Básico):

1. El Cliente llega a un terminal PDV con mercancías para comprar.
 2. El Cajero comienza una nueva venta.
 3. El Cajero introduce el identificador del artículo.
 4. ...
- El Cajero repite los pasos 3-4 hasta que se indique.
5. ...

³ Nótese que el sistema que se está estudiando en sí mismo debería considerarse un actor cuando juega un rol de actor colaborando con otros sistemas.

Extensiones (o Flujos Alternativos)

Las extensiones son muy importantes. Indican todos los otros escenarios o bifurcaciones, tanto de éxito como de fracaso. Podemos observar que en el ejemplo de caso de uso completo, la sección Extensiones es considerablemente más larga y compleja que la correspondiente al Escenario Principal de Éxito; esto es normal y de esperar. También se conocen como “Flujos Alternativos”.

En la escritura de casos de uso completos, la combinación del camino feliz y los escenarios de extensión deberían satisfacer “casi” todos los intereses del personal involucrado. Este punto está limitado, puesto que algunos intereses se podrían capturar mejor como requisitos no funcionales escritos en la Especificación Complementaria en lugar de en los casos de uso.

Los escenarios de extensión son bifurcaciones del escenario principal de éxito y, por tanto, pueden ser etiquetados de acuerdo con él. Por ejemplo, en el Paso 3 del escenario principal podría haber un identificador de artículo inválido, bien porque no se introdujo correctamente o bien porque el sistema no lo conoce. Una extensión se etiqueta como “3a”; primero identifica la condición y después la respuesta. Una extensión alternativa al Paso 3 se etiqueta como “3b” y así sucesivamente.

Extensiones (o Flujos Alternativos):
3a. Identificador no válido:
1. El Sistema señala el error y rechaza la entrada.
3b. Hay muchos artículos de misma categoría y tener en cuenta una única identidad del artículo no es importante (ej. 5 paquetes de hamburguesas vegetales):
1. El Cajero puede introducir el identificador de la categoría del artículo y la cantidad.

Una extensión tiene dos partes: la condición y el manejo.

Guía: Escriba la condición como algo que pueda ser detectado por el sistema o un actor. Para contrastar:

5a. El Sistema detecta un fallo en la comunicación con el servicio externo del sistema de cálculo de impuestos:
5a. El sistema de cálculo de impuestos externo no funciona:

Se prefiere el primer estilo porque se trata de algo que el sistema puede detectar; el último es una inferencia.

El manejo de la extensión se puede resumir en un paso, o incluir una secuencia, como en este ejemplo, que también ilustra la notación utilizada para indicar que una condición puede tener lugar en una serie de pasos:

3-6a. El Cliente le pide al Cajero que elimine un artículo de la compra:
1. El Cajero introduce el identificador del artículo para eliminarlo de la compra.
2. El Sistema muestra la suma parcial actualizada.

Al final del manejo de la extensión, por defecto, el escenario se une de nuevo con el escenario principal de éxito, a menos que la extensión indique otra cosa (como interrumpir el sistema).

Algunas veces, un punto de extensión particular es bastante complejo, como en la extensión “pago a crédito”. Esto puede ser un motivo para expresar la extensión como un caso de uso aparte.

Este ejemplo de extensión también muestra la notación para expresar fallos en las extensiones.

7b. Pago a crédito:
1. El Cliente introduce la información de su cuenta de crédito.
2. El Sistema envía la petición de autorización del pago al Sistema externo de Servicio de Autorización de Pagos, y solicita la aprobación del pago.
2a. El Sistema detecta un fallo en la colaboración con el sistema externo:
1. El Sistema señala el error al Cajero.
2. El Cajero le pide al Cliente un modo de pago alternativo.
3 ...

Si es deseable describir una condición de extensión que puede ser posible durante cualquiera (o al menos la mayoría) de los pasos, se pueden utilizar las etiquetas *a, *b,...

*a. En cualquier momento el Sistema falla:
Para dar soporte a la recuperación y registro correcto, asegura que todos los estados y eventos significativos de una transacción puedan recuperarse desde cualquier paso del escenario.
1. El Cajero reinicia el Sistema, inicia la sesión, y solicita la recuperación al estado anterior.
2. El Sistema reconstruye el estado anterior.

Requisitos especiales

Si un requisito no funcional, atributo de calidad o restricción se relaciona de manera específica con un caso de uso, se recoge en el caso de uso. Esto incluye cualidades tales como rendimiento, fiabilidad y facilidad de uso, y restricciones de diseño (a menudo, en dispositivos de entrada/salida) que son obligados o se consideran probables.

Requisitos especiales:
• Interfaz de usuario con pantalla táctil en un gran monitor de pantalla plana. El texto debe ser visible a un metro de distancia.
• Tiempo de respuesta para la autorización de crédito de 30 segundos el 90% de las veces.
• Internacionalización del lenguaje del texto que se muestra.
• Reglas de negocio que se puedan añadir en ejecución en los pasos 3 y 7.

Un consejo clásico del UP es registrar estos requisitos con el caso de uso, y constituye un lugar razonable al escribir primero el caso de uso. Sin embargo, muchos expertos encuentran útil reunir al final todos los requisitos no funcionales en la Especificación Complementaria, para favorecer la gestión del contenido, comprensión y legibilidad por-

que, normalmente, se tienen que considerar estos requisitos en conjunto durante el análisis arquitectural.

Lista de tecnología y variaciones de datos

A menudo, encontramos variaciones técnicas en cómo se debe hacer algo, pero no en qué, y es importante registrarlo en el caso de uso. Un ejemplo típico es una restricción técnica impuesta por el personal involucrado con respecto a las tecnologías de entrada o salida de datos. Por ejemplo, uno de los interesados podría decir, “El sistema PDV debe soportar la entrada de la cuenta de crédito utilizando un lector de tarjetas y el teclado”. Nótese que éstos son ejemplos de decisiones o restricciones de diseño anticipadas; en general, deben evitarse decisiones de diseño prematuras, pero algunas veces son obvias o inevitables, especialmente en relación con las tecnologías de entrada/salida.

También es necesario entender las variaciones en los esquemas de los datos, como utilizar UPCs o EANs para los identificadores de los artículos, codificados mediante el código de barras.

Esta lista es el lugar para situar tales variaciones. También es útil registrar las variaciones en los datos que podrían capturarse en un paso particular.

Lista de tecnología y variaciones de datos

3a. El identificador del artículo se introduce mediante un escáner láser de código de barras (si está presente el código de barras) o a través del teclado.

3b. El identificador del artículo podría ser cualquier esquema de código UPC, EAN, JAN o SKU.

7a. La entrada de la información de la cuenta de crédito se lleva a cabo mediante un lector de tarjetas o el teclado.

7b. La firma de los pagos a crédito se captura en un recibo de papel. Pero en dos años, pronosticamos que muchos clientes querrán que se capture la firma digital.

Sugerencia

Esta sección no debería contener múltiples pasos para representar la variación de comportamiento en diferentes casos. Si es necesario, dígalos en la sección Extensiones.

6.8. Objetivos y alcance de un caso de uso

¿Cómo deberían descubrirse los casos de uso? Es típico no estar seguro si algo es un caso de uso válido (o de manera más práctica, útil). Las tareas se pueden agrupar a muchos niveles de granularidad, desde uno o unos pocos pasos pequeños, hasta actividades de nivel de empresa.

¿A qué nivel y alcance deberían expresarse los casos de uso?

Las siguientes secciones presentan las ideas sencillas de los procesos y objetivos del negocio elementales, como marco para la identificación de los casos de uso de una aplicación.

Casos de uso para los procesos del negocio elementales

¿Cuál de éstos es un caso de uso válido?

- Negociar un Contrato con el Proveedor
- Gestionar las Devoluciones
- Iniciar Sesión

Podría argumentarse que todos ellos son casos de uso a diferentes niveles, dependiendo de los límites del sistema, actores y objetivos. La evaluación de estos candidatos se presenta después de una introducción a los procesos del negocio elementales.

En lugar de preguntar en general: “¿qué es un caso de uso válido?”, una pregunta más relevante para el caso de estudio PDV es: ¿cuál es el nivel útil para expresar los casos de uso en el análisis de requisitos de la aplicación?

Guía: El caso de uso EBP

Para el análisis de requisitos de una aplicación informática, céntrese en los casos de uso al nivel de procesos del negocio elementales (EBPs, Elementary Business Processes).

EBP es un término que procede del campo de la ingeniería de procesos del negocio⁴, y se define como:

Una tarea realizada por una persona en un lugar, en un instante, como respuesta a un evento del negocio, que añade un valor cuantificable para el negocio y deja los datos en un estado consistente, ej. Autorizar Crédito, o Solicitar Precio (se perdió la fuente original).

Esto se puede tomar demasiado literalmente: ¿Está mal considerar un caso de uso como un EBP si se requieren dos personas, o si una persona tiene que pasear? Probablemente no; sin embargo, la impresión general de la definición es casi correcta. No se trata de un pequeño paso como “eliminar una línea de pedido” o “imprimir el documento”. Sino que el escenario principal de éxito está formado probablemente por cinco o diez pasos. No tarda días y múltiples sesiones, como “negociar un contrato con el proveedor”; es una tarea que se aborda en una única sesión. Probablemente dura entre unos pocos minutos y una hora. Como con la definición del UP, hace hincapié en añadir valor observable y cuantificable al negocio, y llega a un acuerdo en el que el sistema y los datos se encuentran en un estado estable y consistente.

⁴ EBP es parecido al término **tarea de usuario** en la ingeniería de usabilidad, aunque el significado es menos estricto en ese dominio.

Un error típico de los casos de uso es definir muchos casos de uso a un nivel muy bajo; es decir, como un paso simple, subfunción o subtarea en un EBP.

Violaciones razonables de la guía EBP

Aunque los casos de uso “base” de una aplicación deberían satisfacer la guía EBP, normalmente es útil crear “sub” casos de uso separados que representan subtarefas, o pasos, en un caso de uso base. Pueden existir casos de uso que no sean EBP; potencialmente existen muchos a un nivel inferior. La guía sólo se utiliza para encontrar el nivel dominante de casos de uso en el análisis de requisitos de una aplicación; esto es, el nivel en el que nos tenemos que centrar para nombrarlos y escribirlos.

Por ejemplo, una subtarea o extensión como “pago a crédito” podría repetirse en varios casos de uso base. Es conveniente separarlo en un caso de uso propio (que no satisface la guía EBP) y conectarlo a varios casos de uso base, para evitar duplicaciones del texto.

En el Capítulo 25 estudiaremos las cuestiones acerca de las relaciones de casos de uso.

Casos de uso y objetivos

Los actores tienen objetivos (o necesidades) y utilizan las aplicaciones para ayudarles a satisfacerlos. En consecuencia, un caso de uso de nivel EBP se denomina caso de uso de nivel de **objetivo de usuario**, para remarcar que sirve (o debería servir) para satisfacer un objetivo de un usuario del sistema, o el actor principal.

Esto nos lleva a recomendar el siguiente procedimiento:

1. Encontrar los objetivos de usuario.
2. Definir un caso de uso para cada uno.

Esto supone un ligero cambio de énfasis para el modelador de casos de uso. En lugar de preguntar: “¿Cuáles son los casos de uso?”, uno comienza preguntando: “¿Cuáles son tus objetivos?”. De hecho, el nombre del caso de uso para un objetivo de usuario debería reflejar dicho objetivo, para resaltar este punto de vista. Objetivo: capturar o procesar una venta; caso de uso: *Procesar Venta*.

Nótese que debido a esta simetría, la guía EBP se puede aplicar igualmente para decidir si un objetivo o un caso de uso se encuentran a un nivel adecuado.

Por tanto, he aquí una idea clave con respecto a la investigación de los objetivos de usuario frente a la investigación de casos de uso:

Imagine que estamos juntos en un taller de requisitos. Nos podríamos preguntar:

- “¿Qué haces?” (una pregunta bastante orientada al caso de uso) o;
- “¿Cuáles son tus objetivos?”

Es más probable que las respuestas a la primera pregunta reflejen soluciones y procedimientos actuales, y las complicaciones asociadas a ellos.

Las respuestas a la segunda pregunta, especialmente combinada con una investigación para ascender en la jerarquía de objetivos (“¿cuál es el objetivo de ese objetivo?”), abren la visión de soluciones nuevas y mejoradas, centradas en añadir valor al negocio, y llegar al corazón de lo que el personal involucrado quiere del sistema que se está estudiando.

Ejemplo: aplicación de la guía EBP

Un analista de sistemas responsable del descubrimiento de los requisitos del sistema NuevaEra, está investigando los objetivos de usuario. La conversación transcurre de esta forma durante un taller de requisitos:

Analista de sistemas: “¿Cuáles son algunos de sus objetivos en el contexto de uso de un sistema PDV?”

Cajero: “Uno, iniciar la sesión rápidamente. También, capturar las ventas.”

Analista de sistemas: “¿Cuál cree que es el objetivo de nivel más alto que motiva el inicio de sesión?”

Cajero: “Intento identificarme en el sistema, de este modo puede validar que estoy autorizado para utilizar el sistema que captura ventas y otras tareas.”

Analista de sistemas: “¿Más alto que ése?”

Cajero: “Evitar robos, alteración de datos, y mostrar información privada de la compañía.”

Obsérvese que la estrategia del analista de buscar de manera ascendente en la jerarquía de objetivos para encontrar los objetivos de usuario de un nivel superior que todavía satisfagan la guía EBP, para obtener el objetivo real detrás de la acción, y también para entender el contexto de los objetivos.

“Evitar robos...” es de un nivel superior a un objetivo de usuario; podría llamarse objetivo de empresa, y no es un EBP. Por tanto, aunque puede inspirar nuevas formas de pensar en el problema y las soluciones (como eliminar sistemas PDV y cajeros completamente), lo vamos a dejar a un lado por ahora.

Disminuyendo el nivel del objetivo a “identificarme en el sistema y ser validado” se acerca al nivel de objetivo de usuario. ¿Pero es el nivel EBP? No añade valor observable o cuantificable al negocio. Si el responsable del comercio pregunta: “¿Qué hiciste hoy?” y dices “¡Inicié la sesión 20 veces!”, no se impresionaría. En consecuencia, es un objetivo secundario, siempre disponible para hacer algo útil, y no es un EBP u objetivo de usuario. En cambio, “capturar una venta” cumple el criterio para ser un EBP u objetivo de usuario.

Otro ejemplo podría ser, en algunas tiendas existe un proceso denominado “abrir caja”, en el cual un cajero inserta su propia bandeja del cajón de caja en el terminal, inicia la sesión, e indica al sistema el dinero que hay en caja. *Abrir caja* es un caso de uso de nivel EBP (o nivel de objetivo de usuario); el paso de inicio de sesión, en lugar de ser un caso de uso de nivel EBP, es un objetivo de subfunción para llevar a cabo el objetivo de abrir caja.

Objetivos y casos de uso de subfunción

Aunque “identificarme y ser validado” (o “iniciar sesión”) se ha eliminado como objetivo de usuario, es un objetivo de nivel más bajo, denominado **objetivo de subfunción** —subobjetivos que dan soporte a un objetivo de usuario—. Sólo deberían escribirse casos de uso de manera ocasional para estos objetivos de subfunción, aunque es un problema típico que observan los expertos cuando se les pide que evalúen y mejoren (normalmente que simplifiquen) un conjunto de casos de uso.

No es ilegal escribir casos de uso para objetivos de subfunción, pero no siempre es útil, ya que añade complejidad al modelo de casos de uso; puede haber cientos de objetivos de subfunción —o casos de uso de subfunción— en un sistema.

Un punto importante es que el número y granularidad de los casos de uso influyen en el tiempo y la dificultad para entender, mantener y gestionar los requisitos.

El motivo válido, más común para representar un objetivo de subfunción como un caso de uso, es cuando la subfunción se repite o es una precondition en muchos casos de uso de nivel de objetivos de usuario. Este hecho se cumple probablemente en el caso de “identificarme y ser validado”, que es una precondition de la mayoría, si no todos, los otros casos de uso de nivel de objetivos de usuario.

En consecuencia, podría escribirse como el caso de uso *Autenticar Usuario*.

Objetivos y casos de uso pueden ser compuestos

Normalmente, los objetivos son compuestos, desde el nivel de empresa (“ser rentable”), que incluyen muchos objetivos intermedios a nivel de uso de la aplicación (“se capturan las ventas”), que a su vez incluyen objetivos de subfunción dentro de las aplicaciones (“la entrada es válida”).

De manera análoga, los casos de uso se pueden escribir a niveles diferentes para satisfacer estos objetivos, y pueden estar compuestos de casos de uso de nivel inferior.

Estos diferentes niveles de objetivos y casos de uso son una fuente típica de confusión en la identificación del nivel adecuado de los casos de uso de una aplicación. La guía EBP proporciona una orientación para eliminar casos de uso de nivel excesivamente bajo.

6.9. Descubrimiento de actores principales, objetivos y casos de uso

Los casos de uso se definen para satisfacer los objetivos de usuario de actores principales. Por tanto, el procedimiento básico es:

- 1. Elegir los límites del sistema. ¿Es sólo una aplicación software, el hardware y la aplicación como un todo, que lo utiliza más de una persona o una organización completa?

- 2. Identificar los actores principales —aquellos que tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema—.
- 3. Para cada uno, identificar sus objetivos de usuario. Elevarlos al nivel de objetivos de usuario más alto que satisfaga la guía EBP.
- 4. Definir los casos de uso que satisfagan los objetivos de usuario; nombrarlos de acuerdo con sus objetivos. Normalmente, los casos de uso del nivel de objetivo de usuario se corresponderán uno a uno con los objetivos de usuario, aunque hay al menos una excepción, como se verá.

Paso 1: Elegir el límite del sistema

Para este caso de estudio, el propio sistema PDV es el sistema que se está diseñando; todo lo que queda fuera de él, está fuera de los límites del sistema, incluyendo el cajero, el servicio de autorización de pagos, etcétera.

Si no está clara la definición de los límites del sistema que se está diseñando, se puede aclarar definiendo lo que está fuera —los actores principales externos y de apoyo—. Una vez identificados los actores externos, los límites se vuelven más claros. Por ejemplo, ¿se encuentra la responsabilidad de autorización de pagos completa en los límites del sistema? No, hay un actor del servicio externo de autorización de pagos.

Pasos 2 y 3: Identificar los actores principales y objetivos

Es artificial establecer de manera estricta que la identificación de los actores principales es antes que los objetivos de usuario; en un taller de requisitos, la gente pone en común sus ideas y dan lugar a una mezcla de ambos. Algunas veces los objetivos ponen de manifiesto a los actores, o viceversa.

Guía: Centrar la discusión en los actores principales en primer lugar, ya que establece el marco para las investigaciones posteriores.

Preguntas útiles para encontrar los actores principales y objetivos

Además de los actores principales y objetivos de usuario obvios, las preguntas siguientes ayudan a identificar otros que se podrían haber pasado:

- | | |
|---|--|
| ¿Quién arranca y para el sistema? | ¿Quién se encarga de la administración del sistema? |
| ¿Quién gestiona a los usuarios y la seguridad? | ¿Es un actor el “tiempo” porque el sistema hace algo como respuesta a un evento de tiempo? |
| ¿Existe un proceso de control que reinicie el sistema si falla? | ¿Quién evalúa la actividad o el rendimiento del sistema? |

- ¿Cómo se gestionan
las actualizaciones de software?
¿Actualizaciones automáticas o no?
- ¿Quién evalúa los registros?
¿Se recuperan de manera remota?

Actores principales y de apoyo

Recordemos que los actores principales tienen objetivos de usuario que se satisfacen mediante el uso de los servicios del sistema. Acuden al sistema para que les ayude. Al contrario que los *actores de apoyo*, que proporcionan servicios al sistema que se está diseñando. Por ahora, nos centraremos en encontrar los actores principales, no los de apoyo.

Recordemos también que los actores principales pueden ser —entre otras cosas— otros sistemas informáticos, como procesos software “guardianes” (“*watchdog*”).

Sugerencia

Desconfía si ninguno de los actores principales se corresponde con un sistema informático externo.

La lista actor-objetivo

Recoge los actores principales y sus objetivos de usuario en una lista actor-objetivo. En términos de los artefactos del UP, debería corresponderse con una sección del artefacto Visión (que se describirá en el capítulo siguiente).

Por ejemplo:

Actor	Objetivo	Actor	Objetivo
Cajero	procesar ventas procesar alquileres gestionar las devoluciones abrir caja cerrar caja ...	Administrador del Sistema	añadir usuarios modificar usuarios eliminar usuarios gestionar seguridad gestionar las tablas del sistema ...
Director	poner en marcha suspender operación ...	Sistema de Actividad de Ventas	analizar los datos de ventas y rendimiento
...	...	...	...

El Sistema de Actividad de Ventas es una aplicación remota a la que se le solicitará con frecuencia datos de ventas desde cada nodo PDV en la red.

Dimensión de la planificación del proyecto

En la práctica, esta lista tiene columnas adicionales para la prioridad, esfuerzo y riesgo; esto se tratará brevemente en el Capítulo 36.

La complicada realidad

Esta lista parece ordenada, pero la realidad de su creación es cualquier cosa salvo eso. Son necesarias muchas “tormentas de ideas” y discusiones durante los talleres de requisitos. Consideremos el ejemplo anterior que ilustraba la aplicación de la regla EBP al objetivo de “iniciar sesión”. Durante el taller, mientras se crea esta lista, el cajero podría proponer “iniciar sesión” como un objetivo de usuario. El analista de sistemas profundizará y subirá el nivel del objetivo, más allá del mecanismo de bajo nivel de inicio de sesión (el cajero posiblemente estaba pensando en utilizar un cuadro de diálogo en una GUI), al nivel de “identificar y autenticar al usuario”. Pero, el analista entonces se da cuenta de que no cumple la guía EBP, y lo descarta como objetivo de usuario. Por supuesto, la realidad es incluso algo diferente a esto puesto que un analista con experiencia cuenta con un conjunto de heurísticas, procedentes de experiencias o estudios anteriores, una de las cuales es “la autenticación de los usuarios es rara vez un EBP”, y de este modo es probable que se elimine rápidamente.

El actor principal y los objetivos de usuario dependen del límite del sistema

¿Por qué es el cajero, y no el cliente, el actor principal del caso de uso de *Procesar Venta*? ¿Por qué no aparece el cliente en la lista actor-objetivo?

La respuesta depende del límite del sistema que se esté diseñando, como se ilustra en la Figura 6.1. Si consideramos la empresa o el servicio de caja como un sistema agregado, el cliente *es* un actor principal, con el objetivo de obtener artículos o servicios, y marcharse. Sin embargo, desde el punto de vista de únicamente el sistema PDV (que es la elección del límite del sistema para este caso de estudio), éste atiende el objetivo del cajero (y de la tienda) de procesar la venta del cliente.

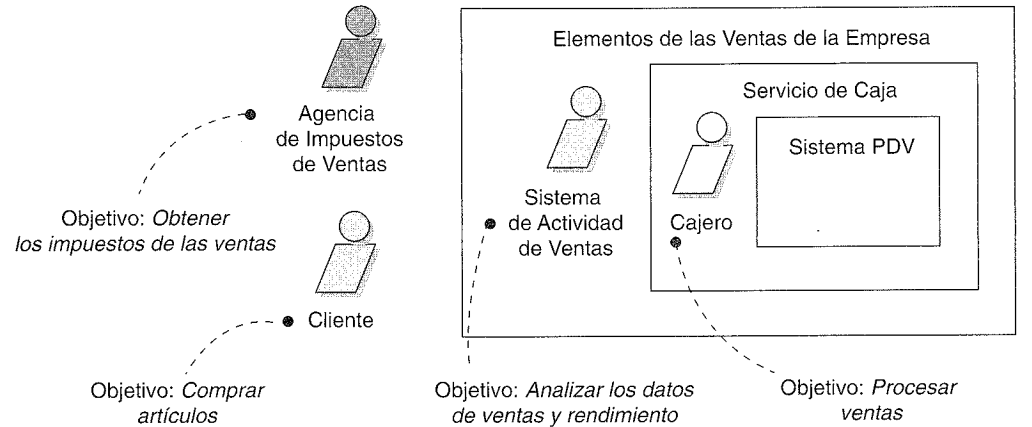


Figura 6.1. Actores principales y objetivos con diferentes límites del sistema.

Actores y objetivos por medio del análisis de eventos

Otro enfoque para ayudar en la búsqueda de los actores, objetivos y casos de uso, es identificar los eventos externos. ¿Cuáles son, de dónde proceden y por qué? A menudo,

un grupo de eventos pertenecen al mismo objetivo o caso de uso de nivel EBP. Por ejemplo:

Evento Externo	Parte del Actor	Objetivo
introducir línea de venta	Cajero	procesar una venta
introducir el pago	Cliente o Cajero	procesar una venta
...	...	

Paso 4: Definir los casos de uso

Por lo general, definimos un caso de uso de nivel EBP por cada objetivo de usuario. Nombramos el caso de uso de manera similar al objetivo de usuario; por ejemplo, Objetivo: procesar una venta; Caso de Uso: *Procesar Venta*.

También, nombre los casos de uso comenzando con un verbo.

Una excepción típica a un caso de uso por objetivo, es agrupar objetivos separados CRUD⁵ (crear, recuperar, actualizar, eliminar) en un caso de uso CRUD, llamado, por convención, *Gestionar<X>*. Por ejemplo, los objetivos “editar usuario”, “eliminar usuario”, etcétera, todos se satisfacen en el caso de uso *Gestionar Usuarios*.

“Definir casos de uso” comprende varios niveles de esfuerzo, variando desde unos pocos minutos, para recoger simplemente los nombres, hasta semanas, con el fin de escribir las versiones en formato completo. La última sección del proceso UP de este capítulo, sitúa este trabajo —cuándo y cuánto— en el contexto del desarrollo iterativo y del UP.

6.10. Enhorabuena: se han escrito los casos de uso y no son perfectos

La necesidad de comunicación y participación

El equipo del PDV NuevaEra va escribiendo casos de uso en múltiples talleres de requisitos, a través de una serie de iteraciones de desarrollo cortas, añadiendo incrementalmente al conjunto, y refinando y adaptando en base a la retroalimentación. Los expertos en la materia de estudio, los cajeros y programadores participan activamente en el proceso de escritura. No existen intermediarios entre los cajeros, otros usuarios y los desarrolladores, sino que se establece una comunicación directa.

Está bien, pero no es suficiente. Las especificaciones de requisitos escritas dan la impresión de ser correctas; pero no lo son. Los casos de uso y otros requisitos todavía

⁵ N. del T.: Acrónimo de Create-Retrieve-Update-Delte.

no serán correctos —garantizado—. Carecen de información crítica y contienen afirmaciones erróneas. La solución no es la aptitud del proceso “en cascada” de esforzarse mucho por recopilar los requisitos perfectos y completos al principio, aunque, por supuesto, lo hacemos lo mejor que podemos en el tiempo disponible. Pero nunca será suficiente.

Se necesita un enfoque diferente. Una buena parte del enfoque es el desarrollo iterativo, pero se necesita algo más: *comunicación personal continua*. Comunicación y participación cercana y continua —diaria— entre los desarrolladores y alguien que entienda el dominio y pueda tomar decisiones sobre los requisitos. Alguno de los programadores se puede acercar y en cuestión de segundos obtener aclaraciones, en cualquier momento que surja una duda. Por ejemplo, las prácticas de XP [Beck00] incluyen una excelente recomendación: *los usuarios deben tener dedicación a tiempo completo al proyecto, permaneciendo en la sala del proyecto*.

6.11. Escritura de casos de uso en un estilo esencial independiente de la interfaz de usuario

¡Nuevo y mejorado! Razones a favor de utilizar las huellas dactilares

Investigar y preguntar acerca de los objetivos, en lugar de las tareas y procedimientos fomenta que se centre la atención en la esencia de los requisitos —la intención detrás de ellos—. Por ejemplo, durante un taller de requisitos, el cajero podría decir que uno de sus objetivos es “iniciar la sesión”. El cajero, probablemente, estaría pensando en una GUI, cuadro de diálogo, identificador de usuario (ID) y contraseña (*password*). Éste es un mecanismo para alcanzar un objetivo, en lugar del objetivo en sí mismo. Mediante la investigación ascendente de la jerarquía de objetivos (“¿Cuál es el objetivo del objetivo?”), el analista del sistema llega hasta el objetivo independiente del mecanismo: “identificarse y conseguir la autenticación”, o incluso un objetivo de nivel superior: “prevenir robos...”.

Este proceso de descubrimiento puede abrir la visión a soluciones nuevas y mejoradas. Por ejemplo, los teclados y ratones con lectores biométricos, normalmente para las huellas dactilares son ahora habituales y baratos. Si el objetivo es “identificación y autenticación”, ¿por qué no hacerlo rápido y fácil, utilizando un lector biométrico en el teclado? Pero, la respuesta adecuada conlleva también un análisis de usabilidad, como conocer el perfil de los usuarios típicos del sistema. ¿Qué pasa si tienen los dedos cubiertos de grasa? ¿Tienen dedos?

Escritura en estilo esencial

Esta idea se ha resumido en varias guías de casos de uso como “no considere la interfaz de usuario; céntrese en la intención”. [Cockburn01]. La motivación y notación la ha estudiado Larry Constantine de manera más completa, en el contexto de la creación de interfaces de usuario (UIs) mejores y de la ingeniería de usabilidad [Constantine94,

CL99]. Constantine denomina al estilo de escritura **esencial** cuando evita los detalles de UI y se centra en la intención real del usuario⁶.

En un estilo de escritura esencial, la narración se expresa al nivel de las *intenciones* de los usuarios y *responsabilidades* del sistema, en lugar de sus acciones concretas. Son independientes de los detalles acerca de la tecnología y los mecanismos, especialmente aquellos relacionados con la UI.

Escriba casos de uso en un estilo esencial; no considere la interfaz de usuario y céntrese en la intención del actor.

Todos los ejemplos de casos de uso anteriores de este capítulo, como *Procesar Venta*, se escribieron con la intención de seguir un estilo esencial.

Nótese que el diccionario define *objetivo* como sinónimo de intención [MW89], lo que ilustra la conexión entre la idea del estilo *esencial* de Constantine y el punto de vista orientado al objetivo que se puso de relieve anteriormente en este capítulo. De hecho, muchos pasos de las *intenciones* de los actores, también se pueden caracterizar como *objetivos* de subfunción.

Ejemplos de contraste

Estilo esencial

Supongamos que el caso de uso *Gestionar Usuarios* requiere identificación y autenticación. El estilo esencial, inspirado en Constantine, utiliza el formato en dos columnas. Sin embargo, se puede escribir en una columna.

Intención del Actor	Responsabilidad del Sistema
1. El Administrador se identifica	2. Autenticar la identidad
3. ...	

En el formato de una columna esto se muestra como:

...
1. El Administrador se identifica
2. El Sistema autentica la identidad
3. ...

La solución de diseño para estas intenciones y responsabilidades está muy abierta: lectores biométricos, interfaces gráficas de usuario (GUIs), etcétera.

⁶ El término procede de los “modelos esenciales” en el *Análisis de Sistemas Esenciales* [MP84].

Estilo concreto—evitar durante el trabajo de requisitos inicial

En contraste, hay un estilo de **caso de uso concreto**. En este estilo, se incluyen en el texto del caso de uso las decisiones acerca de la interfaz de usuario. El texto podría incluso mostrar instantáneas de las pantallas de las ventanas, discutir la navegación de las ventanas, la manipulación de los elementos de la GUI, etcétera. Por ejemplo:

- ...
1. El Administrador introduce su ID y contraseña en el cuadro de diálogo (ver Dibujo 3).
2. El Sistema autentica al Administrador.
3. El Sistema muestra la ventana de “edición de usuarios” (ver Dibujo 4).
4. ...

Estos casos de uso concretos podrían ser útiles para ayudar en el diseño de la GUI detallada y concreta en etapas posteriores, pero no son adecuados durante el trabajo del análisis de requisitos inicial. Durante el trabajo de requisitos inicial, “no considere la interfaz de usuario, céntrese en la intención”.

6.12. Actores

Un actor es cualquier cosa con comportamiento, incluyendo el propio sistema que se está estudiando (SuD, *System under Discussion*) cuando solicita los servicios de otros sistemas⁷. Los actores principales y de apoyo aparecerán en los pasos de acción del texto del caso de uso. Los actores no son solamente roles que juegan personas, sino también organizaciones, software y máquinas. Hay tres tipos de actores externos con relación al SuD:

- **Actor principal:** tiene objetivos de usuario que se satisfacen mediante el uso de los servicios del SuD. Por ejemplo, el cajero.
 - ¿Por qué se identifica? Para encontrar los objetivos de usuario, los cuales dirigen los casos de uso.
- **Actor de apoyo:** proporciona un servicio (por ejemplo, información) al SuD. El servicio de autorización de pago es un ejemplo. Normalmente se trata de un sistema informático, pero podría ser una organización o una persona.
 - ¿Por qué se identifica? Para clarificar las interfaces externas y los protocolos.
- **Actor pasivo:** está interesado en el comportamiento del caso de uso, pero no es principal ni de apoyo; por ejemplo, la agencia tributaria del gobierno.
 - ¿Por qué se identifica? Para asegurar que *todos* los intereses necesarios se han identificado y satisfecho. Los intereses de los actores pasivos algunas veces son sutiles o es fácil no tenerlos en cuenta, a menos que estos actores sean identificados explícitamente.

⁷ Éste fue un refinamiento y mejora para las definiciones de actores alternativas, incluyendo las de las primeras versiones de UML y el UP [Cockburn97]. Las antiguas definiciones excluían, de manera inconsistente, el SuD como actor, incluso cuando recurría a servicios de otros sistemas. Todas las entidades podrían jugar múltiples *roles*, incluyendo el SuD.

6.13. Diagramas de casos de uso

UML proporciona notación para los diagramas de casos de uso con el fin de ilustrar los nombres de los casos de uso y los actores, y las relaciones entre ellos (ver Figura 6.2).

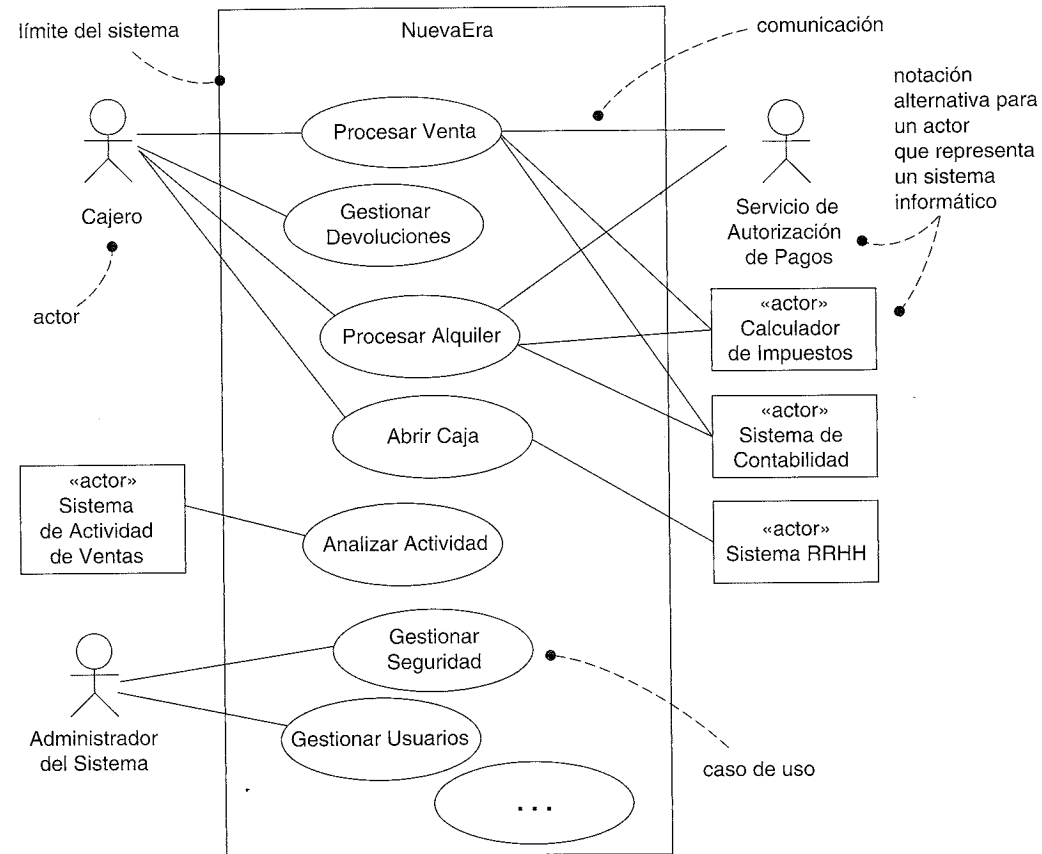


Figura 6.2. Diagrama de contexto de casos de uso parcial.

Los diagramas de caso de uso y las relaciones entre los casos de uso son secundarios en el trabajo con los casos de uso. Los casos de uso son documentos de texto. Trabajar con los casos de uso significa escribir texto.

Un signo típico de un modelador de casos de uso novato (o un estudiante) es la preocupación por los diagramas de casos de uso y las relaciones entre los casos de uso, en lugar de escribir texto. Los expertos mundiales en casos de uso, como Anderson, Fowler, Cockburn, entre otros, minimizan la importancia de los diagramas de casos de uso y las relaciones entre los casos de uso, y en lugar de eso se centran en escribir. Con eso como advertencia, un simple diagrama de casos de uso proporciona un conciso diagrama de contexto visual del sistema, que muestra los actores externos y cómo utilizan el sistema.

Sugerencia

Dibuje un diagrama de casos de uso sencillo junto con la lista actor-objetivo.

Un diagrama de casos de uso es una excelente representación del contexto del sistema; conforma un buen **diagrama de contexto**, esto es, muestra los límites de un sistema, lo que permanece fuera de él, y cómo se utiliza. Sirve como herramienta de comunicación que resume el comportamiento de un sistema y sus actores. La Figura 6.2 presenta una muestra de diagrama de contexto de casos de uso *parcial* para el sistema NuevaEra.

Sugerencias en la realización de los diagramas

La Figura 6.3 muestra algunos consejos sobre los diagramas. Nótese que la caja del actor contiene el símbolo «actor». Este símbolo se denomina **estereotipo UML**; se trata de un mecanismo para clasificar un elemento en cierto modo. El nombre de un estereotipo se escribe entre comillas francesas —signo ortográfico especial formado por un único carácter (no “<<” y “>>”) conocido sobre todo por su uso en la tipografía francesa para indicar una cita—.

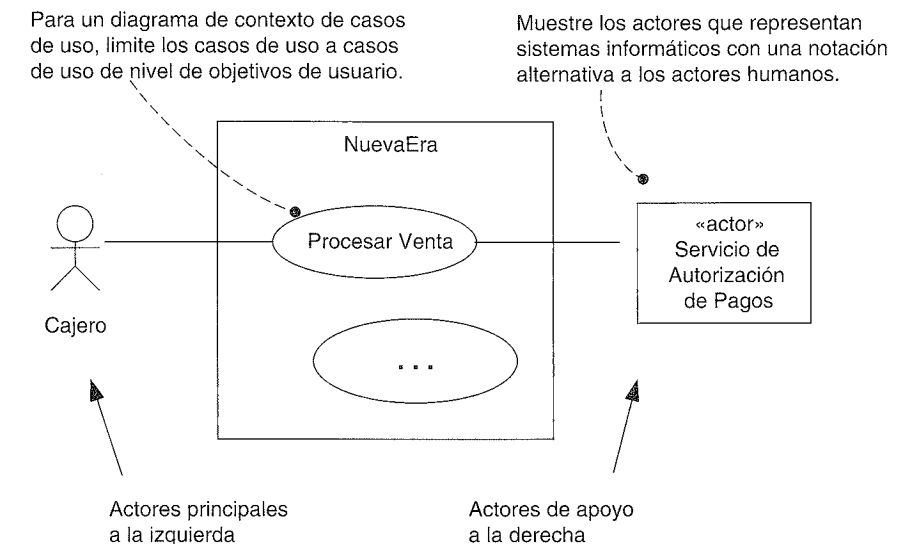


Figura 6.3. Sugerencias sobre la notación.

Para clarificar, algunos prefieren destacar los actores que se corresponden con sistemas informáticos externos con una notación alternativa, como se ilustra en la Figura 6.4.

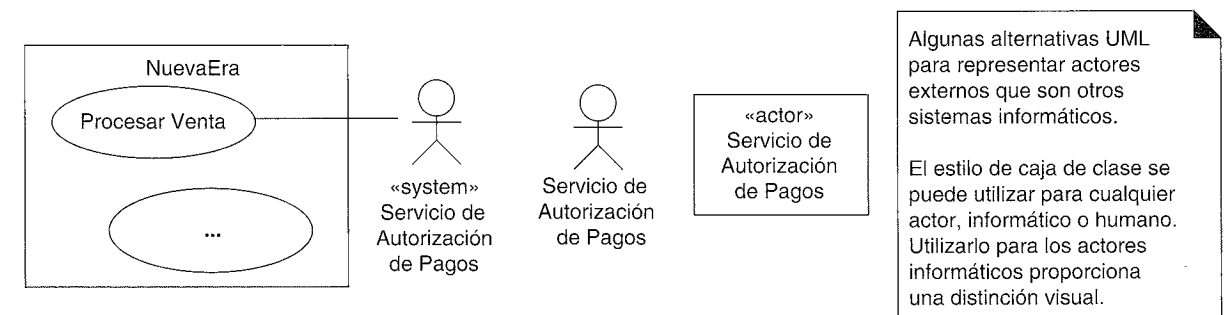


Figura 6.4. Notación alternativa para los actores.

Advertencia sobre el exceso de diagramas

Reiteramos que la importancia del trabajo de los casos de uso es escribir texto, no diagramas o centrarse en las relaciones entre los casos de uso. Si una organización dedica muchas horas (o peor, días) trabajando en los diagramas de casos de uso y debatiendo las relaciones en los casos de uso, en lugar de centrarse en escribir texto, se ha desaprovechado el esfuerzo.

6.14. Requisitos en contexto y listas de características de bajo nivel

Como queda implícito en el título del libro *Uses Cases: Requirements in Context* [GK00], una motivación clave de la idea de caso de uso es considerar y organizar los requisitos en el contexto de los objetivos y escenarios de uso de un sistema. Eso es bueno —mejora la cohesión y comprensión—. Sin embargo, los casos de uso no son los únicos artefactos de requisitos necesarios. Algunos requisitos no funcionales, reglas de dominio y contexto, y otros elementos difíciles de ubicar, se capturan mejor en la Especificación Complementaria, que se describirá en el siguiente capítulo.

Una idea detrás de los casos de uso es sustituir las listas de características de bajo nivel (típicas de los métodos de requisitos tradicionales) por los casos de uso (con algunas excepciones). Estas listas tendían a ser como sigue, normalmente agrupadas por áreas funcionales:

ID	Característica
CARAC1.9	El Sistema aceptará entradas de los identificadores de los artículos.
...	...
CARAC2.4	El Sistema registrará los pagos a crédito en el sistema de cuentas por cobrar.

De tales listas detalladas de las características de bajo nivel se puede aprovechar algo. Sin embargo, la lista completa no se recoge en media página; es más probable que ocupe docenas o cientos de páginas. Esto nos conduce a algunos obstáculos, que los casos de uso ayudan a abordar. Éstos incluyen:

- Listas de funciones largas y detalladas no relacionan los requisitos en un contexto cohesivo; las diferentes funciones y características aparecen progresivamente como una “lista de la lavandería” inconexa de elementos. En cambio, los casos de uso sitúan los requisitos en el contexto de las historias y objetivos de uso del sistema.
- Si se utilizan tanto los casos de uso como las listas de características detalladas, hay duplicación. Más trabajo, mayor volumen para escribir y leer, más problemas de consistencia y sincronización.

Sugerencia

Esfuércese en sustituir las listas detalladas de características de bajo nivel por casos de uso.

Son aceptables las listas de características de alto nivel del sistema

Es típico y útil resumir la funcionalidad del sistema con una lista breve de características de alto nivel, denominadas características del sistema, en un documento de Visión. A diferencia de las 100 páginas de características detalladas de bajo nivel, la lista de características del sistema tiende a incluir sólo unas pocas docenas de elementos. La lista proporciona un resumen muy conciso de la funcionalidad del sistema, independiente de la vista del caso de uso. Por ejemplo:

Resumen de características del sistema

- capturar las ventas.
- autorización del pago (crédito, débito, cheque).
- administración del sistema para los usuarios, seguridad, tablas de códigos y constantes, etcétera.
- procesamiento automático de ventas sin conexión, cuando fallan los componentes externos.
- transacciones en tiempo real, en base a los estándares industriales, con sistemas de terceras partes, que incluye inventario, contabilidad, recursos humanos, cálculo de impuestos y servicios de autorización de pagos.
- definición y ejecución de reglas de negocio adaptadas y que se pueden añadir en ejecución en puntos típicos, fijados en los escenarios de proceso.
- ...

Esto se estudiará en el siguiente capítulo.

¿Cuándo son apropiadas las listas de características detalladas?

Algunas veces los casos de uso no encajan realmente; algunas aplicaciones exigen un punto de vista dirigido por las características. Por ejemplo, los servidores de aplicaciones, productos de bases de datos y otros sistemas *middleware* o *back-end* necesitan ante todo ser considerados y evolucionar en términos de las *características* (“Necesitamos el soporte de XML en la siguiente versión”). Los casos de uso no se ajustan de manera natural a estas aplicaciones o al modo en que necesitan evolucionar en términos de las fuerzas del mercado.

6.15. Los casos de uso no son orientados a objetos

No hay nada orientado a objetos en los casos de uso; uno no está realizando un análisis orientado a objetos si escribe casos de uso. Esto no es un defecto sino una aclaración. De hecho, los casos de uso constituyen una herramienta para el análisis de requisitos ampliamente aplicable, que se puede utilizar en proyectos no orientados a objetos, lo cual

incrementa su utilidad como método de requisitos. Sin embargo, como estudiaremos, los casos de uso son una entrada fundamental en las actividades clásicas de A/DOO.

6.16. Casos de uso en el UP

Los casos de uso son vitales y centrales en el UP, que fomentan el **desarrollo dirigido por casos de uso**. Esto implica:

- Los requisitos se recogen principalmente en casos de uso (el Modelo de Casos de Uso); otras técnicas de requisitos (como las listas de funciones) son secundarias, si es que se utilizan.
- Los casos de uso son una parte importante de la planificación iterativa. El trabajo de una iteración se define —en parte— eligiendo algunos escenarios de caso de uso, o casos de uso completos. Los casos de uso son una entrada clave para hacer estimaciones.
- Las **realizaciones de los casos de uso** dirigen el diseño. Es decir, el equipo diseña objetos y subsistemas que colaboran para ejecutar o realizar los casos de uso.
- Los casos de uso, a menudo, influyen en la organización de los manuales de usuario.

El UP diferencia entre casos de uso del sistema y del negocio. Los **casos de uso del sistema** son los que se han estudiado en este capítulo, como *Procesar Venta*. Se crean en la disciplina Requisitos, y forman parte del Modelo de Casos de Uso.

Los **casos de uso del negocio** se escriben con menos frecuencia. Si se hace, se crean en la disciplina Modelado del Negocio, como parte de un esfuerzo de reingeniería de los procesos de negocio a gran escala, o para ayudar a entender el contexto de un nuevo sistema en el negocio. Describen una secuencia de acciones de un negocio como un todo para cumplir un objetivo de un **actor del negocio** (un actor en el entorno del negocio, como un cliente o un proveedor). Por ejemplo, en un restaurante, un caso de uso del negocio es *Servir una Comida*.

Casos de uso y especificación de requisitos a lo largo de las iteraciones

Esta sección reitera la idea clave del UP y el desarrollo iterativo: medir el tiempo y el nivel de esfuerzo de la especificación de requisitos a lo largo de las iteraciones. La Tabla 6.1 presenta una muestra (no una receta) de la estrategia del UP sobre el modo de desarrollar los requisitos.

Nótese que el equipo técnico comienza construyendo los fundamentos de la producción del sistema cuando, quizás, sólo se han detallado el 10% de los requisitos, y de hecho, se retrasa de manera deliberada la continuación del trabajo de los requisitos concertados hasta casi el final de la primera iteración de elaboración.

Ésta es la diferencia clave entre el proceso iterativo y el proceso en cascada: el desarrollo con calidad de producción de los fundamentos del sistema comienza rápidamente, mucho antes de conocer todos los requisitos.

Obsérvese que cerca del final de la primera iteración de elaboración, hay un segundo taller de requisitos, durante el que quizás, el 30% de los casos de uso se escriben en

Tabla 6.1. Muestra del esfuerzo de los requisitos a lo largo de las primeras iteraciones; no es una receta.

Disciplina	Artefacto	Comentarios y nivel de esfuerzo de los requisitos				
		Inicio 1 semana	Elab 1 4 semanas	Elab 2 4 semanas	Elab 3 3 semanas	Elab 4 3 semanas
Requisitos.	Modelo de Casos de Uso.	2 días de taller de requisitos. Se identifican por el nombre la mayoría de los casos de uso y se resumen en un párrafo breve. Sólo el 10% se escribe en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de 2 días. Se obtiene un mejor entendimiento y retroalimentación a partir del trabajo de implementación, entonces se completa el 30% de los casos de uso en detalle.	Cerca del final de esta iteración, tiene lugar un taller de requisitos de dos días. Se obtiene una mejor comprensión y retroalimentación a partir del trabajo de implementación, entonces se completa el 50% de los casos de uso en detalle.	Repetir, se completa el 70% de todos los casos de uso en detalle.	Repetir con la intención de clarificar y escribir en detalle del 80-90% de los casos de uso. Sólo una pequeña parte de éstos se construyen durante la elaboración; el resto se aborda durante la construcción.
Diseño.	Modelo de Diseño.	Nada.	Diseño de un pequeño conjunto de requisitos de alto riesgo significativos desde el punto de vista de la arquitectura.	Repetir.	Repetir.	Repetir. Deberían ahora estabilizarse los aspectos de alto riesgo significativos para la arquitectura.
Implementación.	Modelo de Implementación (código, etc.)	Nada.	Implementar esto.	Repetir. Se construye el 5% del sistema final.	Repetir. Se construye el 10% del sistema final.	Repetir. Se construye el 15% del sistema final.
Gestión del Proyecto.	Plan de Desarrollo de SW.	Estimación muy imprecisa del esfuerzo total.	La estimación comienza a tomar forma.	Un poco mejor...	Un poco mejor...	Ahora se pueden establecer racionalmente la duración global del proyecto, los hitos más importantes, estimación del coste y esfuerzo.

detalle. Este análisis de requisitos escalonado se beneficia de la retroalimentación a partir de la construcción de un poco del núcleo del software. La retroalimentación incluye la evaluación del usuario, pruebas y “conocimiento de lo que no conocemos” mejorado. Es decir, el acto de construir software rápidamente hace que surjan suposiciones y preguntas que necesitan aclararse.

Momento de la creación de los artefactos del UP

La Tabla 6.2 muestra algunos de los artefactos del UP y un ejemplo de la planificación de sus momentos de comienzo y refinamiento. El Modelo de Casos de Uso comienza en la fase de inicio, con quizás sólo el 10% de los casos de uso escritos con algo de detalle. La mayoría se escriben incrementalmente a lo largo de las iteraciones de la fase de elaboración, de manera que, al final de la elaboración se ha escrito un gran cuerpo de casos de uso detallados y otros requisitos (en la Especificación Complementaria), proporcionando una base realista para hacer una estimación precisa hasta el final del proyecto.

Tabla 6.2. Muestra de los artefactos UP y evolución temporal. c - comenzar; r – refinar.

Disciplina	Artefacto Iteración →	Inicio I1	Elab. E1...En	Const. C1...Cn	Trans. T1...T2
Modelado del Negocio	Modelo del Dominio		c		
Requisitos	Modelo de Casos de Uso	c	r		
	Visión	c	r		
	Especificación Complementaria	c	r		
	Glosario	c	r		
Diseño	Modelo de Diseño		c	r	
	Documento de Arquitectura SW		c		
	Modelo de Datos		c	r	
Implementación	Modelo de Implementación		c	r	r
Gestión del Proyecto	Plan de Desarrollo SW	c	r	r	r
Pruebas	Modelo de Pruebas		c	r	
Entorno	Marco de Desarrollo	c	r		

Casos de uso en la fase de inicio

La siguiente discusión detalla la información presentada en la Tabla 6.1.

No todos los casos de uso se escriben en formato completo durante la fase de inicio. Más bien, suponga que se lleva a cabo un taller de requisitos durante dos días al comienzo del estudio de NuevaEra. La primera parte del día se dedica a identificar los objetivos y el personal involucrado, y especular sobre lo que queda dentro y fuera del alcance del proyecto. Se escribe una tabla de casos de uso actor-objetivo y se presenta con el proyector del ordenador. Se inicia el diagrama de contexto de casos de uso. Tras unas pocas horas, quizás se identifican unos 20 objetivos de usuario (y por tanto, casos de uso de nivel de usuario), que incluye *Procesar Venta*, *Gestionar Devoluciones*, etcétera. La mayoría de los casos de uso interesantes, complejos o arriesgados, se escriben en formato breve; cada uno escrito con una duración media de dos minutos. El equipo comienza a formarse un esquema de alto nivel de la funcionalidad del sistema.

Después de esto, entre el 10% y el 20% de los casos de uso que representan las funciones complejas principales, o que son especialmente arriesgadas en alguna dimensión, se escriben en formato completo; el equipo investiga con algo más de profundidad para entender mejor la magnitud, complejidad y demonios ocultos en el proyecto, a través de una pequeña muestra de casos de uso interesantes. Quizás esto puede implicar dos casos de uso: *Procesar Venta* y *Gestionar Devoluciones*.

Se utiliza una herramienta de gestión de requisitos integrada con un procesador de texto para la escritura, y el trabajo se muestra por medio de un proyector mientras el equipo colabora en el análisis y la escritura. Se escriben las listas de *Intereses* y *Personal Involucrado* para estos casos de uso, para descubrir requisitos más refinados (y quizás costosos) funcionales y no funcionales —o cualidades del sistema— claves, como la fiabilidad y el rendimiento.

El objetivo del análisis no es completar los casos de uso de manera exhaustiva, sino dedicar unas horas a comprenderlos mejor.

El promotor del proyecto necesita decidir si merece la pena un estudio profundo (esto es, la fase de elaboración). La intención del trabajo de inicio no es hacer este estudio, sino adquirir una idea de poca fidelidad (y claramente propensa a errores) acerca del alcance, riesgo, esfuerzo, viabilidad técnica, y análisis del negocio, para decir avanzar, dónde comenzar si se hace, o si parar.

Quizás la fase de inicio del proyecto NuevaEra duró cinco días. La combinación del taller de requisitos de dos días y su análisis de casos de uso breve, y otros estudios durante la semana, condujeron a tomar la decisión de continuar con la fase de elaboración para el sistema.

Casos de uso en la elaboración

La siguiente discusión detalla la información presentada en la Tabla 6.1.

Se trata de una fase de múltiples iteraciones de duración fija (por ejemplo, cuatro iteraciones) en las cuales se construyen incrementalmente partes del sistema arriesgadas, de alto valor o significativas desde el punto de vista de la arquitectura, y se identifican y clarifican la “mayoría” de los requisitos. La retroalimentación de los pasos concretos de programación influye e informa del conocimiento de los requisitos por parte del equipo, que se refina de manera iterativa y adaptable. Quizás se aborda un taller de requisitos de dos días en cada iteración —cuatro talleres—. Sin embargo, no se estudian todos los casos de uso en cada taller. Se priorizan; los primeros talleres se centran en un subconjunto de los casos de uso más importantes.

En cada siguiente taller de requisitos breve, es el momento de adaptar y refinar la visión de los requisitos principales, que serán inestables en las primeras iteraciones y se irán estabilizando en las últimas. Por tanto, hay una interacción iterativa entre el descubrimiento de los requisitos y la construcción de partes del software.

Durante cada taller de requisitos se refinan los objetivos de usuario y la lista de casos de uso. Se escriben, y reescriben, la mayoría de los casos de uso, en formato completo. Al final de la elaboración, se escriben en detalle del “80 al 90%” de los casos de uso. Para el sistema PDV con 20 casos de uso de nivel de objetivo de usuario, 15 o más de los más complejos y arriesgados deberían investigarse, escribirse y reescribirse en formato completo.

Nótese que la elaboración conlleva programar partes del sistema. Al final de esta etapa, el equipo NuevaEra no sólo debería tener una mejor definición de los casos de uso, sino también algo de software ejecutable de calidad.

Casos de uso en la construcción

La etapa de construcción está compuesta de iteraciones de duración fija (por ejemplo, 20 iteraciones de dos semanas cada una) que se centra en completar el sistema, una vez que las principales cuestiones arriesgadas e inestables se han establecido en la elaboración. Tendrá lugar todavía la escritura de casos de uso menores y quizás talleres de requisitos,

pero mucho menos de lo que se hizo en la elaboración. En esta etapa, la mayoría de los requisitos funcionales y no funcionales principales deberían haberse estabilizado de manera iterativa y adaptable. La intención no es dar a entender que los requisitos se congelan o el estudio termina, sino que el grado de cambio es mucho menor.

6.17. Caso de estudio: casos de uso en la fase de inicio de NuevaEra

Como se ha descrito en la sección anterior, no todos los casos de uso se escriben en el formato completo durante la fase de inicio. El Modelo de Casos de Uso de esta fase para el caso de estudio podría detallarse como sigue:

Completo	Informal	Breve
Procesar Venta	Procesar Alquiler	Abrir Caja
Gestionar Devoluciones	Analizar Actividad de Ventas	Cerrar Caja
	Gestionar Seguridad	Gestionar Usuarios
	...	Poner en Marcha
		Suspender Operación
		Gestionar Tablas del Sistema
		...

6.18. Lecturas adicionales

La guía de casos de uso de mayor éxito, traducida a varios idiomas es *Writing Effective Use Cases* [Cockburn01]⁸. Por buenas razones se ha convertido en el libro de casos de uso más ampliamente leído y seguido y, por tanto, se recomienda como referencia fundamental. Este capítulo se basa, y es consistente, con su contenido. Sugerencia: no rechace el libro como consecuencia del uso de iconos para los diferentes niveles de casos de uso por parte de los autores, o el énfasis temprano en los niveles y la taxonomía de casos de uso. Los iconos son opcionales y no muy importantes. Y aunque el debate acerca de los niveles y objetivos podría parecer al principio que distrae la atención a aquellos nuevos en los casos de uso, los que han trabajado con ellos durante algún tiempo estiman que el nivel y alcance de los casos de uso son cuestiones prácticas claves, porque su incomprensión es una fuente típica de complicaciones en el modelado de casos de uso.

“Structuring Use Cases with Goals” [Cockburn97] es el artículo sobre casos de uso más citado, disponible on-line en www.usecases.org.

Use Cases: Requirements in Context [GK00] es otro texto útil. Destaca el importante punto de vista —como establece el título— de que los casos de uso no son únicamente

⁸ Nótese que Cockburn rima con *slow burn* (N. del T.: En una comunicación personal, el autor nos ha comentado que introdujo esta aclaración a petición de A. Cockburn para señalar que su apellido no se pronuncia como cock-burn y evitar chistes por la connotación sexual.)

otro artefacto de los requisitos, sino que constituyen el vehículo central que dirige el trabajo de los requisitos y la información.

Otra lectura que merece la pena destacar es *Applying Use Cases: A Practical Guide* [SW98], escrita por un profesor y experto en casos de uso que entiende y comunica cómo aplicar los casos de uso en un ciclo de vida iterativo.

6.19. Artefactos UP y contexto del proceso

Como se ilustra en la Figura 6.5 los casos de uso influyen en muchos artefactos UP.

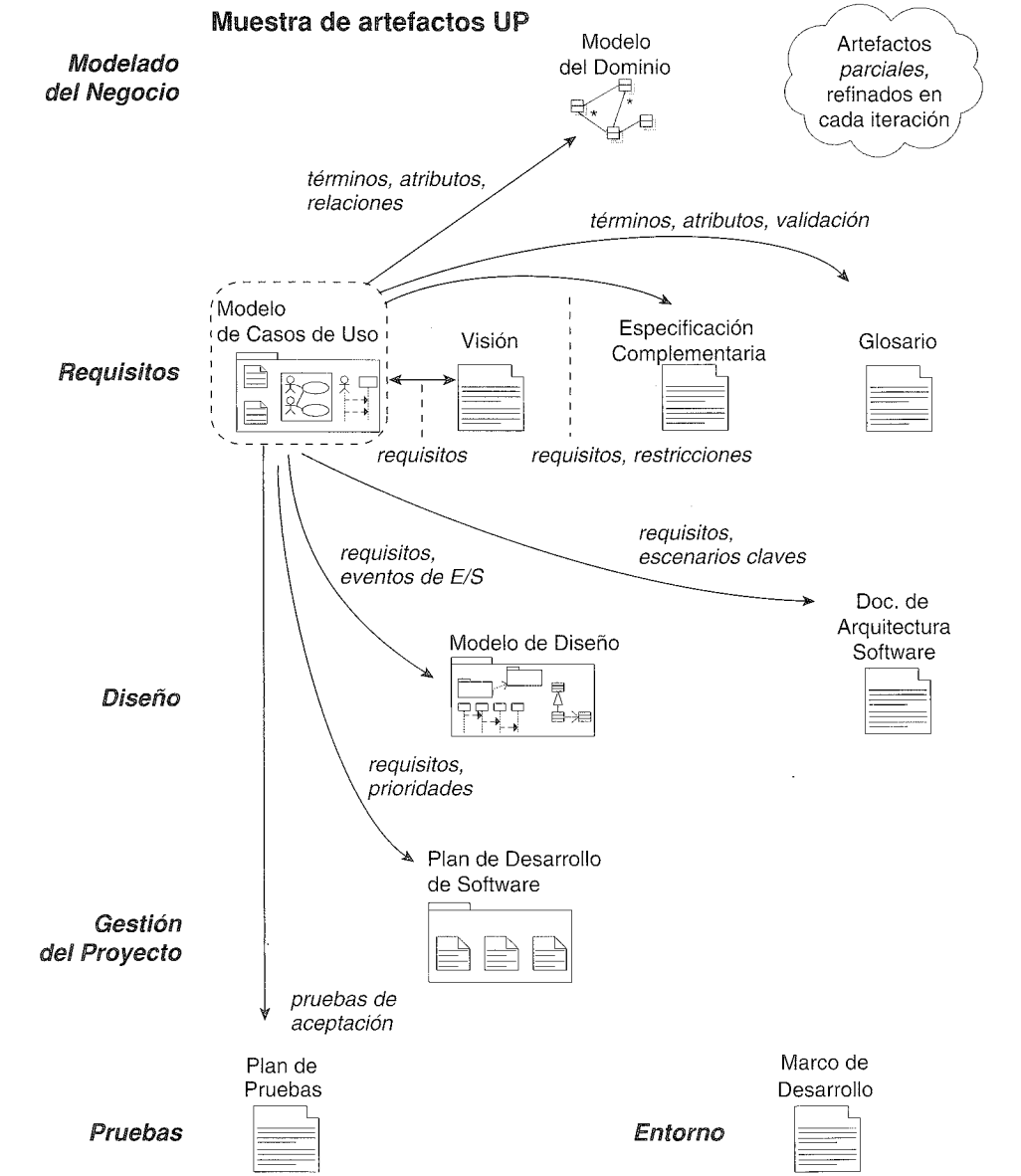


Figura 6.5. Muestra de la influencia entre los artefactos UP.

En el UP, el trabajo de los casos de uso es una actividad de la disciplina de requisitos que podría inicializarse durante un taller de requisitos. La Figura 6.6 presenta algunos consejos acerca del momento y el lugar para llevar a cabo este trabajo.

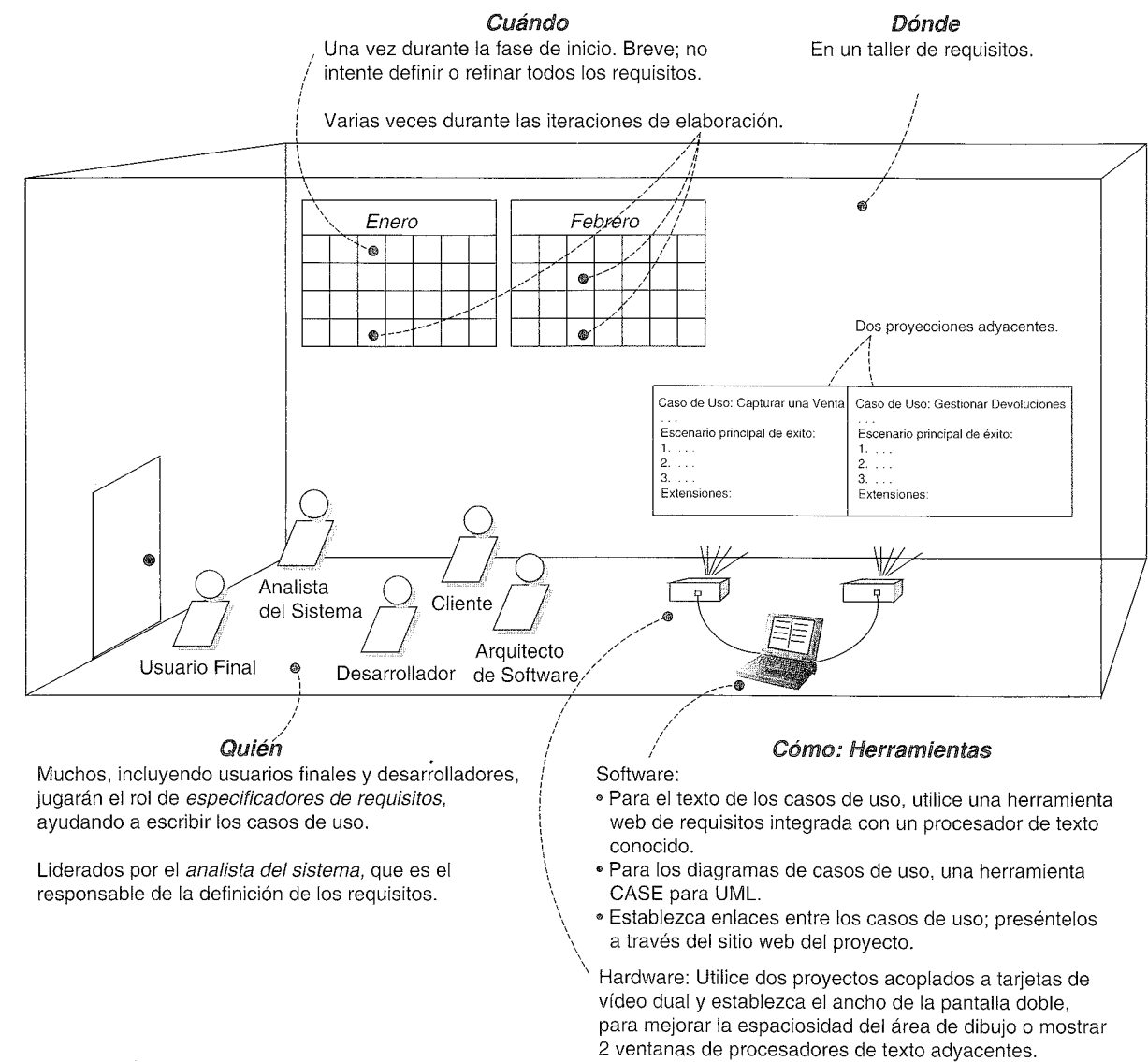


Figura 6.6. Proceso y establecimiento del contexto.

IDENTIFICACIÓN DE OTROS REQUISITOS

Cuando las ideas fallan, las palabras vienen muy bien.
Johann Wolfgang von Goethe

Objetivos

- Escribir una Especificación Complementaria, Glosario y Visión.
- Comparar y contrastar las características del sistema con los casos de uso.
- Relacionar la Visión con otros artefactos y con el desarrollo iterativo.
- Definir los atributos de calidad.

Introducción

No es suficiente escribir casos de uso. Existen otros tipos de requisitos que son necesarios identificar, como los relacionados con documentación, empaquetado, soporte, licencia, etcétera. Éstos se recogen en la **Especificación Complementaria**.

El **Glosario** almacena los términos y definiciones; puede también jugar el rol de diccionario de datos.

La **Visión** resume la “visión” del proyecto. Sirve para comunicar de manera concisa las grandes ideas acerca de por qué se propuso el proyecto, cuáles son los problemas, quiénes son las personas involucradas, qué necesitan, y cuál podría ser la apariencia de la solución propuesta.

Citando literalmente:

La Visión define la vista que tienen las personas involucradas del producto que se va a desarrollar, especificada en términos de las necesidades y características claves de dichas personas. Al contener un esquema de los principales requisitos previstos, proporciona la base contractual para los requisitos técnicos más detallados [RUP].

7.1. Ejemplos del PDV NuevaEra

El objetivo de los siguientes ejemplos no es presentar de manera exhaustiva la Visión, el Glosario y Especificación Complementaria, ya que algunas de las secciones —aunque útiles para el proyecto— no son relevantes para los objetivos de aprendizaje¹. La finalidad del libro son las principales técnicas en el diseño de objetos, análisis de requisitos con casos de uso, análisis orientado a objetos, no los problemas del PDV o las sentencias de Visión. Por tanto, sólo se hace referencia brevemente a algunas secciones, para establecer las conexiones entre el trabajo previo y futuro, destacar las cuestiones que merecen la pena, proporcionar una idea del contenido, y avanzar rápidamente.

7.2. Ejemplo NuevaEra: Especificación Complementaria (Parcial)

Especificación Complementaria			
Historia de revisiones			
Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

Introducción

Este documento es el repositorio de todos los requisitos del PDV NuevaEra que no se capturan en los casos de uso.

Funcionalidad

(Funcionalidad común entre muchos casos de uso)

Registro y gestión de errores

Registrar todos los errores en almacenamiento persistente.

¹ El crecimiento gradual del alcance no es sólo un problema de los requisitos, sino de la *escritura* sobre requisitos.

Reglas de negocio conectables

En varios puntos de los escenarios de varios casos de uso (pendientes de ser definidos) soportar la capacidad de adaptar la funcionalidad del sistema con un conjunto arbitrario de reglas que se ejecutan en ese punto o evento.

Seguridad

Todo uso requiere la autenticación de los usuarios.

Facilidad de uso

Factores humanos

El cliente será capaz de ver la información en un gran monitor del PDV. Por tanto:

- Se debe ver el texto fácilmente a una distancia de 1 metro.
- Evitar colores asociados con formas comunes de daltonismo.

Velocidad, comodidad y procesamiento libre de errores, son lo más importante en el procesamiento de ventas, ya que el comprador desea irse rápidamente, o perciben la experiencia de compra (y al vendedor) como menos positiva.

El cajero está mirando a menudo al cliente o los artículos, no a la pantalla del ordenador. Por tanto, se deben comunicar las señales y avisos con sonidos, en lugar de sólo mediante gráficos.

Fiabilidad

Capacidad de recuperación

Si se produce algún fallo al usar un servicio externo (autorización de pago, sistema de contabilidad,...) intentar solucionarlo con una solución local (ej. almacenar y remitir) para, no obstante, completar un venta. Se necesita mucho más análisis aquí...

Rendimiento

Como se mencionó en los factores humanos, los compradores quieren completar el proceso de ventas *muy* rápido. Un cuello de botella potencial es la autorización de pagos externa. El objetivo es conseguir la autorización en menos de 1 minuto, el 90% de las veces.

Soporte

Adaptabilidad

Los diferentes clientes del PDV NuevaEra tienen necesidades de procesamiento y reglas de negocio únicas en el procesamiento de una vena. Por tanto, en varios puntos definidos en el escenario (por ejemplo, cuando se inicia una nueva venta, cuando se añade una nueva línea de venta) se habilitarán reglas de negocio conectables.

Facilidad para cambiar la configuración

Clientes diferentes desean que varíe la configuración de la red para sus sistemas PDV, como clientes gruesos *versus* delgados, en dos capas *versus* N-capas, etcétera. Además, desean poder cambiar estas configuraciones, para reflejar sus cambios en el negocio y necesidades de rendimiento. Por tanto, el sistema será configurable para reflejar estas necesidades. Se necesita mucho más análisis en este área para descubrir las áreas y grado de flexibilidad, y el esfuerzo para conseguirla.

Restricciones de Implementación

La dirección de NuevaEra insiste en una solución utilizando las tecnologías Java, previendo que esto mejorará a largo plazo la portabilidad y soporte, además de facilitar el desarrollo.

Componentes adquiridos

- El sistema de cálculo de impuestos. Debe soportar sistemas de cálculo conectables de diferentes países.

Componentes de libre distribución

En general, recomendamos maximizar el uso de componentes Java de libre distribución en este proyecto.

Aunque es prematuro diseñar y elegir los componentes de manera definitiva, sugerimos los siguientes como candidatos posibles:

- Framework para registros JLog
- ...

Interfaces

Interfaces y hardware destacables

- Monitor con pantalla táctil (detectado por el sistema operativo como monitor corriente, y los movimientos de contacto como eventos del ratón)
- Escáner láser de código de barras (normalmente unido a un teclado especial, y el software considera las entradas escaneadas como entradas del teclado)
- Impresora de recibos
- Lector de tarjetas de crédito/débito
- Lector de firmas (pero no en la primera versión)

Interfaces software

Para la mayoría de los sistema de colaboración externos (calculador de impuestos, contabilidad, inventario, ...) necesitamos ser capaces de conectar diversos sistemas y, por tanto, diversas interfaces.

Reglas de dominio (negocio)

ID	Regla	Grado de variación	Fuente
REGLA1	Se requiere la firma para pagos a crédito. años la mayoría de los clientes	Se continuará solicitando la "firma" de los compradores, aunque en 2 años los clientes solicitarán la firma en un aparato de captura digital, y en 5 años esperamos que se demande la nueva "firma" digital única soportada por la ley americana.	La política de prácticamente todas las compañías de autorización de crédito.
REGLA2	Reglas sobre los impuestos. Hay que añadir un impuesto	Alto. La ley sobre impuestos cambia anualmente, en todos los niveles de gobierno.	ley

continúa

ID	Regla	Grado de variación	Fuente
	a las ventas. Ver los estatutos del gobierno para los detalles actuales.		
REGLA3	Las devoluciones de los pagos a crédito sólo pueden efectuarse como crédito en las cuentas de crédito de los compradores, no en efectivo.	Bajo.	Política de la compañía sobre la autorización de crédito.
REGLA4	Reglas de descuento al comprador. Ejemplo: Empleado: 20% Clientes preferentes: 10% Antiguos: 15%	Alto. Cada tienda utiliza reglas diferentes.	Política de la tienda.
REGLA5	Reglas de descuento de venta (nivel de transacción) Se aplica al total antes de los impuestos. Por ejemplo: 10% de descuento si el total es mayor de 100 €. 5% de descuento los lunes. 10% de descuento en las ventas de hoy entre las 10am y las 3pm. 50% de descuento en Tofu hoy desde las 9am hasta las 10am.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.
REGLA6	Reglas de descuento de artículo (nivel de línea de venta). Ejemplo: 10% de descuento en tractores esta semana. Comprando 2 hamburguesas vegetales llévase 1 gratis.	Alto. Cada tienda utiliza reglas distintas, y pueden cambiar diariamente o cada hora.	Política de la tienda.

Cuestiones legales

Recomendamos algunos componentes de libre distribución si sus restricciones de licencia se pueden resolver para permitir la reventa de artículos que incluyen software de libre distribución. Todas las reglas de impuestos se tienen que aplicar, por ley, durante las ventas. Nótese que pueden cambiar con frecuencia.

Información en dominios de interés

Fijación de precios

Además de las reglas de fijación de precios que se describen en la sección de reglas del dominio, observe que los artículos tienen un *precio original*, y opcionalmente, un *precio rebajado*. El precio de los artículos (antes de los descuentos adicionales) es el precio rebajado, si existe. Las organizaciones mantienen el precio original incluso si hay un precio rebajado, por razones de contabilidad e impuestos.

Gestión de pagos a crédito y débito

Cuando el servicio de autorización de pagos aprueba un pago electrónico, de crédito o débito, son los responsables del pago al vendedor, no el comprador. En consecuencia, por cada pago, el vendedor necesita registrar la suma de dinero que le deben en las cuentas, desde el servicio de autorización. Normalmente, por las noches, el servicio de autorización llevará a cabo una transferencia electrónica de fondos a la cuenta de los vendedores por la suma de dinero total del día, menos un (pequeño) cargo por transacción que cobra el servicio.

Impuesto de ventas

Los cálculos de los impuestos de las ventas pueden ser muy complejos, y pueden cambiar regularmente como respuesta a la legislación en todos los niveles de gobierno. Por tanto, es aconsejable delegar el cálculo a software de terceras partes (de los que hay varios disponibles). Los impuestos se pueden deber a la ciudad, al gobierno regional y agencias nacionales. Algunos artículos podrían estar exentos de impuestos sin requisitos, o exentos dependiendo del comprador o el receptor objetivo (por ejemplo, un granjero o un niño).

Identificadores de artículos: UPCs, EANs, SKUs, código de barras y lectores de códigos de barras

El PDV NuevaEra necesita dar soporte a varios esquemas de identificador de artículos. UPCs (Códigos de Producto Universales, *Universal Product Codes*), EANs (Numeración de Artículos Europea, *European Article Numbering*) y SKUs (Unidades de Mantenimiento de Stock, *Stock Keeping Units*) son tres tipos comunes de sistemas de identificación para los artículos que se venden. El Número de Artículo Japonés (JANs, *Japanese Article Number*) es un tipo de la versión EAN.

SKUs son identificadores completamente arbitrarios definidos por el vendedor.

Sin embargo, los UPCs y EANs tienen un componente estándar y normativo. Diríjase a www.adams1.com/pub/russadam/upccode.html para obtener una buena visión general. También puede ver www.uc-council.org y www.ean-int.org.

7.3. Comentario: Especificación Complementaria

La Especificación Complementaria captura otros requisitos, información y restricciones que no se recogen fácilmente en los casos de uso o el Glosario, que comprende los atributos o requisitos de calidad “URPS+” de todo el sistema. Fíjese que los requisitos específicos de un caso de uso pueden (y probablemente deben) escribirse en primer lugar con el caso de uso, en una sección *Requisitos Especiales*, pero algunos prefieren también reunirlos en la Especificación Complementaria. Los elementos de la Especificación Complementaria podrían comprender:

- Requisitos FURPS+ —funcionalidad, facilidad de uso, fiabilidad, rendimiento y soporte—.
- Informes.
- Restricciones de software y hardware (sistemas operativos y de red...).
- Restricciones de desarrollo (por ejemplo, herramientas de proceso y desarrollo).
- Otras restricciones de diseño e implementación.

- Cuestiones de internacionalización (unidades, idiomas...).
- Documentación (usuario, instalación, administración) y ayuda.
- Licencia y otras cuestiones legales.
- Empaquetado.
- Estándares (técnico, seguridad, calidad).
- Cuestiones del entorno físico (por ejemplo, calor o vibración).
- Cuestiones operacionales (por ejemplo, ¿cómo se gestionan los errores o con qué frecuencia se hacen las copias de seguridad?).
- Reglas del dominio o negocio.
- Información en los dominios de interés (por ejemplo, ¿cuál es el ciclo completo de gestión de pagos a crédito?).

Las **restricciones** no son comportamientos, sino otro tipo de restricciones del diseño o el proyecto. También son requisitos, pero se denominan comúnmente “restricciones” para remarcar su influencia *restrictiva*. Por ejemplo:

Debe utilizar Oracle (tenemos un contrato de licencia con ellos)

Debe funcionar bajo Linux (disminuirá el coste)

Sugerencia

Las decisiones y restricciones de diseño tempranas (“elaboración prematura”) son casi siempre una mala idea, así que merece la pena desconfiar y cuestionarlas, especialmente durante la fase de inicio, cuando se ha analizado muy poco con cuidado. Algunas restricciones se imponen por causas inevitables, como restricciones legales, o una interfaz de un sistema externo existente al que se debe invocar.

Atributos de calidad

Algunos requisitos se denominan **atributos de calidad** [BCK98] (o “-ilities”) de un sistema. Éstos incluyen facilidad de uso (*usability*), fiabilidad (*reliability*), etcétera. Nótese que se refieren a cualidades del sistema, no a que estos requisitos sean necesariamente de calidad alta (la palabra está sobrecargada en inglés). Por ejemplo, la cualidad de soporte (*supportability*) podría elegirse deliberadamente que fuese baja si el producto no se pretende que sirva a largo plazo.

Hay de dos tipos:

1. Observables en la ejecución (funcionalidad, facilidad de uso, fiabilidad, rendimiento, etc.).
2. No observables en ejecución (soporte, pruebas, etc.).

La funcionalidad se especifica en los casos de uso, como otros atributos de calidad relacionados con casos de uso específicos (por ejemplo, las cualidades de rendimiento en el caso de uso *Procesar Venta*).

Otros atributos de calidad FURPS+ del sistema se describen en la Especificación Complementaria.

Aunque la funcionalidad es un atributo de calidad válido, en su uso común, el término “atributo de calidad” casi siempre se refiere a “cualidades del sistema que no sean la funcionalidad”. En este libro, el término se utiliza de este modo. Esto no es exactamente lo mismo que los requisitos no funcionales, que es un término más amplio que incluye *todo* excepto la funcionalidad (por ejemplo, empaquetado y licencia).

Cuando nos ponemos nuestro “gorro de arquitecto”, los atributos de calidad de todo el sistema (y por tanto, la Especificación Complementaria donde se recogen) son especialmente interesantes porque —como veremos en el Capítulo 32— el análisis y el diseño de la arquitectura tienen que ver en gran medida con la identificación y resolución de los atributos de calidad en el contexto de los requisitos funcionales. Por ejemplo, suponga que uno de los atributos de calidad es que el sistema NuevaEra debe ser bastante tolerante a fallos cuando fallen los servicios remotos. Desde el punto de vista de la arquitectura, esto tendrá una influencia global en las decisiones de diseño a gran escala.

Entre los atributos de calidad existen interdependencias, lo que implica compromisos. Un ejemplo sencillo en el PDV podría ser, “muy fiable (tolerante a fallos)” y “fácil de probar” son opuestos, puesto que hay muchas formas sutiles de que pueda fallar un sistema distribuido.

Reglas del dominio (negocio)

Las reglas del dominio [Ross97, GK00] dictan el modo en el que podría operar un dominio o negocio. No son requisitos de ninguna aplicación, aunque, a menudo, los requisitos de una aplicación se ven afectados por las reglas del dominio. Las políticas de la compañía, leyes físicas y leyes gubernamentales, son reglas de dominio típicas.

Se denominan comúnmente **reglas del negocio**, que son el tipo más común, pero ese término está limitado, ya que existen aplicaciones software que no son de gestión de un negocio, como simulación del clima o logística militar. Una simulación del clima incluye “reglas del dominio”, relacionadas con las leyes y relaciones físicas, que afectan a los requisitos de la aplicación.

Con frecuencia, resulta útil identificar y registrar aquellas reglas del dominio que afectan a los requisitos, normalmente materializados en los casos de uso, porque pueden clarificar el contenido de un caso de uso ambiguo o incompleto. Por ejemplo, en el PDV NuevaEra, si alguien pregunta si en el caso de uso *Procesar Venta* debería escribirse una alternativa a los pagos a crédito que no necesite la captura de la firma, existe una regla del negocio (REGLA1) que aclara que no se permitirá por ninguna compañía de autorización de crédito.

Advertencia

Las reglas no son requisitos de la aplicación. No registre las características del sistema como reglas. Las reglas describen las restricciones y comportamientos del modo de trabajar del dominio, no de la aplicación.

Información en los dominios de interés

A menudo, a los expertos del dominio que se está estudiando, les resulta útil escribir (o proporcionar URLs con) explicaciones de dominios relacionados con el nuevo sistema software (ventas y contabilidad, la geofísica de los flujos subterráneos de petróleo/agua/gas,...), para presentar el contexto y ayudar a la comprensión por parte del equipo de desarrollo. Podría contener referencias importantes a expertos o a literatura, fórmulas, leyes u otras referencias. Por ejemplo, los misterios de los esquemas de codificación UPC y EAN, y la interpretación del código de barras, los deben entender, en cierta medida, el equipo de NuevaEra.

7.4. Ejemplo NuevaEra: Visión (Parcial)

Visión			
Historia de revisiones			
Versión	Fecha	Descripción	Autor
Borrador de Inicio	10 Enero, 2031	Primer borrador. Para refinarse principalmente durante la elaboración.	Craig Larman

El análisis del ejemplo es ilustrativo, pero ficticio

Introducción

Prevemos una aplicación de punto de venta (PDV) tolerante a fallos de próxima generación, PDV NuevaEra, con flexibilidad para poder soportar variación en las reglas del negocio del cliente, múltiples mecanismos de terminal e interfaz de usuario, y la integración con múltiples sistemas de terceras partes.

Orientación

Oportunidad del negocio

Los productos PDV existentes no son adaptables al negocio del cliente, en términos de permitir variar las reglas de negocio y los diseños de la red (por ejemplo cliente delgado o no, arquitectura en 2, 3 ó 4 capas). Además, no permiten su extensión de manera adecuada cuando se incrementan los terminales y crece el negocio. Y ninguno permite trabajar en línea o desconectados, adaptándose dinámicamente dependiendo de los fallos. Ninguno se integra fácilmente con muchos sistemas de terceras partes. Ninguno admite nuevas tecnologías como los PDAs móviles. El mercado se siente insatisfecho debido a este estado inflexible de cosas, y demandan un PDV que rectifique esta situación.

Enunciado del problema

Los sistemas PDV tradicionales son inflexibles, intolerantes a fallos, y difíciles de integrar con sistemas de terceras partes. Esto da lugar a problemas en el oportuno procesamiento de las ventas, en el establecimiento de procesos mejorados que no concuerdan con el software, y con los datos de contabilidad e inventario precisos y oportunos para dar soporte a la planificación y medidas, entre otras cuestiones. Esto afecta a los cajeros, encargados del almacén, administradores del sistema y a la gestión empresarial.

Enunciado de la posición en el mercado del producto

— Resumen conciso de a quién está dirigido el producto, las características relevantes, y qué lo diferencia de la competencia.

Alternativas y competencia...

Descripción del personal involucrado

Demografía de mercado...

Resumen del personal involucrado (No usuarios)...

Resumen de Usuarios...

Objetivos de alto nivel y problemas claves del personal involucrado

El taller de requisitos de un día con los expertos en la materia que se está estudiando y otras personas involucradas, y encuestas a varios distribuidores, nos llevaron a los siguientes objetivos y problemas claves:

Objetivo de alto nivel	Prioridad	Problemas e inquietudes	Soluciones actuales
Procesamiento de ventas rápido, robusto e integrado.	Alta	Se reduce la velocidad cuando se incrementa la carga. Pérdida de la capacidad de procesamiento de las ventas si los componentes fallan. Carencia de información actualizada y precisa de la contabilidad y otros sistemas debido a que no está integrado con los sistemas de contabilidad, inventario y RRHH. Da lugar a dificultades en las medidas y la planificación. Imposibilidad de adaptar las reglas de negocio a requisitos del negocio únicos. Dificultad al añadir nuevos tipos de terminales o interfaces de usuario (por ejemplo, PDAs móviles).	Los productos PDV existentes permiten el procesamiento básico de ventas, pero no abordan estos problemas.
...	...	...	...

Objetivos de nivel de usuario

Los usuarios (y los sistemas externos) necesitan un sistema para satisfacer sus objetivos:

- *Cajero*: procesar las ventas, gestionar las devoluciones, abrir y cerrar caja.
- *Administrador del sistema*: gestionar los usuarios, gestionar la seguridad, gestionar las tablas del sistema.
- *Director*: poner en marcha, suspender operación.
- *Sistema de actividad de ventas*: analizar los datos de las ventas.
- ...

Entender quiénes son los participantes y sus problemas

Reunir las entradas de la Lista Actor-Objetivos y la sección de los casos de uso de Intereses del personal involucrado

Podría ser la Lista Actor-Objetivo creada durante el modelado de casos de uso o un resumen más conciso

Entorno de usuario...

Visión general del producto

Perspectiva del producto

El PDV NuevaEra residirá, normalmente, en tiendas; si se utilizan terminales móviles se encontrarán muy próximos a la red de la tienda, en el interior o en el exterior. Proporcionará servicios al usuario, y colaborará con otros sistemas, como se indica en la Figura Visión-1.

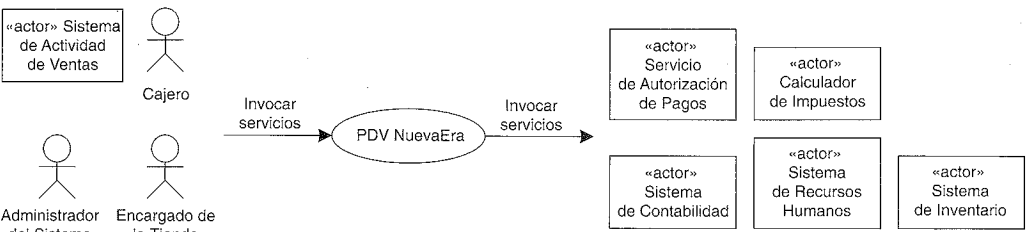


Figura Visión-1. Diagrama de contexto del sistema PDV NuevaEra.

Resumido a partir del diagrama de casos de uso

Los diagramas aparecen con diversos detalles, pero todos muestran los actores externos importantes para el sistema

Similar a la lista Actor-Objetivo, esta tabla relaciona objetivos, beneficios y soluciones, pero a un nivel más alto, no únicamente relacionado con los casos de uso

Resume el valor y características diferenciadoras del producto

Como se presenta abajo, las características del sistema constituyen un formato conciso para resumir la funcionalidad

Resumen de los beneficios

Característica soportada	Beneficio del personal involucrado
Funcionalmente, el sistema proporcionará todos los servicios típicos que requiere la organización de las ventas, incluyendo la entrada de las ventas, autorización de pagos, gestión de devoluciones, etcétera.	Servicios de punto de venta rápidos y automáticos.
Detección de fallos automática, cambiando a procesamiento local sin conexión cuando los servicios no estén disponibles.	Procesamiento de ventas continuado cuando fallan los componentes externos.
Reglas de negocio "conectables" en varios puntos del escenario durante el procesamiento de las ventas.	Configuración flexible de la lógica del negocio.
Transacciones en tiempo real con sistemas de terceras partes, haciendo uso de protocolos estándares industriales.	Información de ventas, contabilidad e inventario oportuna y precisa, para abordar las mediciones y la planificación.
...	...

Suposiciones y dependencias...

Coste y fijación del precio...

Licencia e instalación...

Resumen de las características del sistema

- Entrada de ventas.
- Autorización de pagos (crédito, débito, cheque).